



Machine Learning-Driven Combinatorial Optimization: A Systematic Review

Xinwei Wang¹ · Xinyong Yu² · Zhaopan Wang² · Zeying Si² · Guangyu Wu² · Xichao Su³ · Shaohui Qu⁴ · Biao Xiong⁴ · Haijun Peng¹ · Xin Li² · Lei Wang²

Received: 8 April 2026 / Revised: 19 May 2026 / Accepted: 2 June 2026

© The Author(s) under exclusive licence to International Center for Numerical Methods in Engineering (CIMNE) 2026

Abstract

This survey focuses on routing and scheduling, two cornerstone Combinatorial optimization problems (COPs) domains in logistics and manufacturing that are increasingly required to operate under large-scale and dynamic constraints. Traditional methods often struggling with scalability and dynamic constraints. The integration of machine learning (ML) has introduced powerful new solvers. This survey comprehensively reviews ML advances for COPs from 2020 to 2026, focusing on four key paradigms: supervised learning, reinforcement learning, unsupervised learning, and large language models (LLMs). We categorize methods into three frameworks: end-to-end neural solvers that directly generate solutions, hybrid solvers that use ML to guide classical optimization algorithms, and generative approaches that model solution distributions. Each paradigm is analyzed in terms of model architectures, integration strategies, and applications to routing, scheduling, and other COPs. We also examine the emerging role of LLMs as optimizers, programmers, and collaborative tools. To support rigorous cross-paradigm comparison, we establish a unified comparative framework that evaluates methods along seven dimensions—solution quality, scalability, generalization, computational efficiency, data independence, feasibility assurance, and theoretical guarantees—and provide coarse ratings for representative approaches across all paradigms, including LLM-based methods. Building on this analysis, the paper identifies core challenges such as generalization across scales and distributions, the efficiency–quality trade-off, theoretical guarantees, data dependency, dynamic constraint handling, feasibility assurance for generative models, and robustness under non-stationary environments, and outlines future directions for building more robust and practical ML-driven optimization systems.

Xinwei Wang and Xinyong Yu contributed equally to this work.

✉ Xin Li
lixin31@dlut.edu.cn

✉ Lei Wang
wanglei@dlut.edu.cn

Xinwei Wang
wangxinwei@dlut.edu.cn

Xinyong Yu
xinyong_yu@mail.dlut.edu.cn

Zhaopan Wang
wangzhaopan@mail.dlut.edu.cn

Zeying Si
sizeying@mail.dlut.edu.cn

Guangyu Wu
wuguangyu@mail.dlut.edu.cn

Xichao Su
suxich@126.com

Shaohui Qu
269886049@qq.com

Biao Xiong
xiongb001@126.com

Haijun Peng
hjpeng@dlut.edu.cn

¹ Department of Engineering Mechanics, State Key Laboratory of Structural Analysis, Optimization and CAE Software for Industrial Equipment, Dalian University of Technology, Dalian 116024, China

² School of Mathematical Sciences, Dalian University of Technology, Dalian 116024, China

³ Naval Aviation University, Yantai 264001, China

⁴ PLA Joint Logistics Support Force University of Engineering, Chongqing 401331, China

Abbreviations

A2C	Advantage Actor-Critic	OR	Operations Research
ABC	Artificial Bee Colony	PDTSP	Pickup and Delivery Traveling Salesman Problem
AC	Actor-Critic	PDR	Priority Dispatching Rules
AGV	Automated Guided Vehicle	PCTSP	Prize-Collecting Traveling Salesman Problem
AI	Artificial Intelligence	PIN	Permutation-Invariant Network
ATSP	Asymmetric Traveling Salesman Problem	PPO	Proximal Policy Optimization
CVRP	Capacitated Vehicle Routing Problem	PSO	Particle Swarm Optimization
CNN	Convolutional Neural Network	PtrNet	Pointer Network
COP	Combinatorial Optimization Problem	RHO	Rolling Horizon Optimization
CP	Constraint Programming	RL	Reinforcement Learning
CTSP	Capacitated Traveling Salesman Problem	RNN	Recurrent Neural Network
D3QN	Double Duel DQN	RP	Routing Problem
DE	Differential Evolution	S2S-AC	Sequence-to-Sequence Advantage Actor-Critic
DL	Deep Learning	SAC	Soft Actor-Critic
DNN	Deep Neural Network	SA	Simulated Annealing
DQN	Deep Q Network	SDVRP	Split Delivery Vehicle Routing Problem
DRL	Deep Reinforcement Learning	SDP	Semidefinite Programming
DGE	Disjunctive Graph Embedding	SEC	Subtour Elimination Constraint
DTSP	Dynamic Traveling Salesman Problem	SL	Supervised Learning
DVRP	Dynamic Vehicle Routing Problem	SP	Scheduling Problem
DyFJSP	Dynamic Flexible Job Shop Scheduling Problem	SPT	Shortest Processing Time Rule
DyJSP	Dynamic Job Shop Scheduling Problem	STSP	Symmetric Traveling Salesman Problem
FJSP	Flexible Job Shop Scheduling Problem	StTSP	Stochastic Traveling Salesman Problem
GA	Genetic Algorithm	TSP	Traveling Salesman Problem
GAN	Generative Adversarial Network	TSP-O	TSP with Obstacles
GAT	Graph Attention Network	TSPTW	Traveling Salesman Problem with Time Windows
GCN	Graph Convolutional Neural Network	TS	Tabu Search
GIN	Graph Isomorphism Network	UL	Unsupervised Learning
GNN	Graph Neural Network	VAE	Variational Autoencoder
GP	Graph Pointer Network	VRP	Vehicle Routing Problem
GTN	Graph Transformer Network	VRPTW	Vehicle Routing Problem with Time Windows
HFSP	Hybrid Flexible Job Shop Scheduling Problem	WOA	Whale Optimization Algorithm
HGAT	Heterogeneous Graph Attention Network		
ILP	Integer Linear Programming		
JSSP	Job Shop Scheduling Problem		
L2D	Learning to Dispatch		
LKH	Lin-Kernighan-Helsgaun		
LLH	Low-Level Heuristic		
LLM	Large Language Model		
LP	Linear Programming		
LSTM	Long Short-Term Memory Network		
MAJSP	Multi-Agent Job Shop Scheduling		
MDP	Markov Decision Process		
MIP	Mixed Integer Programming		
MIS	Maximum Independent Set		
ML	Machine Learning		
MPGN	Multi-Pointer Graph Network		
m-TSP	Multiple Traveling Salesman Problem		
NN	Neural Network		
OP	Orienteering Problem		

1 Introduction**1.1 The Importance and Challenges of Combinatorial Optimization Problems**

Combinatorial optimization problems (COPs) constitute an important branch of mathematical optimization. They aim to identify optimal solutions (e.g., minimum cost or maximum profit) from a finite discrete decision space subject to given constraints. Although such problems can in principle be solved by exhaustive enumeration, the number of feasible solutions grows exponentially with the size of the problem; consequently, the vast majority of practically relevant COPs are NP-hard and cannot be solved to optimality in polynomial time [1, 2]. Among the broad COPs landscape, this survey places routing and scheduling at the center, as they are

both ubiquitous in real-world systems and representative of the key challenges (scale, stochasticity, and dynamic constraints) that drive the need for ML-based solvers.

Despite their computational complexity, COPs play an indispensable role in many critical domains, including national defense, manufacturing, logistics and transportation, communication networks, and managerial decision-making [3–9]. In particular, routing problems (RPs) such as the traveling salesman problem (TSP) and the vehicle routing problem (VRP) aim to plan optimal transport or visitation routes and lie at the core of logistics and supply chain management. Scheduling problems (SPs), such as the job shop scheduling problem (JSSP) and the flexible job shop scheduling problem (FJSP), concern the efficient allocation of production resources and tasks and directly impact the performance of manufacturing systems. In addition, the knapsack problem, the graph coloring problem, the facility location problem, and other related problems are also widely applied across various production and manufacturing domains. [10–14]. Research on COPs is therefore of crucial importance for improving the efficiency and intelligence of socioeconomic activities.

1.2 Traditional Solution Methods and Their Limitations

To tackle the computational challenges of COPs, researchers have developed two major classes of solution methods, namely exact algorithms and heuristic algorithms. Exact methods primarily include branch-and-bound [15], cutting-plane methods [16], and dynamic programming [17, 18]. These approaches are designed to guarantee the global optimum. However, the worst-case computational complexity of exact algorithms continues to grow exponentially with the size of the problem, making them difficult to apply to large-scale instances or scenarios with stringent real-time requirements [1, 15, 16]. Heuristic algorithms, on the contrary, trade off between efficiency and solution quality by seeking high-quality feasible solutions within acceptable time limits. This class of methods can be divided as follows:

- **Traditional heuristics:** Traditional heuristics for COPs typically comprise constructive and improvement methods [19]. Representative constructive heuristics include the nearest neighbor heuristic [20], the shortest processing time rule (SPT) [21], and the earliest due date rule [19]. Representative improvement heuristics include k-opt [22], local search [23], and variable neighborhood search [24].
- **Metaheuristics:** Methods such as genetic algorithms (GA) [25, 26], simulated annealing (SA) [27, 28], tabu search (TS) [29, 30], ant colony optimization

(ACO) [4, 31], particle swarm optimization (PSO) [32, 33] and differential evolution (DE) [34, 35] provide high-level, problem-independent search frameworks with strong generality.

- **Hyper-heuristics** [36, 37]: This approach elevates the search space from the solution space to the heuristic space by automatically selecting or generating low-level heuristics (LLHs) to solve problems, aiming for stronger cross-domain generalization capabilities. In previous classifications, hyper-heuristic methods have typically been divided into two categories, namely selection hyper-heuristics [38, 39] and generation hyper-heuristics [40, 41]. Among these, genetic programming [42] is the most widely recognized example of generative hyper-heuristic algorithms.

Despite their widespread success, the inherent limitations of traditional methods have become increasingly apparent. Exact algorithms struggle to handle large scale instances. Heuristic approaches often require extensive iterative search, incur high computational costs, and are heavily dependent on expert knowledge for algorithm design and parameter tuning. Moreover, traditional methods typically lack adaptability to dynamic environments and must often be re-solved when the problem changes slightly. These drawbacks have driven researchers to turn to machine learning (ML) methods, particularly for routing and scheduling in dynamic and time-critical settings, which have emerged prominently in recent years.

1.3 The Rise and Development Context of Machine Learning Methods

In fact, the integration of combinatorial optimization and ML has a long history. It dates back to 1985, when Hopfield employed neural networks to solve the TSP [43]. Since then, many researchers have pursued this direction [44–47]. Early attempts yielded limited success due to the computational and algorithmic constraints of the time, whereas meta-heuristic algorithms also within the artificial intelligence (AI) domain were better matched to the prevailing technology and hardware and therefore enjoyed wide adoption. With the evolution of hardware and algorithms, neural networks experienced their third wave of development two decades ago [48]. Bolstered by deep neural networks (DNN), numerous ML methods began to demonstrate extraordinary capabilities. Over the past two decades, ML has achieved major successes in image recognition [49], natural language processing [50], reinforcement learning (RL) [51], and large language models (LLMs) [52]. This resurgence has led to the renewed introduction of ML paradigms into combinatorial optimization, producing strong empirical results. In

the last ten years, research in this area has progressed from supervision-dependent methods to autonomous exploration, and from problem specific models toward general-purpose tools.

Figure 1 illustrates that the overall pipeline for solving COPs can be decomposed into several stages: modeling, feature extraction, solving algorithm, verification, and explainability. Thanks to the strong data-processing capabilities of modern ML, ML techniques have already penetrated each of these stages. Among them, the solving algorithm stage is particularly critical.

Over the past decade, early explorations on learning-based solvers for COPs have mainly focused on Supervised Learning (SL). The central idea is to treat a COP as an input–output mapping task, where the model is trained on pairs of problem instances and high-quality solutions, and learns to directly predict solutions that approximate or mimic those produced by expert-designed algorithms and classical solvers. The pointer networks (PtrNets) proposed by Vinyals et al. in 2015 is a seminal work in this paradigm, which leveraged an attention mechanism to handle variable-sized output dictionaries and provided the first end-to-end solutions for problems such as the TSP [53]. Subsequently, Graph Neural Networks (GNNs) were introduced to better capture the underlying topological structure of the problems [54], while Transformers, with their strong ability to model global dependencies, have shown excellent performance in handling long sequences and complex constraints [55]. A key advantage of SL is its good convergence behavior when high-quality labels are available. However, its performance is heavily dependent on the quality of labeled data, which is typically generated by traditional solvers.

To overcome this reliance on supervised labels, deep reinforcement learning (DRL) provides an alternative path by formulating COPs as sequential decision-making processes. Inspired by DeepMind’s breakthrough with AlphaGo [51], agents interact with the problem environment and autonomously learn optimization policies from reward signals, such as the negative tour length. The offline training, online

inference paradigm of DRL endows it with great potential for real-time decision making and generalization, and it has rapidly evolved into a core paradigm for applying ML to COPs. Research in this line explores combinations of various neural architectures and RL algorithms, using them not only as end-to-end optimizers [56] but also as decision evaluators embedded within traditional metaheuristic frameworks to guide the search process [57].

In parallel, unsupervised learning (UL) especially methods based on deep generative models—offers yet another avenue for solving COPs. In recent years, generative adversarial networks (GANs), variational autoencoders (VAEs), and diffusion models have been adapted to COPs setting. These approaches treat discrete solutions as structured objects to be generated, and aim to obtain high-quality solutions by learning and sampling from the solution-space distribution. Representative works such as DIFUSCO apply graph diffusion models to TSP and related problems, demonstrating unique advantages in mitigating autoregressive error accumulation and generating diverse solutions [58].

More recently, LLMs have begun to bring about potentially transformative changes to COPs solving. Built upon the Transformer architecture, LLMs possess strong world knowledge, logical reasoning, and code generation capabilities, giving rise to entirely new application paradigms. LLMs not only possess strong logical reasoning capabilities but also offer flexible natural-language interaction, which in principle allows them to be integrated into all stages of the solution process shown in Fig. 1. Researchers have explored using LLMs as *optimizers* that iteratively refine solutions under prompt guidance [59], or as *programmers* that automatically generate heuristic algorithm code [60]. There are even attempts to build end-to-end automated pipelines that solve COPs directly from natural language problem descriptions [61]. Although still in an early stage, LLM-based approaches exhibit extremely low usage barriers and remarkable generalization in handling complex constraints, and thus hold great promise for further lowering the practical barriers to applying COPs methods.

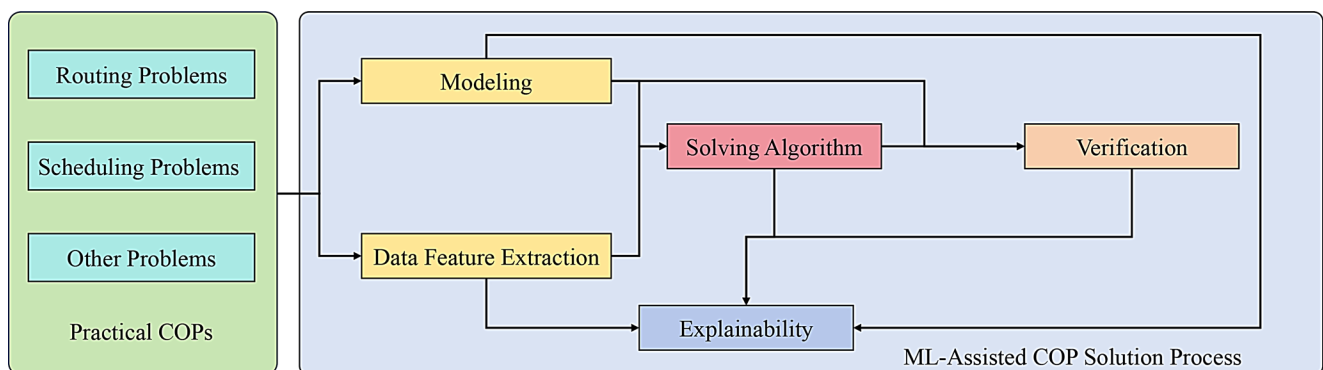


Fig. 1 The solution process for COPs

1.4 Comparison of Existing Surveys and the Motivation of This Work

In summary, ML has injected new vitality into combinatorial optimization. From SL to RL, and from unsupervised generative models to LLMs, each new paradigm has extended the frontier of COPs solving and driven the field toward more intelligent, more general, and more user-friendly methods. In recent years, research on applying ML to COPs has grown explosively, accompanied by a large number of related surveys. However, most of these surveys focus on specific COPs or individual ML paradigms. An overview of surveys from the past five years is given in Table 1.

1.5 Scope and Highlights of This Survey

To provide a clear picture of recent progress on ML-based methods for COPs, this survey organizes and synthesizes a large body of work published between 2020 and 2025. The distinctive features of this survey are summarized as follows:

- **Unified classification of research paradigms:** The surveyed methods are organized into a unified taxonomy of learning paradigms for solving COPs, covering SL, UL, RL, as well as recent LLM-based approaches. In particular, we explicitly incorporate emerging paradigms where LLMs act as optimizers, programmers, or collaborators, thereby providing a structured view of how different learning paradigms are currently used for COPs.
- **Problem-focused synthesis on routing and scheduling:** The survey places special emphasis on two core application domains of COPs, namely RPs and SPs, and systematically summarizes representative ML-based approaches in these areas. At the same time, it also reports typical progress on other classical COPs, such as knapsack problems, graph matching, and the max-cut problem, to give a balanced overview of commonly studied tasks.
- **Curated open-source resources:** For each major category of problems and methods, we collect and organize available open-source resources [90]. A

Table 1 A comparative review of machine learning approaches for solving COPs

Literature	ML methods					COPs		
	SL	UL	Q-Learning	DRL	LLM	RP	SP	Other probs.
Bengio et al. (2021) [62]	✓	✓	✓	✓	×	✓	×	✓
Talbi et al. (2021) [63]	✓	✓	✓	✓	×	✓	✓	✓
Li et al. (2021) [64]	×	×	×	✓	×	✓	✓	✓
Karimi-Mamaghan et al. (2022) [65]	✓	✓	✓	×	×	✓	✓	✓
Li et al. (2022) [66]	✓	×	×	✓	×	✓	×	×
Mazyavkina et al. (2022) [67]	×	×	×	✓	×	✓	✓	✓
Qin et al. (2022) [68]	×	×	✓	✓	×	✓	×	✓
Da Costa Oliveira et al. (2023) [69]	✓	✓	✓	×	×	×	✓	✓
Zhang et al. (2023) [70]	✓	✓	✓	✓	×	✓	✓	✓
Zhang et al. (2023) [71]	✓	✓	✓	✓	×	×	✓	✓
Cappart et al. (2023) [72]	✓	✓	✓	✓	×	✓	×	✓
Szénási et al. (2024) [73]	✓	✓	×	×	×	✓	✓	✓
Zhang et al. (2024) [74]	×	×	×	✓	×	×	✓	×
Yang et al. (2024) [75]	×	×	✓	×	×	✓	✓	✓
Modrak et al. (2024) [76]	×	×	✓	✓	×	×	✓	×
Jalali Khalil Abadi et al. (2024) [77]	×	×	×	✓	×	×	✓	×
Erdem et al. (2024) [78]	✓	×	✓	✓	×	✓	×	×
Fu et al. (2024) [79]	×	×	✓	✓	×	×	✓	×
Smit et al. (2025) [80]	×	✓	×	✓	×	×	✓	×
Lv et al. (2025) [81]	×	×	×	✓	×	×	✓	×
Martins et al. (2025) [82]	×	×	✓	✓	×	✓	✓	✓
Chung et al. (2025) [83]	×	×	×	✓	×	✓	✓	✓
Arishi et al. (2025) [84]	×	×	✓	✓	×	✓	×	×
Sui et al. (2025) [85]	✓	×	×	✓	✓	✓	×	×
Wu et al. (2025) [86]	×	×	×	×	✓	✓	✓	✓
Zhang et al. (2025) [87]	✓	✓	✓	✓	×	✓	✓	✓
Da Ros et al. (2025) [88]	×	×	×	×	✓	✓	✓	✓
Tao et al. (2025) [89]	✓	✓	×	✓	✓	✓	✓	✓
Ours	✓	✓	✓	✓	✓	✓	✓	✓

curated list of these resources is released to facilitate reproduction and further research.

- **Unified multi-dimensional comparative analysis:** Beyond a descriptive taxonomy, this survey establishes a unified comparative framework that systematically evaluates representative methods across seven dimensions: solution quality, scalability, generalization, computational efficiency, data independence, feasibility assurance, and theoretical guarantees. Each method category—including end-to-end SL/RL solvers, ML-OR collaborative frameworks, unsupervised generative models, and LLM-based approaches—is assessed with coarse ratings on a 0-to-5 scale, enabling direct cross-paradigm comparison. This analytical layer goes beyond existing surveys by providing readers with actionable insights into the relative strengths and limitations of each paradigm under varying problem scales, data regimes, and constraint structures (Tables 5–9).
- **Discussion of open challenges and potential directions:** The final part of this survey is devoted to a forward-looking discussion of current limitations in ML for COPs, together with potential solution directions. Beyond conventional challenges such as generalization, balancing computational efficiency and solution quality, theoretical guarantees, data dependency, and dynamics and constraint scalability, we identify two specific and underexplored research gaps: (i) the systematic lack of feasibility guarantees in solutions generated by neural solvers and generative models, and (ii) the absence of mechanisms for handling non-stationary environments and integrating risk-aware objectives into ML-based COP methods. These gaps are grounded in the qualitative patterns revealed by our comparative framework, with the aim of providing targeted guidance for future research.

1.6 Organization of This Paper

The remainder of this paper is organized as follows. Section 2 systematically introduces the core models of COPs, classical modeling approaches (such as mathematical programming, MP and constraint programming, CP), and traditional solution algorithms (including exact methods and heuristic algorithms), thereby laying the foundation for subsequent discussions. Section 3 presents the fundamentals of ML, covering the principal learning paradigms (SL, UL, and RL) and the neural architectures commonly employed for COPs (PtrNet, GNN, Transformer-based models, etc.). Section 4 provides an in-depth treatment of SL applications in COPs, with emphasis on its dual roles as an end-to-end optimizer and as a decision evaluator when integrated with exact and heuristic algorithms. Section 5 offers a comprehensive

synthesis of RL methods and their taxonomy in COPs, detailing how Q-learning and DRL have been applied as end-to-end optimizers or decision evaluators and how they have been deeply integrated with heuristic, metaheuristic, and hyper-heuristic frameworks. Section 6 reviews advances in UL for COPs, focusing primarily on generative models such as diffusion model, GNN, and VAE. Section 7 examines emerging applications of LLMs in COPs, assessing their potential and challenges when employed as optimizers, programmers, modeling and evaluation tools, or as the core of integrated pipelines. Section 8 discusses the current challenges and prospective directions for future research. Finally, Sect. 9 concludes the survey.

2 Foundations of Combinatorial Optimization Problems

COPs are characterized by the search for an optimal configuration from a finite, discrete set of possibilities, subject to specific constraints. Their inherent computational complexity, primarily stemming from the exponentially growing solution space, necessitates a deep understanding of both modeling paradigms and solution methodologies. This section lays the conceptual and algorithmic groundwork for the survey. We begin by introducing the principal modeling frameworks—*mathematical programming*, *constraint programming*, and *graph and network models*—that translate real-world decision problems into formal, tractable formulations. Subsequently, we systematically review the core classes of solution algorithms, ranging from exact methods that guarantee optimality to heuristic and metaheuristic strategies designed for scalability and practical utility. Finally, we outline typical problem domains, with a focus on routing and scheduling problems, which serve as primary benchmarks and application drivers for the ML techniques discussed in later sections. This foundational overview establishes the essential vocabulary and context for understanding the integration and impact of modern ML approaches in combinatorial optimization.

2.1 Modeling Methods

2.1.1 Mathematical Programming

MP is a fundamental and versatile modeling approach for combinatorial optimization and operations research (OR) problems. Its core principle is to formulate practical problems with objective functions and constraints, using solvers to find optimal solutions within the feasible region. Mixed integer programming (MIP) is commonly used when the

problem involves linear objectives and constraints alongside discrete decisions.

MIP models optimization problems using linear objective functions and linear constraints, while incorporating integer or binary decision variables to represent discrete choices (e.g., yes/no decisions), and continuous variables for aspects like resource allocation or time. A typical MIP formulation is:

$$\begin{aligned} \min \quad & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b}, \\ & \mathbf{x} \in \mathbb{Z}^p \times \mathbb{R}^{n-p} \end{aligned} \quad (1)$$

The key strengths of MIP lie in its generality and interpretability. It provides a unified framework for modeling various COPs such as the TSP, VRP, facility location, production scheduling, and supply chain design. Using commercial solvers (e.g., Gurobi, CPLEX), MIP can guarantee globally optimal solutions for small- to medium-scale instances, supported by precise convergence analysis via dual bounds. However, MIP's computational complexity is typically NP-hard due to integer variables. Even with advanced techniques like branch-and-bound, solving large-scale or real-time problems to global optimality often becomes intractable.

Thus, MIP is best suited for moderate-sized problems with well-defined constraints, especially when exact solutions are required. Examples include warehouse location, batch scheduling, and energy generation planning. For large-scale, real-time, or highly nonlinear problems, MIP is often combined with heuristics, decomposition methods, or ML to achieve feasible and efficient solutions.

2.1.2 Constraint Programming

Optimization problems in the real world are often highly complex, with objective functions and constraints that contain nonlinear components or complex relationships that are difficult to describe precisely. CP has been developed as an effective method to model and solve such complex problems.

CP is a class of modeling and solving methods centered around constraints. The basic idea is to describe the problem through variables and their domains, then apply a set of constraints to limit the possible combinations of these variables. Unlike methods centered on an objective function, CP focuses more on characterizing the feasible solutions of the problem. During the solving process, the variable domains are typically narrowed through constraint propagation, and combined with search techniques, the solution space is gradually explored until a solution that satisfies all the constraints is found. This method emphasizes the declarative

nature of the problem, meaning that the modeler only needs to describe the conditions to be met, without explicitly specifying how to solve the problem.

A typical CP problem can be represented as a triplet:

$$P = (X, \mathcal{D}, C) \quad (2)$$

where $X = \{x_1, x_2, \dots, x_n\}$ represents a set of decision variables. $\mathcal{D} = \{\mathcal{D}(x_1), \mathcal{D}(x_2), \dots, \mathcal{D}(x_n)\}$ represents the domains of the variables, where $\mathcal{D}(x_i)$ is the set of all possible values for variable x_i . $C = \{c_1, c_2, \dots, c_m\}$ represents the set of constraints; each constraint c_j defines the allowed value combinations for a subset of variables.

Under the constraint set C , the goal is to find a variable assignment:

$$\mathcal{A} = \{x_1 = z_1, x_2 = z_2, \dots, x_n = z_n\}, \quad z_i \in \mathcal{D}(x_i) \quad (3)$$

such that for every constraint $c_j \in C$, the assignment $\mathcal{A}$ satisfies c_j .

In some problems, an objective function $\mathcal{F}(X)$ is introduced, transforming the problem into finding the optimal solution within the feasible solution set:

$$\min \mathcal{F}(X) \quad \text{s.t.} \quad (X, \mathcal{D}, C) \quad (4)$$

$$\max \mathcal{F}(X) \quad \text{s.t.} \quad (X, \mathcal{D}, C) \quad (5)$$

Equation (2) defines the three essential components of a CP model. Equation (3) represents a feasible assignment that satisfies all constraints. Equations (4) and (5) provide the optimization objectives under constraints. This formal description highlights the core modeling idea of CP, characterizing the feasible space of the problem through variables, domains, and constraints, thus laying the foundation for subsequent propagation and search.

The advantage of CP lies in its natural and intuitive modeling, which allows practical problems to be directly expressed in terms of variables and constraints. It is highly flexible and scalable. Its general solving framework makes it applicable to different types of problems, and it performs well in terms of interpretability. However, this method also has certain limitations. The efficiency of solving often depends on the effectiveness of constraint propagation and search strategies. When propagation is insufficient, the search space quickly expands, leading to a decline in computational efficiency. Moreover, in handling complex optimization objective functions, CP is typically less efficient than methods such as mathematical programming, and its adaptability to problems with continuous variables is limited.

2.1.3 Graphs and Networks

Graphs and networks constitute another fundamental modeling approach for COPs. In this paradigm, a real system is abstracted as a graph $G = (V, E)$, where vertices represent entities such as locations, tasks, or states, and edges represent feasible relations such as reachability, dependency, or transmission, often with associated weights, capacities, or costs. Optimization techniques for graphs and networks are then applied to this representation. Representative models include shortest paths [91, 92], maximum flow and minimum cut [93], and minimum-cost flow [94]. Their basic mathematical formulations are as follows.

Single-Source Shortest Path:

$$d(v) = \min_{(u,v) \in E} \{ d(u) + w(u, v) \}, \quad d(s) = 0, \quad (6)$$

where $w(u, v)$ is the edge weight. Computation along a topological order solves it in one pass on directed acyclic graphs (DAGs) [91]; for general graphs with nonnegative weights, Dijkstra's algorithm applies [92].

Maximum Flow:

$$\begin{aligned} \max \quad & \sum_{(s,v) \in E} f_{sv} \quad \text{s.t.} \quad \sum_{(u,v) \in E} f_{uv} \\ & - \sum_{(v,w) \in E} f_{vw} = 0 \quad (\forall v \neq s, t), \quad 0 \leq f_{uv} \leq c_{uv}, \end{aligned} \quad (7)$$

i.e., maximize the flow from source s to sink t under capacity c_{uv} and flow conservation; augmenting paths correspond to cuts, yielding the max-flow/min-cut equality [93].

Minimum-Cost Flow:

$$\begin{aligned} \min \quad & \sum_{(u,v) \in E} c_{uv} f_{uv} \quad \text{s.t.} \quad \sum_{(u,v) \in E} f_{uv} \\ & - \sum_{(v,w) \in E} f_{vw} = b(v), \quad 0 \leq f_{uv} \leq U_{uv}, \end{aligned} \quad (8)$$

where c_{uv} is the per-unit cost, $b(v)$ is the node supply, and U_{uv} is the capacity upper bound; common solution methods include potential-based and shortest-path improvements, cycle canceling, and the network simplex algorithm [94].

2.2 Solution Algorithms

2.2.1 Exact Algorithms

Cutting-Plane Method

The cutting-plane method is a classical exact algorithm that progressively adds valid inequalities (cuts) to the linear

programming (LP) relaxation feasible region, gradually approximating the integer programming convex hull and eventually converging to the integer optimal solution. Consider the integer programming problem:

$$\min \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{Ax} \leq \mathbf{b}, \quad \mathbf{x} \in \mathbb{Z}_+^n \quad (9)$$

Its linear relaxation is:

$$\min \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{Ax} \leq \mathbf{b}, \quad \mathbf{x} \in \mathbb{R}_+^n \quad (10)$$

The algorithm starts by solving the linear relaxation. If a fractional solution $\mathbf{x}^*$ is obtained, it generates a valid inequality $\alpha^\top \mathbf{x} \leq \beta$ that is satisfied by all integer feasible solutions but violated by $\mathbf{x}^*$. This cut is added to the constraint set and the process repeats until an integer solution is found.

This method can strengthen operation sequencing constraints in SPs and ensure path connectivity through subtour elimination constraints (SECs) $\sum_{i \in S, j \notin S} x_{ij} \geq 1$ in the TSP. Its advantages include theoretical rigor and guaranteed global optimality, though the convergence rate depends on cut quality.

Branch-and-Bound Method

Branch-and-bound employs a divide-and-conquer strategy, systematically partitioning the solution space and utilizing bound information for pruning. The algorithm starts by solving the linear relaxation to obtain a lower bound

$$L = \min \{ \mathbf{c}^\top \mathbf{x} \mid \mathbf{Ax} \leq \mathbf{b}, \mathbf{x} \in \mathbb{R}_+^n \}.$$

If a fractional solution $x_j^* \notin \mathbb{Z}$ is obtained, branching operations $x_j \leq \lfloor x_j^* \rfloor$ and $x_j \geq \lceil x_j^* \rceil$ generate two subproblems. Each subproblem's lower bound L_i is compared with the current upper bound UB ; if $L_i \geq UB$, the branch is pruned; otherwise, branching continues.

This method is widely applied to route optimization, production scheduling, and other problems. Its core strengths include guaranteed global optimality and significant search space reduction through pruning. Modern optimization solvers implement highly optimized branch-and-bound algorithms, often combined with cutting-plane methods in branch-and-cut frameworks.

2.2.2 Traditional Heuristic Methods

Constructive Heuristic

Constructive heuristics are a class of heuristic methods based on the greedy principle. The core idea is to gradually build a feasible solution in the solution space. At each decision step, the algorithm selects the currently optimal operation according to a greedy criterion, such as choosing

the operation with the shortest processing time (SPT), the earliest available machine, or the scheduling plan that minimizes local delays. By repeatedly applying this process, a complete feasible solution is eventually obtained.

For SPs, a constructive heuristic typically follows these steps. First, initialize the task and resource states. Second, select the next task to be assigned and its machine based on the scheduling criterion. Third, update the resource status and time schedule. Fourth, repeat steps two and three until all tasks are scheduled. Common greedy criteria include SPT first, earliest start time first, and the critical path method, among others. These criteria can quickly produce a feasible solution, especially in large-scale problem scenarios.

The advantage of constructive heuristics lies in their simplicity and intuitiveness. They are computationally efficient, capable of generating feasible solutions in a short amount of time, and provide an initial solution for subsequent improvement-based heuristics or metaheuristics. Moreover, the implementation process does not rely on complex mathematical models, making them suitable for scheduling environments with high real-time requirements. However, the shortcomings of this method are also apparent. Since it only makes decisions based on local greedy criteria, it is often difficult to escape local optima, and the quality of the solution typically depends on the heuristic rule used. In complex constraint or multi-objective optimization problems, the performance of constructive heuristics is limited, and it is difficult to guarantee solutions close to the optimum.

Improvement Heuristics

Constructive heuristics typically yield solutions of moderate quality, making it necessary to further refine the solution using more advanced heuristic techniques. Neighborhood search is the most commonly used improvement heuristic. Its core principle involves starting from an initial solution and iteratively exploring the neighborhood space of the current solution to find better alternatives, progressively enhancing the solution quality until a termination condition is satisfied.

The definition of a neighborhood is usually formalized as:

$$N(x) = \{x' \mid x' \text{ is obtained from the current solution } x \text{ through local adjustments}\} \quad (11)$$

where x represents the current solution, $N(x)$ represents its neighborhood, and $x' \in N(x)$ represents a neighboring solution. Local adjustment operations can include operation swaps, insertions, or resource reallocations. For example, in the JSSP, neighborhood operations often include swapping the execution order of adjacent operations on the same

machine or inserting a particular operation into a different position.

During the search process, the algorithm selects a candidate solution x' from the neighborhood and updates it according to the evaluation function:

$$x \leftarrow \arg \min_{x' \in N(x)} \mathcal{F}(x') \quad (12)$$

where $\mathcal{F}(x)$ is the objective function. This update rule reflects the optimal neighborhood selection strategy. If random selection or mechanisms with probabilistic acceptance are used, it can help escape local optimum traps. Additionally, k-opt is a commonly used improved heuristic for solving RPs.

The advantage of improvement heuristics is their ability to significantly enhance the quality of the initial solution in a short amount of time. The method is flexible and applicable to various complex COPs. By properly designing the neighborhood structure and search strategy, near-optimal solutions can be obtained within an acceptable computational time. However, its shortcomings include: when the problem scale is too large or the neighborhood is too complex, the computational cost of the search process increases significantly. Furthermore, the algorithm's performance heavily depends on the definition of the neighborhood and update strategy. If poorly designed, it may fall into local optima or converge slowly.

2.2.3 Metaheuristic Methods

Metaheuristic frameworks broadly explore the search space via perturbation and recombination, then intensify the search locally in promising regions. Using either a single solution or a population as carriers, the iterative process comprises generation, evaluation, selection, memory/learning, perturbation/update, and termination. Initially, feasible solutions (together with memory structures when applicable) are generated. New solutions are produced via recombination, neighborhood moves, or probabilistic models. Candidates are evaluated and, if necessary, repaired with respect to objectives and constraints. High-quality and diverse individuals are preserved by selection and memory. Local search or strong perturbations are invoked as needed. At termination, the best solution(s) are output. An abstract iteration with memory $\mathcal{M}^t$, generator G_ϕ , selector S_ψ , and population $\mathcal{P}^t$ can be written as (see the main text for details):

$$\mathcal{P}^{t+1} = S_\psi(\mathcal{P}^t \cup G_\phi(\mathcal{P}^t, \mathcal{M}^t)), \quad \mathcal{M}^{t+1} = \text{UpdateMemory}(\mathcal{M}^t, \mathcal{P}^{t+1}).$$

Single-solution methods often adopt an acceptance rule to balance improvement and exploration:

$$x^{t+1} = x^t + \mathbf{1}\{\text{Accept}(x', x^t \mid t, \mathcal{M}^t)\} (x' - x^t).$$

Metaheuristics are commonly divided into four families: evolutionary, swarm intelligence, physics-inspired, and human-behavior-inspired.

Evolutionary algorithms

Genetic algorithms (GA) involve encoding, selection, crossover, and mutation [95]. Differential evolution (DE) uses difference vectors to drive mutation [96]. For discrete COPs, DE is adapted via binary/permutation encoding, swap/inversion neighborhoods, or set differences. For distinct indices r_1, r_2, r_3 , a mutant vector is generated:

$$v_i = x_{r_1} + F(x_{r_2} - x_{r_3}),$$

then a trial vector u_i is produced via crossover rate CR :

$$u_{ij} = v_{ij} \mathbf{1}\{r_{ij} < CR \vee j = j_{\text{rand}}\} + x_{ij} \mathbf{1}\{r_{ij} \geq CR \wedge j \neq j_{\text{rand}}\},$$

and the solution is updated greedily:

$$x_i^{t+1} = \arg \min\{f(x_i^t), f(u_i)\}.$$

Common crossover and mutation operators for COPs include partially mapped crossover, sequential crossover, segment inversion, etc.

Swarm intelligence

Particle swarm optimization (PSO) treats candidates as particles updated by personal best p_i and global best g [97]. In discrete domains, velocity maps to swap sequences or probability structures:

$$v_i \leftarrow \omega v_i + c_1 r_1 (p_i - x_i) + c_2 r_2 (g - x_i), \quad x_i \leftarrow x_i + v_i.$$

The whale optimization algorithm (WOA) alternates encircling and spiral hunting, often coupled with repair and local improvement [98].

Physics-inspired algorithms

Simulated annealing (SA) [99] samples a candidate x' from a neighborhood $N(x)$. It accepts x' if $\Delta = f(x') - f(x) \leq 0$; otherwise, it accepts with probability $\exp(-\Delta/T)$. Temperature is decreased by:

$$T_{t+1} = \alpha T_t \quad (\alpha \in (0, 1)) \quad \text{or} \quad T_t = \frac{T_0}{\log(1+t)}.$$

For problems like TSP, scheduling, and graph coloring, neighborhoods typically involve swap, insertion, and reversal operations.

Human-behavior-inspired algorithms

Teaching–learning-based optimization (TLBO) [100] consists of a teacher stage and a learner stage. Let μ be the class mean, x^* the current best, and teaching factor $T_f \in \{1, 2\}$. In the teacher stage:

$$x'_i = x_i + r \cdot (x^* - T_f \mu).$$

In the learner stage, pick $j \neq i$; if $f(x_j) < f(x'_i)$, then $x''_i = x'_i + r(x_j - x'_i)$, else $x''_i = x'_i + r(x'_i - x_j)$. In discrete domains, updates are implemented via insertion or probabilistic template adjustments.

2.2.4 Hyper-Heuristic Methods

Hyper-heuristics operate on a space of low-level heuristics (LLHs) rather than directly on the solution space, aiming to improve cross-domain generality [101, 102]. They are broadly classified as selection-based or generation-based, and further combined with constructive/perturbative and offline/online/no-learning dimensions [103]. Frameworks such as HyFlex provide cross-domain benchmarks (e.g., MAX-SAT, bin packing, job-shop scheduling, personnel scheduling) for fair comparison [104].

A hyper-heuristic maintains a set of LLHs $\mathcal{H} = \{h_1, \dots, h_K\}$ and memory $\mathcal{M}^t$. At iteration t , based on context features $\phi(x^t, \mathcal{M}^t)$, it selects or generates an LLH:

$$\begin{aligned} h_t &= \pi(\phi(x^t, \mathcal{M}^t); \theta), \\ x^{t+1} &= \text{Apply}(h_t, x^t), \\ \mathcal{M}^{t+1} &= \text{Update}(\mathcal{M}^t, x^{t+1}, h_t), \end{aligned} \quad (13)$$

where π may be rule-based, statistical, or learned (e.g., multi-armed bandits, reinforcement learning). Performance feedback adjusts π online [101, 103].

A typical selection hyper-heuristic uses a score-and-choose strategy. For each $h \in \mathcal{H}$, maintain recent improvement per cost, diversity contribution, and recency via exponential decay or sliding windows:

$$S_t(h) = \alpha \text{Imp}_t(h) + \beta \text{Div}_t(h) + \gamma \text{Rec}_t(h), \quad (14)$$

then $h_t = \arg \max_{h \in \mathcal{H}} S_t(h)$. This design transfers across problem domains in HyFlex [101, 104].

Generation hyper-heuristics recompose existing heuristic components using templates, grammars, or libraries, learned offline or adapted online, enabling structure and parameter learning for problem families [102, 103].

2.3 Typical Problems

2.3.1 Routing Problems

Routing problems (RPs) are a cornerstone class of COPs and a primary benchmark family in recent ML-based optimization research. They essentially study how to find optimal route arrangements for one or multiple paths within a graph or network, aiming to optimize one or more objectives (such as cost, distance, or time) while satisfying various constraints.

The decisions are discrete (e.g., which customers to visit in what order), and the optimal solution must be found from a vast number of possible path combinations. The vast majority of RPs are NP-hard, meaning that as the problem scale increases, the computational complexity of finding the optimal solution rises sharply, typically requiring advanced exact algorithms or heuristic algorithms for solving.

The traveling salesman problem (TSP) is the most fundamental RP. The goal is to find the shortest route that visits each customer point exactly once and returns to the starting point. It is the cornerstone of all RPs. Its general model is as follows.

$$\min \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (15)$$

$$\text{s.t.} \quad \sum_{j=1}^n x_{ij} = 1, \quad \forall i = 1, \dots, n, \quad (16)$$

$$\sum_{i=1}^n x_{ij} = 1, \quad \forall j = 1, \dots, n, \quad (17)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i, j = 1, \dots, n, \quad (18)$$

$$\text{SECs.} \quad (19)$$

Here, the binary variable $x_{ij} \in \{0, 1\}$ ($\forall i, j = 1, \dots, n$) equals 1 if the edge (i, j) is in the optimal solution. Equation (15) indicates that the objective is to minimize the total distance. Equation (16) is the outdegree constraint, which ensures that each vertex is left exactly once. Equation (17) is the indegree constraint, which ensures that each vertex is entered exactly once. Together, they ensure that each vertex appears exactly once in the path, satisfying the basic condition for a hamiltonian cycle. Equation (18) is the integrality constraint. Furthermore, these two sets of degree constraints alone are insufficient to ensure solution correctness, as subtours might occur; thus, subtour elimination constraints

(SECs) must be added. Different TSP models differ mainly in their implementation of the SECs.

Due to the complexity of real-world application scenarios, the TSP has spawned numerous variants, such as the capacitated vehicle routing problem (CVRP) considering vehicle capacity limits, the prize-collecting traveling salesman problem (PCTSP), the TSP with time windows (TSPTW), and the orienteering problem (OP) with profits or rewards. These variants have wide applications in logistics distribution, path planning, UAV mission assignment, etc., with different problems having distinct constraint conditions and objective functions. For ease of comparison and understanding, Table 2 summarizes the characteristics and applications of several major variants of the TSP.

The vehicle routing problem (VRP) is a generalization of the TSP and is one of the most classic and practical RPs. The goal is to design a set of optimal routes for multiple vehicles from a single depot to serve all customer points. The core challenge involves both assigning customer points to vehicles (clustering) and arranging the visit sequence for each vehicle (routing).

As real-world application scenarios become more complex, the VRP has spawned many variants, including the VRP with time windows (VRPTW) [118], the capacitated VRP (CVRP) [119], the dynamic VRP (DVRP) with dynamic orders or real-time re-optimization needs [120], and the split delivery VRP (SDVRP) [121], among others. These variants introduce additional constraints or optimization objectives for different business requirements, making the problems more aligned with practical logistics distribution scenarios.

2.3.2 Scheduling Problems

Scheduling problems (SPs) are a cornerstone topic in COPs and the other primary focal domain of this survey, especially in manufacturing and resource management. They aim to allocate tasks over time and machines under limited resources and technological constraints, with the goal of optimizing objectives such as makespan, total cost, or energy efficiency. As the number of jobs and resources increases, the state space grows exponentially, making most SPs NP-hard.

Among them, the JSSP serves as a canonical benchmark widely used in manufacturing, transportation, and computing resource management. Its typical objective is to minimize the makespan and related performance metrics [81]. To address real-world requirements, many extensions and variants have been developed, including the FJSP, dynamic job shop scheduling problem (DyJSP), dynamic flexible job shop scheduling problem (DyFJSP), distributed job shop scheduling problem (DiJSP), and the hybrid flow shop

Table 2 Overview of TSP and its major variants

Category	Key Constraint	Problem Characteristic
Symmetric TSP [105]	Path cost satisfies $c_{ij} = c_{ji}$	Undirected graph, standard TSP, suitable for modeling symmetric travel costs
Asymmetric TSP [106]	Path cost asymmetric $c_{ij} \neq c_{ji}$	Directed graph, travel cost depends on direction
TSP with Time Windows [107]	Access time window $[e_i, l_i]$ restriction	Time window penalty or infeasibility constraint; requires visiting cities within given time windows; early arrival requires waiting, late arrival is infeasible
Capacitated TSP [108]	Tour cumulative demand $\leq Q$ or path length $\leq L$	Complex SECs; traveler load/travel distance is limited
Multiple TSP [109]	Multiple travelers cooperate to complete tasks	Combined partitioning and routing optimization, multiple travelers start from the same or different points to perform tasks
Dynamic TSP [110, 111]	Nodes gradually released	Real-time re-optimization or online optimization; cities or tasks appear over time, requiring dynamic path planning
Stochastic TSP [112]	Node or edge weights are random variables	Two-stage planning, chance constraints; robust or expected optimal path under uncertainty
Prize-Collecting TSP [113]	Nodes can be skipped, incurring a penalty	Objective includes penalty term; minimize path cost and penalty while ensuring partial coverage
Orienteering Problem [114]	Nodes have rewards, path budget constraint	Knapsack-style constraint; maximize reward under limited path budget
Pickup and Delivery TSP [115, 116]	Pickup-delivery sequence constraint	Precedence constraint, pickup before delivery; each demand has corresponding pickup and delivery points, route must ensure sequence
TSP with Obstacles [117]	Path must avoid obstacle regions	Geometric constraints, feasible region shrinkage; nodes fixed, paths affected by obstacles

Table 3 Representative variants of SPs

Category	Key Constraint	Problem Characteristic
Job Shop Scheduling Problem [123]	Each job has a fixed operation order; every operation can be processed only on its designated machine	Typical discrete optimization; constraints include precedence relations and machine non-overlap constraints
Flexible Job Shop Scheduling Problem [124]	Extends JSSP with machine flexibility, allowing each operation to choose among compatible machines	Mixed-integer formulation with discrete machine-assignment variables and continuous timing variables
Dynamic Job Shop Scheduling Problem [125]	Dynamic events (e.g., new job arrivals, machine breakdowns) require real-time schedule updates	Combination of online scheduling and rolling-horizon optimization; constraints are reconfigured dynamically
Dynamic Flexible Job Shop Scheduling Problem [126]	Combines machine flexibility with dynamic disturbances	Multi-objective dynamic optimization; often solved via heuristics or reinforcement learning
Distributed Job Shop Scheduling Problem [127]	Jobs must be assigned to multiple independent plants and cannot migrate across plants	Bi-level structure: upper-level job-plant assignment and lower-level in-plant JSSP scheduling
Hybrid Flow Shop Problem [128, 129]	Integrates flexibility with a hybrid flow-shop structure (parallel machines within stages and inter-stage precedence)	Multi-stage complex optimization handling both intra-stage parallelism and inter-stage process constraints

problem (HFSP), as well as multi-objective, energy-aware, and robust scheduling [122]. These models have been extensively applied in smart manufacturing, transportation and logistics, and cloud resource management. A comparative summary of key constraints and modeling characteristics is provided in Table 3.

2.3.3 Other Problems

Beyond routing and scheduling, we briefly summarize other representative COPs that frequently appear in ML literature (Table 4).

3 Foundations of Machine Learning

ML provides a suite of powerful, data-driven methodologies that have been increasingly integrated with combinatorial optimization to develop more adaptive, efficient, and scalable solvers. This section establishes the technical underpinnings necessary for understanding these hybrid approaches. We begin by introducing the three fundamental learning paradigms: SL, UL, and RL, each offering distinct mechanisms for leveraging data within optimization. Subsequently, we review the core NN architectures—including PtrNets, convolutional neural networks (CNNs), Transformers, and GNNs that serve as the primary function approximators in modern DL-based solvers. We also examine the emergent role of LLMs, which introduce new capabilities via natural language interaction and in-context reasoning. Finally, we synthesize these components into

Table 4 Other COPs: constraints and characteristics

Category	Key Constraint	Problem Characteristic
Ranking and Ordering [130, 131]	Produce an ordering that respects preference relations and item relevance	Permutation optimization modeled with integer programming or learning-to-rank objectives; combinatorial and typically NP-hard
Graph Matching [14, 132–135]	Preserve structural consistency between two graphs and align nodes while tolerating noise	Assignment-based optimization with consistency constraints; uses continuous relaxation and discrete search; high computational complexity
Max-Cut [136, 137]	Partition vertices into two sets to maximize the sum of crossing edge weights	Binary discrete optimization; semidefinite programming (SDP) relaxations and approximation algorithms are common
Maximum Independent Set (MIS) [138, 139]	Select a set of vertices with no adjacent pairs	Set-packing style integer programming; solved with branch-and-bound and heuristics
Facility Location [12]	Decide which facilities to open and assign each demand to an open facility while respecting capacity and coverage limits	Mixed-integer linear optimization with decomposition and approximation methods; balances cost and service quality
Knapsack [10, 140, 141]	Choose items within a capacity limit to maximize total value	Zero-one selection model; small instances use dynamic programming; large instances rely on heuristics and approximation
Bin Packing [142, 143]	Pack items into bins without exceeding capacity and reduce the number of bins or waste	Set partitioning and column generation formulations; branch-and-price and constructive heuristics are widely used
Graph Coloring [11, 144]	Assign colors so that adjacent vertices differ while minimizing the number of colors	Assignment or covering formulations; solved with integer programming, constraint programming, and tailored heuristics
Influence Maximization [145]	Select seed nodes under a budget to maximize expected spread under a specified diffusion process	Often submodular and monotone; greedy selection yields a constant-factor guarantee; needs efficient sampling and approximation
Causal Discovery [146]	Output a directed acyclic causal graph consistent with conditional independence tests and intervention evidence when available	Combinatorial search over acyclic graph space with a global acyclicity constraint; uses score-based or constraint-based optimization
Electronic Design Automation [13, 147]	Satisfy layout and electrical rules while meeting timing, area, power, and connectivity targets	Very large-scale mixed-integer and satisfiability style models with hierarchical decomposition and multi-objective trade-offs
Virtual Network Embedding [148]	Map virtual nodes and links to substrate resources while meeting capacity, latency, isolation, and quality of service requirements	Joint node mapping and path routing optimization with decomposition, integer programming, and scalable heuristics
Molecular Optimization and Drug Discovery [149, 150]	Ensure chemical validity and synthesizability while achieving target properties and safety constraints	Multi-objective constrained search in discrete molecular space with property prediction and generative strategies

three overarching methodological frameworks that characterize how ML interacts with combinatorial problems: end-to-end learned solvers, collaborative systems that augment traditional OR algorithms, and generative models that capture solution distributions. This foundational overview equips readers with the essential concepts and taxonomy to navigate the detailed applications and advances presented in the subsequent sections.

3.1 Fundamental Paradigms

3.1.1 Supervised Learning

SL learns mapping relationships between inputs and outputs through labeled datasets. In combinatorial optimization, models such as PtrNets or GNNs are often employed, using optimal paths or variable assignments as labels, and trained end-to-end with loss functions like cross-entropy.

Given a training set $\{(x_i, y_i)\}_{i=1}^N$, SL seeks a parameterized model $f_\theta(x)$ by minimizing the loss function:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i) \quad (20)$$

This approach efficiently captures underlying data patterns and achieves excellent performance when labels are abundant. However, it heavily relies on labeled data, which is costly to obtain, and the models have limited generalization capability, often failing when the data distribution shifts.

To mitigate label dependency, semi-supervised learning combines limited labeled data with abundant unlabeled data, enhancing performance through self-training, graph-based methods, or consistency regularization. The core idea leverages the distributional structure in unlabeled data to robustify decision boundaries, proving particularly valuable in fields like natural language processing and medical imaging where labeling is expensive.

3.1.2 Unsupervised Learning

UL aims to learn the distribution and structure of data without labels. Generative approaches approximate $p(x)$ for density estimation and sampling; non-generative approaches learn representations or structures transferable to downstream tasks. A common approach is representation learning: map raw samples x to more separable and robust features $h(x)$, and then train a lightweight classifier or regressor on a small amount of labels to evaluate discriminability and transferability [151]. Typical objectives include compression and dimensionality reduction via low-rank or sparse structures, clustering and segmentation, anomaly detection, imputation of missing values, and generation. In practice, unsupervised and self-supervised pretraining often reduces labeling cost and improves robustness [151]. From a modeling perspective, generative methods approximate $p(x)$ explicitly or implicitly and allow sampling; non-generative methods do not model $p(x)$ directly but learn structure and separability via clustering, matrix factorization, spectral methods, or contrastive objectives. In systems, the two are often combined: obtain discriminative embeddings first, then use generative models for synthesis, editing, and augmentation [151].

Among generative models, diffusion models decompose generation into forward noising and reverse denoising:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad t = 1, \dots, T, \quad (21)$$

train a noise predictor or score network to approximate the reverse distribution so sampling can start from noise and iteratively denoise; training usually uses mean squared error on the noise $\epsilon_{\theta}(x_t, t)$ [152]. This framework trains stably with strong coverage and alleviates mode collapse issues

common in GANs, at the cost of many sampling steps, which can be accelerated with conditional control [152].

GANs learn via a minimax game between a generator and a discriminator:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))], \quad (22)$$

yielding sharp samples for synthesis and style transfer but potentially suffering from unstable training and mode collapse [153].

VAEs are latent-variable models optimizing the evidence lower bound:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)] - \text{KL}(q_{\phi}(z | x) \| p(z)), \quad (23)$$

with

reparameterization $z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ for stable gradients [154]. VAEs offer probabilistic consistency, structured latent spaces, and uncertainty quantification but may produce overly smooth reconstructions; improvements use decoder capacity, prior design, or perceptual losses [154].

Another class of methods does not model $p(x)$ explicitly but learns structure and representations directly: clustering and graph clustering for grouping, spectral methods and matrix factorization for discovering low-rank structure, sparse coding and nonnegative matrix factorization for interpretable components, and contrastive self-supervision for discriminative embeddings that transfer with few labels [151]. Compared with generative models, these directly target separability, compression, and structural interpretation with efficient training; generative models offer sampling and stronger distribution understanding. In practice, they are often used together [151].

3.1.3 Reinforcement Learning

RL is a class of methods that learn optimal decision-making policies through interaction with an environment. The core idea is that an agent takes actions in different states and adjusts its strategy based on the reward signal received from the environment, aiming to maximize cumulative rewards. RL can be divided into model-based and model-free approaches. Model-free methods, due to their ability to adaptively optimize in complex decision spaces without requiring an accurate environmental model, have shown great potential in approximating solutions for COPs.

Q-learning

Q-learning is a classical model-free, value-based method. Its objective is to iteratively update the state-action value function (Q-function) to approximate the optimal policy. The core update rule is:

$$Q(s_t, a_t) \leftarrow (1 - \eta) Q(s_t, a_t) + \eta \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') \right], \quad (24)$$

where η is the learning rate and γ is the discount factor. This update rule reflects a weighted balance between immediate rewards and optimal future rewards.

The advantage of Q-learning is its simplicity, as it does not rely on an environment model, making it suitable for various complex COP scenarios. Its drawback is that the state-action space grows rapidly with the problem dimensions, making it difficult to store and update the Q-table, which limits its scalability.

Deep Reinforcement Learning

To overcome the scalability issue of traditional Q-learning, DRL introduces neural networks as function approximators to approximate the value function or policy function, enabling efficient learning in large-scale COPs.

In value-based methods, the deep Q-network (DQN) uses a neural network $Q(s, a; \theta)$ to approximate the Q-function. The update is performed by minimizing the mean squared error loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{B}} \left[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2 \right], \quad (25)$$

where $\mathcal{B}$ is the experience replay buffer, and θ^- represents the target network parameters.

In policy optimization methods, the policy gradient directly parameterizes the policy $\pi(a|s; \theta)$ and optimizes it by maximizing the expected return. However, it suffers from high variance and unstable training. To improve performance, the actor-critic (AC) framework combines value and policy methods. The actor outputs the action distribution, and the critic evaluates its value, providing both flexibility and stability. Building on this, proximal policy optimization (PPO) further refines the update method, with the objective function:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad (26)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, and $\hat{A}_t$ is the advantage function estimate. This method enhances return while effectively controlling the magnitude of policy updates, improving training stability.

Overall, DRL methods, with the powerful representational ability of neural networks, can automatically extract features and learn approximate optimal policies in large-scale COPs. DQN is suitable for discrete action spaces,

while policy gradients and PPO are better suited for continuous spaces and complex policy optimization. However, their drawbacks include reliance on large amounts of samples and computational resources for training, and their generalization and stability still need improvement.

3.2 Common Neural Networks in Deep Learning

3.2.1 Pointer Networks

The PtrNet addresses COPs through an encoder-decoder sequence-to-sequence architecture. The core innovation lies in using attention mechanisms as pointers to directly select elements from the input sequence to form the output sequence, effectively handling dynamically sized outputs [53]. The operational mechanism of the PtrNet is shown in Fig. 2.

Given an input sequence $P = \{p_1, \dots, p_n\}$ (e.g., city coordinates), the encoder generates hidden states $\{e_1, \dots, e_n\}$ via long short-term memory networks (LSTM), while the decoder autoregressively produces the solution sequence. At each step i , attention scores are computed based on the current decoder state d_i :

$$u_j^i = v^T \tanh(W_1 e_j + W_2 d_i), \quad j = 1, \dots, n \quad (27)$$

Selection probabilities are obtained via softmax:

$$p(C_i | C_{1:i-1}, P) = \text{softmax}(u^i) \quad (28)$$

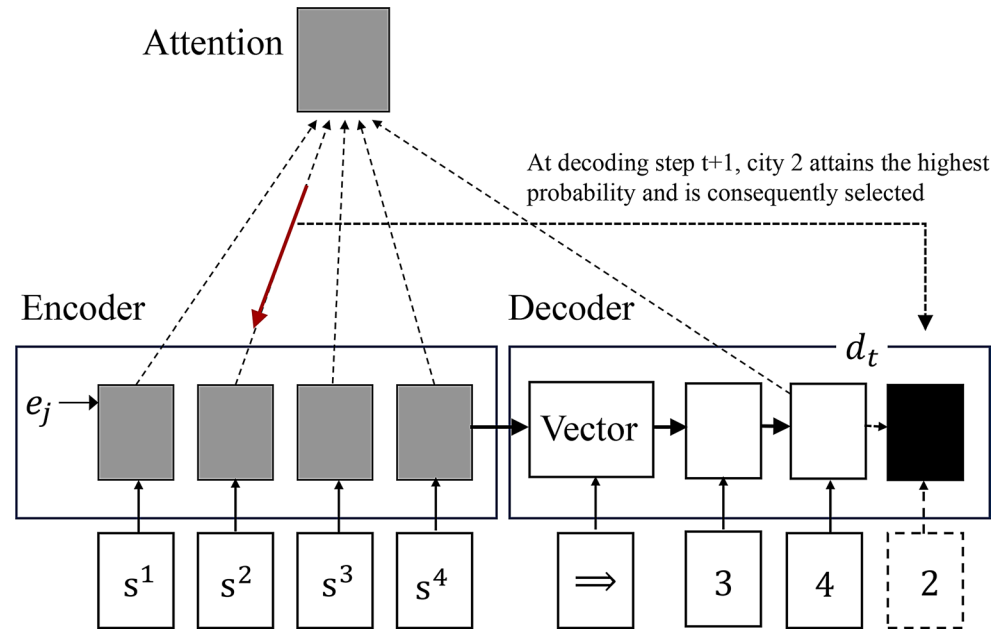
The node with the highest probability is selected as the current output, iterating until a complete solution is generated.

This approach has been successfully applied to RPs like the TSP and VRP, spawning several enhanced architectures. Pointer-generator networks incorporate generation capabilities to handle out-of-vocabulary (OOV) issues [155]. Transformer-based PtrNets improve long-sequence modeling efficiency. Graph pointer networks (GPNs) introduce graph structures to enhance complex constraint handling [156]. Integration with RL enables unsupervised optimization. PtrNets establish an end-to-end neural paradigm for COPs.

3.2.2 Convolutional Neural Networks

CNNs are a type of DNN specifically designed to process data with a grid-like structure, initially gaining widespread use in computer vision tasks. The core structure of a CNN consists of convolutional layers, pooling layers, and fully connected layers. Convolutional layers perform local feature extraction using convolutional kernels, significantly reducing the number of parameters through weight sharing and sparse connections. Pooling layers downsample feature

Fig. 2 Pointer network



dimensions and enhance robustness, while fully connected layers integrate high-level features for final classification or regression. As network depth increases, CNNs can learn hierarchical feature representations, from low-level edges to high-level semantics.

CNNs offer several advantages. First, parameter sharing and sparse connections effectively alleviate the high-dimensional parameter problem inherent in traditional fully connected networks. Second, the hierarchical feature learning approach provides strong expressive power for handling complex structured data. Third, the convolutional and pooling mechanisms endow the model with translational invariance and robustness, making CNNs perform exceptionally well in fields such as image and speech recognition. In recent years, researchers have increasingly applied CNNs to solve COPs, particularly in tasks like scheduling, path planning, and traffic network prediction. By transforming optimization problem instances into matrix or image formats, CNNs efficiently capture spatial local dependencies, providing high-quality feature representations for subsequent optimization and decision-making.

To further enhance performance, CNNs have evolved into various improved architectures. For example, the visual geometry group network significantly enhanced feature extraction ability by increasing network depth. Residual networks introduced residual connections, effectively addressing the vanishing gradient problem in deep networks. Densenet strengthened feature reuse and gradient flow through dense connections. Additionally, the inception network leveraged multi-scale convolutional kernels to enhance the representation of features with different receptive fields. These improvements not only boosted model

training efficiency and generalization ability but also provided rich modeling tools for CNNs in COPs and broader intelligent optimization scenarios.

3.2.3 Transformer

The Transformer model, based on self-attention mechanisms, has revolutionized sequence modeling paradigms. Its core architecture consists of encoder and decoder stacks (Fig. 3(1)), each composed of multiple identical layers. Encoder layers contain multi-head self-attention and feed-forward networks, both with residual connections and layer normalization; decoder layers additionally incorporate masked multi-head attention to ensure autoregressive generation depends only on previously generated content [50].

There are multiple methods for measuring the relevance between input vectors in self-attention, such as the dot-product method [50, 157] and the additive method [158]. Among these, the dot-product method is the most commonly used and is the method employed in the Transformer.

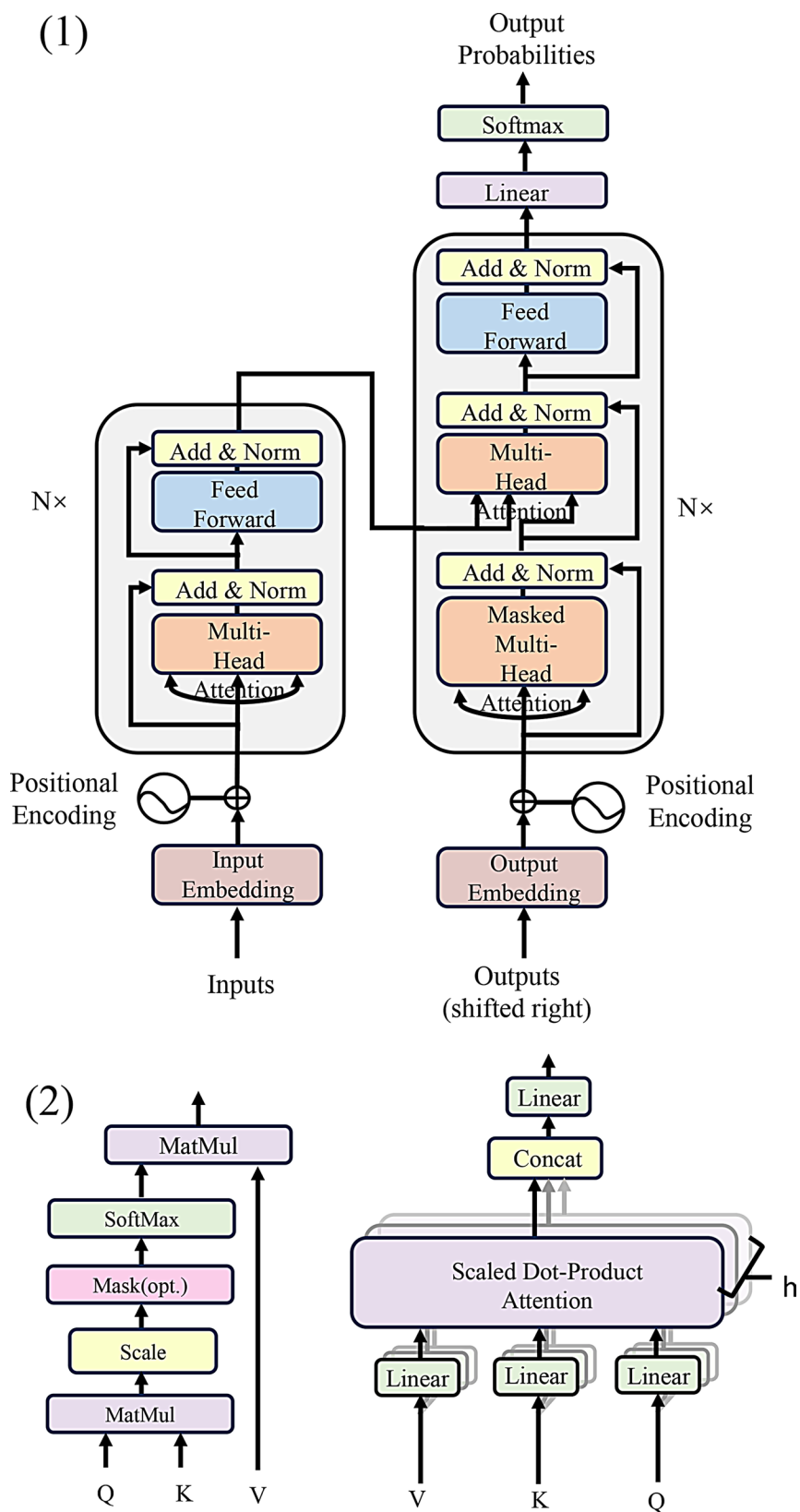
The self-attention mechanism (Fig. 3, (2)left) computes dependencies within input sequences through query, key, and value matrices.

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (29)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (30)$$

The scaling factor $\sqrt{d_k}$ stabilizes training, while softmax normalization produces attention weights.

Fig. 3 (1) Transformer model (2) (left) Scaled dot-product attention. (right) Multi-head attention consists of several attention layers running in parallel



Multi-head attention (Fig. 3, (2)right) performs h parallel self-attention computations.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (31)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (32)$$

This design enables the model to jointly attend to information from different representation subspaces.

Compared to RNNs and CNNs, Transformers support parallel sequence computation, significantly improving training efficiency, and can directly model long-range dependencies. In combinatorial optimization, Transformer-based variants (e.g., Pointer Transformer, Graph Transformer) demonstrate strong potential in routing and scheduling tasks by integrating pointing mechanisms and graph structural information, offering novel neural approaches for complex COPs.

3.2.4 Graph Neural Networks

Graph neural networks (GNNs) are designed for graph-structured data, enabling representation learning under adjacency constraints while leveraging higher-order dependencies. The graph convolutional network (GCN) [159], a foundation of modern spectral GNNs, stacks message-passing, aggregation, and update steps. With adjacency A and features $H^{(l)}$, each layer aggregates neighborhood information to produce $H^{(l+1)}$. This naturally supports inductive and transductive settings. Per-layer complexity is $O(|E|d)$ (hidden dimension d) using sparse multiplication and mini-batch sampling; residuals and normalization improve deep stability. For COPs, GNNs encode constraints and dependencies into graphs, compressing states into differentiable representations for scoring, selection, or feasibility checks in scheduling, routing, and allocation.

GCN approximates spectral convolution with first-order filters and symmetric normalization after adding self-loops:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}\right),$$

where $\tilde{A} = A + I$ and $\tilde{D}$ its degree matrix. This transforms frequency filtering into local smoothing, enabling large-scale sparse implementation. On homophilic graphs (Cora, Citeseer, Pubmed), GCN achieves high accuracy and low latency. In CO, it serves as a fast feasibility scorer or real-time state encoder. Deep stacking may cause over-smoothing; shallow stacks or skip connections mitigate this.

To improve structural discrimination, the graph isomorphism network (GIN) [160] replaces linear propagation with MLPs and uses sum aggregation:

$$h_v^{(k)} = \text{MLP}^{(k)}\left((1 + \epsilon) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}\right),$$

with learnable/fixed ϵ and neighborhood $\mathcal{N}(v)$. Sum aggregation avoids collisions of mean/max aggregation, enhancing expressiveness for graph-level decisions and substructure-sensitive tasks (e.g., scoring routes or packings).

For flexible aggregation, the graph attention network (GAT) [161] assigns adaptive weights via learnable attention:

$$e_{uv} = a^\top [Wh_u \parallel Wh_v], \quad \alpha_{uv} = \text{softmax}_u(e_{uv}),$$

with multi-head aggregation:

$$h_v^{(k)} = \big\|_{m=1}^M \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv}^{(m)} W^{(m)} h_u^{(k-1)}\right).$$

GATs are robust to noisy edges and heterogeneous neighborhoods, assigning different strengths to relation types. In CO, they can focus on critical action/constraint edges, e.g., precedence bottlenecks in JSSP or distributed allocation.

In summary: GCN emphasizes efficient propagation and scalability for large graphs requiring low-latency inference. GIN focuses on expressiveness for structure-sensitive optimization. GAT improves selective aggregation and robustness for heterogeneous or dynamic relations. In CO applications (job scheduling, VRP, coloring, bin packing, resource allocation), each variant has its strengths. With deeper propagation control, hierarchical pooling, and coupling with search, GNNs are becoming core components for NP-hard optimization.

3.3 Large Language Models

The core of LLMs is to model language autoregressively by minimizing negative log-likelihood $\max_{\theta} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$, learning $p_{\theta}(x_t | x_{<t})$ [50]. Transformer architectures provide strong in-context learning capabilities [52]. Empirically, scaling laws relate model size, data, and compute [162, 163]. Advantages include a unified text interface, strong cross-task transfer, fast adaptation with few examples, and extensibility to reasoning, tool use, and planning.

This motivates applying LLMs to CO by describing the problem in natural or structured language and then using LLM reasoning to directly generate solutions or assist classical algorithms.

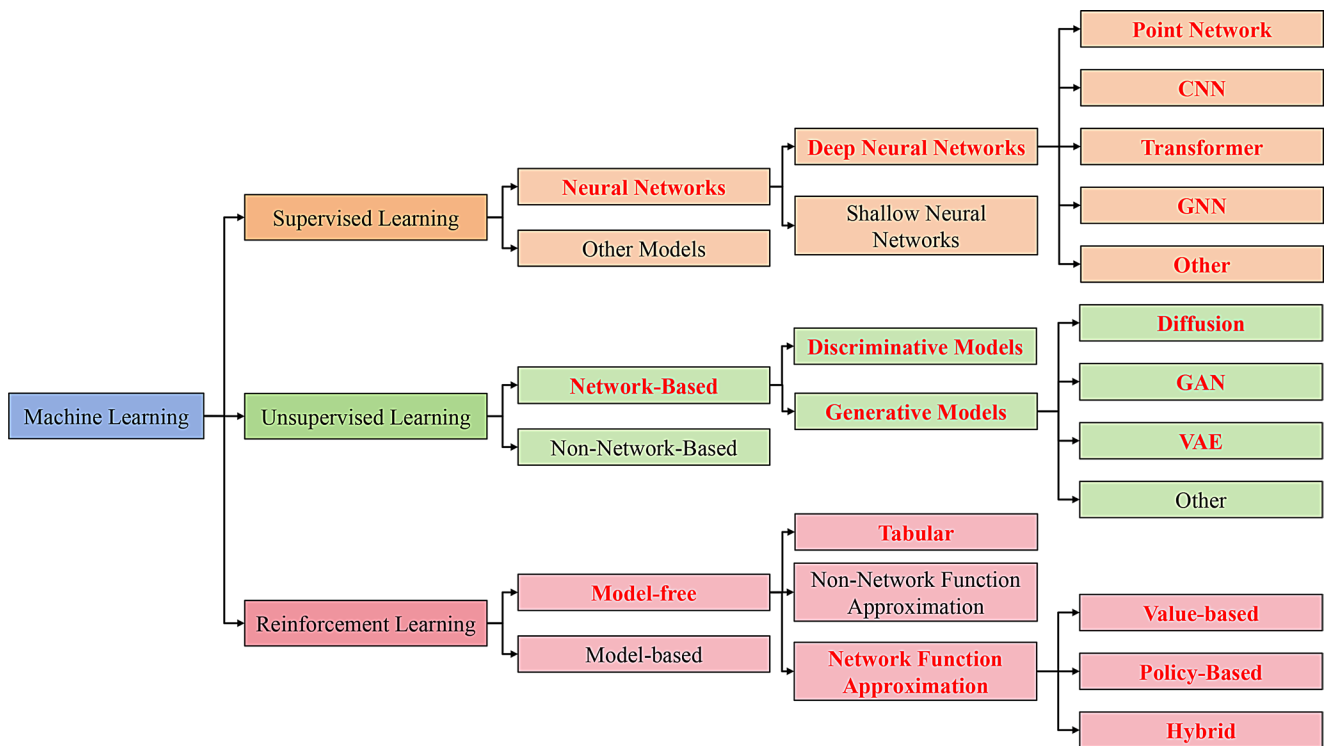


Fig. 4 ML methods applied to solving COPs

Currently, LLM for COPs research follows two technical paths. The first is zero-shot and few-shot prompting as an early exploration approach, where the problem description together with instance data, forms the prompt for a general LLM that directly outputs a feasible solution. This approach shows potential but often lacks guarantees of feasibility and optimality because the models have not explicitly learned the underlying mathematical structure and constraints.

To address this, research has shifted toward task-specific fine-tuning and guided generation. High-quality datasets of problem and optimal solution pairs are constructed using exact solvers or strong heuristics. Instruction fine-tuning is then applied so that the model learns to output valid solution sequences, such as a TSP visit order or a maximum independent set, from structured descriptions. During inference, guided decoding with external constraint checkers or value functions is used to restrict generation to feasible and promising regions of the search space.

Available evidence suggests that purely zero-shot prompting has a performance ceiling. Future directions combine LLM reasoning with domain knowledge by using larger and higher-quality task datasets for instruction fine-tuning and RL; building hybrid architectures where LLMs act as planners or reasoning engines together with search and MIP solvers for exact execution and verification; and employing chain-of-thought reasoning to expose intermediate steps. In this way, LLMs may develop into tools that

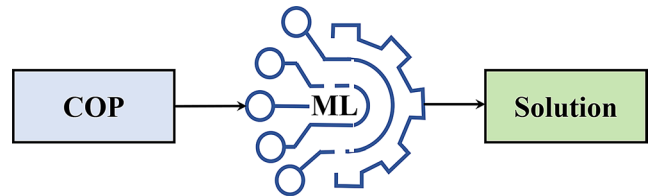


Fig. 5 End-to-End learning-based solvers

understand complex constraints, reason deeply, and solve real combinatorial optimization tasks.

3.4 Machine Learning Driven Paradigms for Solving Combinatorial Optimization Problems

As illustrated in Fig. 1, the solution process for COPs comprises modeling, feature extraction, solving algorithm, verification, and explanation. The solving algorithm stage directly determines solution quality and computational efficiency, making it the core integration point between ML and COPs. Building on the ML foundations introduced above and guided by Figs. 4, 5, 6, 7, this subsection summarizes the main paradigms by which ML has been combined with COPs in recent years.

Figure 4 organizes ML hierarchically from a methodological perspective. At the top level, ML is divided into SL, UL, and RL. These are further refined into specific model families, including deep and shallow neural networks,

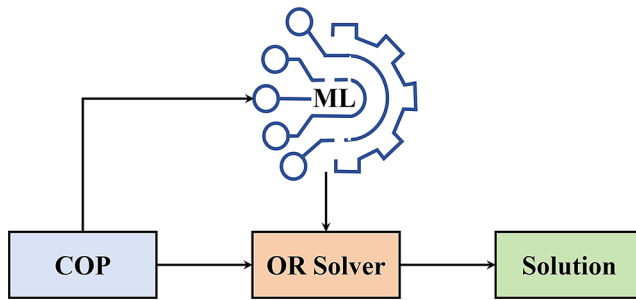


Fig. 6 ML-OR collaborative solving

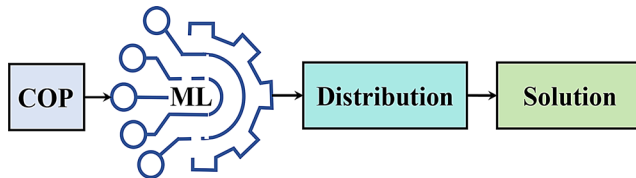


Fig. 7 Solution-distribution modeling via generative models

GNNs, Transformers, GANs, VAEs, diffusion models, and tabular as well as function-approximation-based RL methods. LLMs, as large-scale pretrained generative models built upon Transformers, are closely related to several of these branches. The modules highlighted in red correspond to the models and methods discussed in subsequent chapters.

On this basis, Figs. 5, 6, 7 summarize three representative paradigms for using ML to solve COPs:

- Figure 5: **End-to-end learning-based solvers**, where an ML model directly maps a COP instance to a solution;
- Figure 6: **ML-OR collaborative solving**, where OR solvers handle COPs while ML provides strategic or parametric guidance;
- Figure 7: **Solution-distribution modeling via generative models**, where an ML model learns the probability distribution of high-quality solutions and generates candidates through distributional sampling and filtering.

These paradigms are not mutually exclusive but closely aligned with SL, UL, RL, and LLMs, and often appear in hybrid form in practice. The remainder of this subsection briefly discusses each paradigm and clarifies its relation to subsequent chapters.

3.4.1 End-To-End Learning-Based Solvers

As shown in Fig. 5, the most straightforward idea is to regard a COP as an input–output mapping problem. Given a problem instance, one trains an ML model to directly output a complete solution or its construction sequence. Here, the ML model itself serves as the solver, while traditional OR

algorithms are used only in auxiliary stages such as generating training data or post-processing.

From the perspective of learning paradigms, end-to-end solvers mainly fall into two categories:

- **SL-based:** Networks such as PtrNets, GNNs, and Transformers are trained on high-quality labeled solutions to predict solution sequences directly. These methods enjoy good convergence and can approach labeled-data quality on the training distribution, but rely on traditional solvers for label generation and generalize poorly to out-of-distribution instances.
- **DRL-based:** Solution construction is modeled as sequential decision-making, with the objective function (e.g., negative route length) serving as the reward signal. DRL eliminates dependence on explicit labels and can explore novel strategies, but its training tends to be less stable and more sensitive to reward design and hyperparameters.

The end-to-end paradigm offers fast inference once trained, making it suitable for large-scale repeated solving or real-time scenarios, while automatically learning complex heuristic rules from data. However, guaranteeing feasibility, optimality, interpretability, and robust cross-scale generalization remain open challenges, discussed in detail in Sects. 4 and 5.

3.4.2 ML-OR Collaborative Solving

In contrast to directly replacing the solver with ML (Fig. 5), the paradigm in Fig. 6 preserves the overall OR solver framework and employs ML to optimize key internal decisions. The solver retains core structures such as branch-and-bound, local search, or hyper-heuristics, while ML serves as a decision evaluator or policy generator.

Within this framework, SL, RL, and certain UL methods are widely used. Representative collaboration forms include:

- **With exact algorithms:** SL/RL learns branching variable selection, cutting-plane strategies, or node ordering to reduce search tree size or improve pruning efficiency;
- **With heuristics and metaheuristics:** ML predicts search starting points, candidate move quality, operator selection probabilities, or algorithmic parameters (e.g., SA temperature, neighborhood size), reducing tuning costs and accelerating convergence;
- **With hyper-heuristic frameworks:** ML learns which low-level heuristic to select in each state, enabling automatic heuristic composition and cross-problem transfer.

A major advantage of ML–OR collaboration is partial inheritance of traditional methods’ feasibility and optimality guarantees. This paradigm also facilitates incorporating domain knowledge: experts retain familiar frameworks and delegate only hard-to-tune strategic components to ML. However, it requires deep understanding of solver internals and suitable feature/interface design, and generating internal-state training data can be costly.

3.4.3 Solution-Distribution Modeling via Generative Models

The third paradigm (Fig. 7) estimates the probability distribution of high-quality solutions over the solution space using generative models. During training, high-quality solutions from traditional solvers or heuristics fit this distribution. At inference, final solutions are obtained through sampling, filtering, and optional local improvement.

This paradigm aligns naturally with UL, especially deep generative models. Typical approaches include:

- **VAEs:** mapping high-quality solutions into a continuous latent space for interpolation or search, then decoding back to discrete solutions;
- **GANs:** adversarially training a generator to produce candidates approximating the true solution distribution;
- **Diffusion models:** defining noise–denoising processes over graph- or sequence-structured objects, iteratively growing feasible solutions from random initial samples.

Compared with autoregressive models that construct solutions step by step, generative methods can produce diverse candidates in a single run, mitigating error accumulation and providing richer alternatives for robust optimization and interactive decision making. However, challenges remain in accurately covering tail high-quality solutions, ensuring feasibility, and controlling training costs. Consequently, many studies combine this framework with OR solvers via generate-and-filter/repair/local-search pipelines.

3.4.4 The Role of Large Language Models Across Paradigms

Although Figs. 5, 6, 7 do not dedicate a separate block to LLMs, LLMs can naturally be embedded into all three paradigms. As large generative models, they can serve as end-to-end optimizers generating solutions from natural-language or structured descriptions. They can also act as programmers or collaborators, automatically generating heuristic code, orchestrating external OR solvers, designing search operators, or explaining solver outputs.

At present, LLM-based COP solving remains exploratory, with performance depending heavily on prompt

design, tool availability, and problem scale, and lacking systematic theoretical guarantees. Within the 2020–2026 scope of this survey, LLMs are better viewed as a complement to the three paradigms above rather than a replacement for traditional ML models and OR algorithms. Section 7 provides a systematic overview of LLM-based methods and their roles as optimizers, programmers, collaborators, and modeling/explanation tools.

3.4.5 A Comparative Framework for ML-Based COP Methods

The preceding discussion surveys ML for COPs from two complementary perspectives: the learning paradigm (SL, UL, RL, LLMs) and the solution paradigm (end-to-end solvers, ML–OR collaboration, generative modeling). However, a taxonomy alone is insufficient for rigorous cross-paradigm comparison. The value of a method depends on its performance across varying problem scales, data regimes, constraint structures, and deployment requirements. Therefore, we establish a unified comparative framework to characterize the key properties of ML-based COP methods, associating each dimension with a coarse rating scale from 0 to 5.

(1) Solution Quality (Qual.) measures the objective value achieved, including the gap relative to optimal solutions, BKS, or SOTA methods.

- 0 (☆): Only rough feasible solutions, far inferior to mainstream methods.
- 1 (★): Low quality, serving as a fast initial solution or weak baseline.
- 2 (★☆): Moderate quality with a notable gap to high-performance methods.
- 3 (★★): Good quality, competitive on most benchmarks.
- 4 (★★☆): High quality, typically close to BKS or strong baselines.
- 5 (★★★): Optimal or near-optimal performance, the strongest level within its paradigm.

Remark: For exact algorithms, ★★★ implies provably optimal solutions; for inexact methods, it indicates empirically near-optimal or SOTA performance.

(2) Scalability (Scal.) measures the ability to handle large-scale instances, using TSP node count as a reference anchor for coarse-grained comparison.

- 0 (☆): Very small instances only (e.g., TSP < 20).
- 1 (★): Small instances (e.g., TSP < 100).
- 2 (★☆): Small-to-medium (e.g., TSP 100 – 1000).
- 3 (★★): Medium (e.g., TSP 1000 – 5000).
- 4 (★★☆): Relatively large (e.g., TSP 5000 – 10000).
- 5 (★★★): Very large (e.g., TSP > 10000).

(3) Generalization (Gen.) measures the ability to maintain performance under changes in problem size, instance distribution, or problem setting, encompassing cross-size, cross-distribution, and cross-problem generalization.

- 0 (☆): Almost incapable of transfer; significant degradation with slight changes.
- 1 (★): Works only on instances very close to training conditions.
- 2 (★☆): Some adaptability to limited changes.
- 3 (★★): Basically usable under moderate changes.
- 4 (★★☆): Relatively stable across scales or distributions.
- 5 (★★★): Strong generalization, robustly adapting to changes in scale, distribution, and problem variants.

(4) Computational Efficiency (Eff.) measures online inference or solving speed at deployment, as opposed to training time.

- 0 (☆): Extremely slow, unable to meet online or large-scale requirements.
- 1 (★): Slow, suitable only for offline or small instances.
- 2 (★☆): Moderate, usable within a given time budget.
- 3 (★★): Fast, supporting efficient solving of conventional instances.
- 4 (★★☆): Very fast, suitable for most online or near-real-time scenarios.
- 5 (★★★): Extremely fast, enabling real-time or near-instant solving.

(5) Data Independence (Data Inde.) measures the degree of reliance on high-cost external supervision (e.g., optimal labels, expert trajectories). Higher scores indicate less dependence.

- 0 (☆): Strongly dependent on optimal or high-quality labeled data.
- 1 (★): Dependent on exact or high-cost expert solutions.
- 2 (★☆): Dependent on solver trajectories, expert demonstrations, or high-quality candidate sets.
- 3 (★★): Dependent on auxiliary data such as heuristic samples or search process information.
- 4 (★★☆): Requires only easily obtainable data (random instances, self-supervised signals, or environment interactions).
- 5 (★★★): Requires almost no training data or supervision.

(6) Feasibility (Feas.) measures the algorithm's inherent ability to guarantee constraint satisfaction without additional repair, projection, or filtering.

- 1 (★): No direct feasibility guarantee; outputs frequently violate constraints, requiring explicit repair or post-processing.
- 3 (★★): Partial feasibility consideration; may require penalty terms, repair operators, or specialized decoding under complex constraints.
- 5 (★★★): Feasibility directly guaranteed by algorithm structure; solutions naturally satisfy hard constraints.

(7) Theoretical Guarantee (Guar.) measures the strength of theoretical support, including optimality guarantees, approximation ratios, convergence analysis, and verifiable bounds.

- 1 (★): No rigorous guarantees; relies primarily on empirical performance.
- 3 (★★): Some theoretical analysis (local convergence, conditional bounds, or weak guarantees in specific settings).
- 5 (★★★): Strong theoretical support with rigorous optimality proofs, approximation ratios, or global convergence.

Table 5 summarizes algorithms from a macro perspective. The qualitative ratings are not strict universal rankings, as results vary across problem classes, benchmarks, and implementations. Rather, they reflect general trends observed in recent research (2020–2026) and should be regarded as a concise analytical map complementing the preceding taxonomy.

From the classical optimization perspective, **exact algorithms** remain the strongest reference for solution quality, feasibility, and theoretical guarantees, with primary weaknesses in scalability and computational cost on large NP-hard instances. **Approximation algorithms** are generally more scalable and efficient than exact methods with stronger theoretical support than heuristics, yet their empirical quality is often surpassed by problem-specific heuristics or learning-enhanced methods. **Traditional heuristics, meta-heuristics, and hyper-heuristics** trade formal guarantees for practical flexibility: heuristics offer speed and simplicity; metaheuristics improve quality through broader exploration at additional runtime cost; hyper-heuristics further enhance adaptability by selecting or generating low-level heuristics.

End-to-end SL methods offer high inference efficiency but exhibit strong label dependence and limited generalization. When training data coverage is sufficient and test distributions match training conditions, these methods often approach strong heuristics on standard benchmarks. However, performance degrades significantly under scale extrapolation or distribution shift. Feasibility typically requires specialized decoders, masking mechanisms, or post-processing.

Table 5 Unified comparative framework for combinatorial optimization approaches

Approach	Typical forms	Qual.	Scal.	Gen.	Eff.	Data Inde.	Feas.	Guar.
Exact algorithms	Branch-and-bound, branch-and-cut, dynamic programming, constraint programming, exhaustive search with pruning	★★★	★	★★★	★	★★★	★★★	★★★
Approximation algorithms	LP relaxation with rounding, primal-dual methods, provable approximation schemes	★★	★★☆	★★★	★★☆	★★★	★★★	★★★
Heuristics	Greedy construction, local search, dispatching rules, problem-specific repair/improvement	★★	★★☆	★★☆	★★☆	★★★	★★	★
Metaheuristics	Simulated annealing, tabu search, genetic algorithms, ant colony optimization, large neighborhood search	★★	★★	★★	★☆	★★★	★★	★
Hyper-heuristics	Heuristic selection, heuristic generation, adaptive search control across heuristic sets	★★	★★☆	★★☆	★★	★★★	★★	★
End-to-end SL	PtrNet/GNN/Transformer direct solvers trained on labeled solutions	★★☆	★★	★	★★★	☆	★★	★
End-to-end RL	Constructive DRL policies trained from reward signals	★★☆	★★☆	★☆	★★★	★★☆	★★	★
ML–OR (Exact)	Learned branching, cuts, node selection, warm starts inside exact solvers	★★★	★★	★★	★★	★☆	★★★	★★
ML–OR (Heuristic)	Learned destroy/repair, local search guidance, neural improvement heuristics	★★☆	★★☆	★★	★★	★☆	★★	★
UL Generative	Diffusion, VAE, GAN, autoregressive sampling for candidate solutions	★★☆	★★	★★	★☆	★☆	★	★
UL Discriminative	Objective-driven GNNs, energy minimizers, direct graph-to-solution mappings	★★☆	★★☆	★★	★★★	★★★	★	★
LLMs as optimizers	LLMs directly generating solutions or decision sequences from prompts	★☆	★☆	★★	★☆	★★★	★	★
LLMs as programmers	Heuristic/code generation, solver scripting, formulation assistance	★★	★★	★★	★★	★★★	★★	★★
LLMs as collaborators	Solver orchestration, decomposition, interactive optimization, explanation	★★	★★☆	★★	★★	★★★	★★	★★

End-to-end RL methods directly optimize construction policies via environmental rewards, avoiding dependence on optimal labels. After training, they achieve high inference efficiency suitable for online decision-making and demonstrate strong competitiveness on classic CO tasks. Generalization is generally stronger than pure SL but may still degrade under cross-scale or cross-distribution testing. Feasibility is maintained through stepwise construction, action masking, or penalty terms.

ML–OR collaborative exact solving methods combine high solution quality, strong feasibility, and robust theoretical properties. ML is embedded into components such as branch-and-bound, cut generation, or variable selection, preserving native hard constraint handling and solution quality close to exact algorithms. Scalability and efficiency, while improved over pure exact methods, remain inferior to purely neural inference. Training often requires supervision from solver trajectories, yielding weaker data independence.

ML–OR collaborative heuristic methods combine data-driven adaptability with classical heuristic search flexibility, guiding neighborhood selection, destroy-repair operations, or candidate filtering within existing frameworks. This yields stronger feasibility control and practical robustness than pure end-to-end methods, with highly competitive solution quality on complex-constraint and medium-to-large

instances. Generalization typically outperforms label-dependent end-to-end SL but remains influenced by training data and the underlying heuristic structure. Theoretical guarantees are generally weak, yet these methods often achieve favorable trade-offs among quality, efficiency, and robustness in practice.

UL generative methods (diffusion models, VAEs, GANs, autoregressive generators) learn candidate solution distributions without requiring optimal labels, offering advantages in data independence over SL. When effectively capturing high-quality solution distributions, they rapidly generate batches of competitive candidates suitable for combination with re-ranking or local improvement. However, feasibility guarantees are weak—generated solutions often require filtering or repair—and label-dependent methods such as DIFUSCO typically achieve lower optimality gaps than purely autoregressive approaches.

UL discriminative methods learn direct mappings from instances to decisions through objective-driven GNNs or energy function optimization. Inference is typically more efficient than generative sampling, and data independence can be high since optimal labels are not required. However, these methods generally lack rigorous constraint satisfaction mechanisms and exhibit significant instability under cross-task or strong distribution shift scenarios.

LLMs as optimizers offer flexibility and generality through natural language interfaces and cross-problem descriptive capabilities. However, they remain significantly weaker than dedicated solvers in solution quality, scalability, feasibility, and theoretical guarantees. On large-scale structured problems with strict constraints, results often suffer from instability and constraint violations. This direction is currently best viewed as exploratory, with value primarily in interactivity and generality.

LLMs as programmers do not solve problems directly but generate heuristic code, solving scripts, or formulation schemes. Solution quality depends heavily on external solvers invoked by the generated programs. Compared to direct LLM optimization, this paradigm shows greater potential in feasibility and theoretical properties by embedding outputs into traditional frameworks. Its main advantages lie in lowering development thresholds and accelerating algorithm prototyping.

LLMs as collaborators orchestrate solvers, recommend strategies, and coordinate algorithmic components. By naturally invoking external exact solvers or heuristic methods, scalability and adaptability often surpass direct LLM solving, while feasibility and quality benefit from the scheduled modules' capabilities. This paradigm is likely to become an effective near-term application form, serving as a scheduler and knowledge enhancement layer for optimization systems rather than an independent solving core.

3.4.6 Summary and Relation to Subsequent Chapters

The integration of ML and COPs can be understood along two dimensions: the family of learning paradigms (SL, UL, RL, LLMs and their associated architectures) and the three solution pipelines (end-to-end solvers, ML–OR collaborative solving, and generative solution-distribution modeling). Most existing methods can be categorized within this taxonomy, while many cutting-edge works achieve better performance through paradigm hybridization.

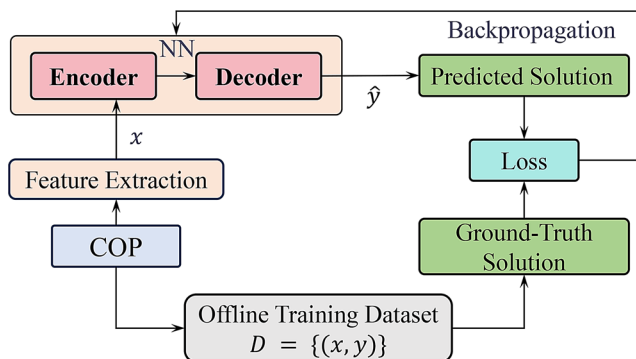


Fig. 8 SL as End-to-End learning-based solvers

The subsequent sections are organized around this unified perspective. Section 4 focuses on SL in end-to-end and ML–OR collaborative solvers (Figs. 5 and 6). Section 5 covers Q-learning and DRL in COPs, including end-to-end RL solvers and their integration with heuristics, meta-heuristics, and hyper-heuristics. Section 6 centers on UL and generative models within the framework of Fig. 7. Section 7 analyzes LLM applications across different COP classes. Through this structure, we aim to provide a comprehensive map of ML-driven combinatorial optimization methods while objectively presenting both opportunities and challenges.

4 Supervised Learning for Combinatorial Optimization Problem

The application of SL to COPs represents a paradigm shift from rule-based heuristic design to data-driven, adaptive solving. This section reviews recent advances in using SL as a core component for solving COPs, focusing on two primary integration paradigms. The first paradigm, *end-to-end SL solvers*, treats COPs as sequence-to-sequence or graph-to-structure mapping tasks, where neural networks directly generate complete solutions. The second paradigm, *SL–OR collaborative solving*, embeds learned models as intelligent components within traditional optimization frameworks—such as exact algorithms, heuristics, metaheuristics, and hyper-heuristics—to enhance their decision-making efficiency and solution quality. By examining representative architectures like PtrNets, GNNs, and transformers, as well as their synergistic combinations with classical operations research methods, this section provides a comprehensive overview of how SL is reshaping the landscape of combinatorial optimization, balancing computational efficiency, generalization capability, and solution optimality.

4.1 End-To-End Learning-Based Solvers

In the research of using SL to solve COPs, an important development is the use of SL as an end-to-end optimizer, directly learning the problem structure and solution strategy, bypassing traditional manually designed heuristics and achieving data-driven optimization. The core idea is to capture the topological information of input graphs through a NN encoder and generate feasible solution sequences through the decoder. This section mainly focuses on training under a SL framework, and Fig. 8 presents the commonly adopted framework of this category.

4.1.1 Pointer Networks

PtrNets pioneered end-to-end neural combinatorial optimization. The model innovatively employs attention as a pointer mechanism, directly mapping the decoding process to input elements and overcoming fixed output dictionary limitations. It employs an encoder-decoder architecture based on RNNs, where selection probabilities are computed through attention scores during decoding. Supervised training enables direct solution sequence generation, achieving remarkable results on TSP and establishing a new research direction in COPs [53].

Later [164], extended PtrNets to the 0–1 knapsack problem, using the pointer selection mechanism to directly generate the item selection sequence. Gu and Yao further applied this network to the maximum clique problem [165] and unconstrained binary quadratic programming (UBQP) [166], demonstrating the transferability of the architecture beyond RPs.

In multi-connectivity mobile edge computing task scheduling [167], proposed the end-to-end method *PtrTasking*, which takes base-station computing power, link states, inter-server communication rates, and task dependency information as inputs, and directly infers task-to-server scheduling decisions through a PtrNet-based architecture. The method uses search-generated feasible schedules as supervision signals, and introduces virtual base station padding and masking mechanisms to handle variable input dimensions, thereby jointly reducing latency and improving service reliability under time-varying MEC environments.

For permutation flow shop scheduling [168], formulated the problem as a mixed integer programming model and used CPLEX to generate optimal job sequences as labeled samples. Given the processing-time vectors of jobs on machines, the PtrNet encoder-decoder architecture learns the conditional probability of the output permutation and directly predicts the job processing order. A validity constraint is further imposed during decoding to avoid repeated job selections.

In urban logistics route prediction [169], introduced *Hyper4Route*, which integrates courier profiles, urban contextual information, and bi-layer hypergraph representations into a unified route prediction framework. The model uses real courier trajectory sequences as supervision signals, captures high-order correlations among pending packages, and employs a pointer decoder to directly output the predicted delivery route while constraining invalid package selections during decoding. For dynamic MEC task management [170], proposed a Pointer Network-based scheduling model for delay-sensitive task execution. The model adopts an LSTM-based encoder-decoder structure with dynamic masking, enabling it to directly decode the task execution sequence at inference time. Experimental results show that the learned scheduler can reduce task drop ratios and waiting times while maintaining lower inference latency than integer and binary programming baselines.

4.1.2 Graph Neural Networks

In COPs, traditional SL models are limited by fixed-dimensional vector inputs, making it difficult to effectively represent relationships and constraints. GNNs abstract tasks and resources as nodes and aggregate local and global information through message passing, making them the mainstream framework for solving such problems.

As illustrated in Fig. 9 [54], introduced a GNN-based solver for TSP. The framework employs graph convolutional layers to learn node and edge representations, producing an edge selection heatmap that is converted into a valid tour via parallel search. Using optimal solutions from concorde as supervision, the model is trained to predict edge inclusion probabilities through binary classification. This non-autoregressive design supports parallel inference, enhancing computational efficiency compared to sequential PtrNets. Nevertheless, while GNNs effectively capture graph topology, their local message-passing paradigm presents limitations in scaling to large instances and modeling global dependencies.

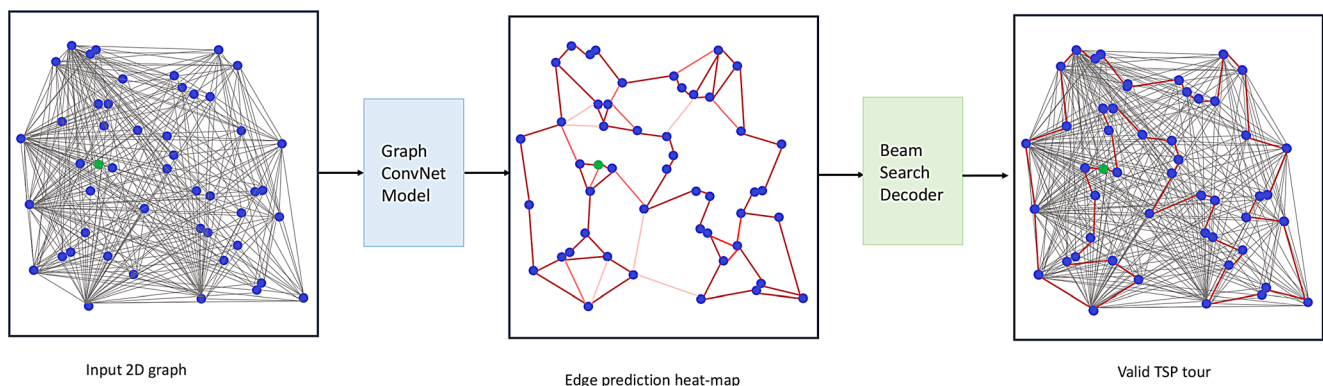


Fig. 9 GNN-based TSP solving framework

In resource allocation for device-to-device (D2D) communication [171], proposed an end-to-end optimization framework based on GNNs. The framework represents the network as a directed graph, with communication links as nodes and interference relationships as edges. The input includes node and edge features. The model utilizes a combination module of CNN and DNNs to directly output D2D activation vectors and D2D–cellular user matching schemes, achieving closed-loop optimization from graph features to resource allocation decisions. During the SL phase, the near-optimal solution generated by constrained cross-entropy is used as the label, and the classification cross-entropy is minimized to reduce the difference between the predicted and reference solutions. [172] propose Scale-Net, a hierarchical neural solver that integrates a U-Net architecture into a multi-task VRP framework. By explicitly capturing multi-scale structural patterns through coarse-to-fine representation, Scale-Net enhances the decoder’s topological awareness while reducing encoder computational overhead.

Beyond path planning and wireless resource allocation, the end-to-end optimization framework of GNN is also applied to other combinatorial optimization tasks. [173] proposed a GNN model based on structure2vec graph embedding and a multi-layer classifier for D2D link scheduling. The method inputs link distance features and uses the near-optimal solution from the fractional programming link scheduling algorithm for SL. For cloud manufacturing and logistics scheduling [174], formulated task assignment and logistics coordination as a heterogeneous graph, where tasks and locations are represented as different types of nodes, and designed a heterogeneous GAT to directly generate scheduling decisions while jointly considering manufacturing and logistics costs. [175] combined bidirectional encoding representations with GAT for course scheduling and used an ILP module to ensure that credit constraints were satisfied.

4.1.3 Transformer

With the development of NN in combinatorial optimization, the transformer architecture, with its powerful global modeling and self-attention mechanisms, overcomes the limitation of fixed-dimensional inputs and effectively captures complex constraint relationships. Compared to CNNs and RNNs, Transformers offer stronger cross-scale transferability and computational parallelism, providing more effective intelligent solutions for COPs.

In the SL paradigm of neural combinatorial optimization [176], proposed the transformer-based knowledge distillation model, adaptive multi-distribution knowledge distillation (AMDKD), aiming to enhance the cross-distribution generalization performance of neural constructive models in path optimization tasks. Focusing on the VRP, this method

constructs multiple teacher models pre-trained on diverse task distributions and employs an autoregressive transformer to generate policy trajectories for each instance. An adaptive weighting strategy transfers knowledge from these multiple teachers to a lightweight student model. Trained using a soft label distillation mechanism, the approach achieves cross-task knowledge transfer without relying on a RL framework.

[177] proposed a general constraint modeling framework based on SL and transformer, constructing dedicated graph structures for JSSP and TSP. This framework combines the local message-passing mechanism of GNNs with the global attention mechanism of transformer. By using laplacian positional encodings and random walk structural encodings, the model enhances its ability to represent sequential and structural information, effectively modeling the complex relationships between operation sequences and machine constraints. In addition, the research team open-sourced four combinatorial optimization benchmark datasets, providing a scalable technical framework for general combinatorial optimization research.

To address the frequent occurrence of dynamic events such as machine failures and job rework in dynamic job shop scheduling [178], proposed an end-to-end model based on transformer and spatial pyramid pooling, aimed at making efficient scheduling decisions in dynamic scenarios. The model uses a 6-layer transformer encoder to capture priority dependencies between operations and machine resource conflicts, incorporates the spatial pyramid pooling layer to resolve feature dimension inconsistencies, and outputs 8 scheduling priority rules through the decoder.

4.1.4 Other Networks

In addition to Ptr-Net, GNN, and Transformer, researchers have also explored other network forms with structural priors for special constraint problems or non-path-based optimization tasks. For example [179], proposed Permutation-Invariant Networks, which model the fixed fleet constraints of the CVRP using set invariance, achieving end-to-end SL optimization. [180] applied DNNs in MEC task offloading, demonstrating the application of SL in a broader range of IP and SPs. Although these methods may not be optimal for path optimization tasks, they provide insights into structural priors and application extensions, proving the applicability of SL4COP in more general optimization scenarios.

4.2 SL-OR Collaborative Solving

In the research of applying SL to COPs, aside from the end-to-end solution generation paradigm, another important approach is to use NN as intelligent evaluators or

decision-makers, embedding them into key decision-making processes of traditional exact solvers. These methods aim to replace expensive or heuristic rules with data-driven models, significantly improving search efficiency and solution quality while maintaining the solver's interpretability and controllability. Figure 10 illustrates the typical workflow of this class of methods.

4.2.1 Supervised Learning Combined with Exact Algorithms

While traditional exact algorithms guarantee optimal solutions, their high computational overhead often limits their applicability in large-scale scenarios. End-to-end SL solvers offer a faster alternative but typically suffer from poor generalization and heavy reliance on high-quality training data. To bridge this gap, integrating SL to learn branching policies within an exact framework has emerged as a promising direction. This approach accelerates the solving process while maintaining solution quality and constraint feasibility, providing a robust strategy for complex industrial-scale SPs. [181] conducted pioneering work on integrating SL with exact optimization. Their study formalized the branch variable selection problem in MIP as a learnable prediction task. Using strong branching strategies as reference, they collected high-quality branching decisions as supervision signals. By extracting static and dynamic features including variable bounds, relaxation solution values, and constraint relations, they trained ML models to approximate strong branching decisions. During inference, this approach generates near-optimal branching decisions at a fraction of the computational cost of strong branching, significantly accelerating the branch-and-bound process while maintaining solution accuracy. This groundbreaking work established the methodological foundation for subsequent research

using GNNs to model MIP constraint structures and further enhance branch-and-bound efficiency.

Following the foundation laid by Khalil et al. on learning branching strategies, subsequent research has continued to expand this direction. For example [182], were the first to use GCN to model the variable-constraint structure of MIP and mimic strong branching strategies through SL, significantly improving generalization ability. [183] further proposed the MIP-GNN framework, realizing multi-task learning and deep integration with standard solvers. These studies collectively demonstrate the potential of DL as an intelligent decision evaluator in exact solving workflows, enabling traditional branch-and-bound and pruning mechanisms to operate in a more data-driven and adaptive manner.

Beyond its application in the MIP framework, the combination of DL and exact algorithms has also demonstrated significant effectiveness in complex SPs. To address the solving complexity of the JSSP [184], proposed a decomposition and coordination framework. This framework first decomposes the problem through shifted linear relaxation and utilizes the enhanced PtrNet to predict the solutions of subproblems. The model aggregates multiple parameters to generate starting cost features, thereby simplifying the input, while combining a masking mechanism to ensure solution feasibility. For complex instances, branch cutting algorithms further optimize the solution, and multiplier coordination is employed to achieve the global optimal solution.

For the dynamic complexity fluctuations in the FJSP [185], proposed the BCxCP framework. This method uses the heterogeneous GANs to apply BC for learning the CP solving logic, and combines the CP capability predictor to dynamically evaluate the complexity of subproblems. When the complexity score exceeds 0.98, the framework will switch to CP for exact solving.

In industrial applications, the constraint characteristics of different scheduling scenarios vary significantly, which motivates the integration of learning models with exact optimization methods. To meet the high real-time requirements of power scheduling for multi-electric aircrafts [186], proposed a two-stage solution framework. The first stage uses an ensemble DNNs model to solve integer variables, and the second stage simplifies the mixed integer quadratic programming problem into a quadratic programming problem to optimize continuous variables. For the steelmaking-continuous casting refining scheduling scenario [187], proposed a framework combining decision imitation and exact optimization. This framework uses a gated recurrent unit-DNN model to mimic engineer decisions, while optimizing the makespan using an MIP model.

In addition to directly assisting in scheduling decisions, SL can also be used to accelerate the solving process of exact

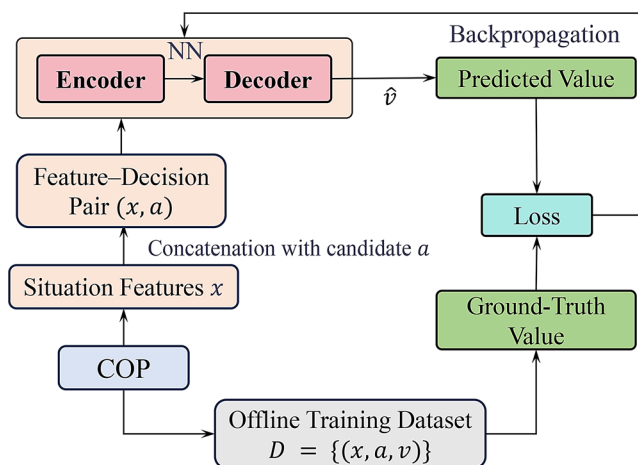


Fig. 10 SL-OR collaborative solving

algorithms. [188] proposed a branch-and-bound acceleration method based on graph convolutional neural networks (GCN), modeling the JSSP and discrete SP as mixed MILP, and utilizing the solving constraint integer programs solver to record optimal branching decisions as supervisory signals to train the GCN and accelerate the solving process.

4.2.2 Supervised Learning Combined with General Heuristic Algorithms

The integration of SL with classical heuristic methods enhances key steps such as candidate set construction, edge weight prediction, and subproblem partitioning, improving the guidance quality and search efficiency of heuristics. This approach retains the stability and scalability of heuristic frameworks while leveraging data and structural features, demonstrating strong performance in path planning and industrial scheduling tasks.

In path optimization problems, a representative work is the NeuroLKH model proposed by Xin et al. This study focuses on the candidate set generation phase of the lin kerrighan helsgaun (LKH) algorithm for the TSP [189]. Traditional LKH uses a heuristic measure based on α -nearness to filter candidate edges, but it lacks robustness under complex distributions. NeuroLKH utilizes a sparse-GNN to predict edge scores and node penalties, thus generating a better candidate edge set in a data-driven manner. The LKH algorithm then performs k-opt search. Experimental results show that this method significantly improves candidate set quality and outperforms the original LKH in solution performance across several TSP benchmark datasets, fully demonstrating the enhancement of heuristic mechanisms by SL. As shown in Fig. 11, the overall framework of NeuroLKH takes a sparse graph structure as input, learns the candidate edge selection strategy through a deep model, and seamlessly integrates with the LKH solving process.

After the success of the NeuroLKH framework, related studies further expanded on this idea from three aspects, such as generalization ability, scalability, and dynamic adaptability. [190] enhanced the cross-scale generalization

performance of LKH using GNN, enabling the model to perform more stably on TSP instances of different scales. [191] proposed a general SL framework that adapts to optimization instances with different distributions and scales, improving the model's transferability. [192] built a SL-driven adaptive sampling strategy for dynamic task scenarios, used to generate high-quality candidate solutions in real time to adapt to rapidly changing task constraints. [193] further proposed a hybrid framework combining GNNs with gradient boosting decision trees (GBDT) for heuristic optimization in integer programming, achieving higher solution efficiency and solution quality on large-scale instances. [194] proposes a GNN framework with random walk encoding and graph rewiring to improve solution prediction. The random walk mechanism captures multi-scale node features, while graph rewiring mitigates long-range dependency issues in sparse graphs. Experiments on TSP and CVRP show that Garnet generates accurate edge probability heatmaps, guiding search to near-optimal solutions with robust out-of-distribution generalization.

Beyond path optimization problems, the combination of supervised learning and traditional heuristic algorithms has also been applied to real-time scheduling and resource allocation. [195] addressed the partitioned scheduling problem in real-time systems by using graph-based supervised learning to imitate heuristic allocation decisions. In this method, FBBFFD, best-fit, and worst-fit heuristics are used to generate task-to-processor labels, while a GAT captures task attributes and interference relationships to learn partitioning decisions. During inference, the trained model directly predicts processor assignments, and the resulting partition is further checked by response time analysis to ensure schedulability.

4.2.3 Supervised Learning Combined with Metaheuristic Algorithms

Compared to general heuristic algorithms, metaheuristic algorithms have stronger global search capabilities. SL enhances or replaces key components of heuristic

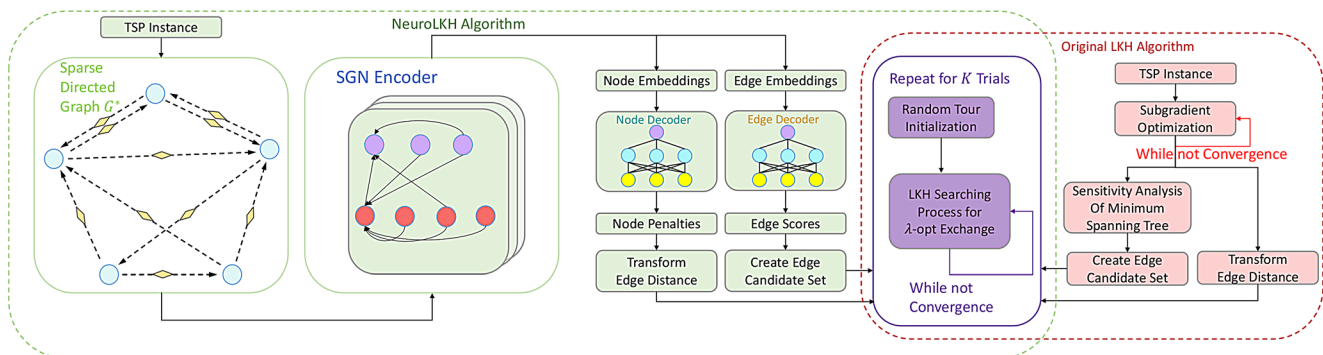


Fig. 11 NeuroLKH framework diagram

algorithms by learning data distributions and graph structural features, improving search efficiency while maintaining stability and scalability. This integration has shown significant advantages in path optimization, scheduling, and industrial manufacturing.

In the field of path optimization [196], addressed the limited generalization capability of end-to-end deep models on large-scale TSP by proposing a lightweight neural combinatorial optimization framework. They introduced a hybrid strategy combining graph decomposition and local search. As illustrated in Fig. 12, the method builds a lightweight model based on a residual GCN. During the inference phase, it decomposes the large TSP instance into multiple local subgraphs through operations such as graph sampling, graph transformation, and heatmap merging. A sliding window mechanism enables local solving, with sub-solutions ultimately merged into a complete solution via pruning and recombination strategies. The study demonstrates the effectiveness and scalability of integrating DL with metaheuristic methodologies.

Subsequently [197], proposed a local search framework guided by GNNs, further highlighting the search guidance role of SL in metaheuristic algorithms. This method uses GNN to predict the potential benefits of local neighborhood operations and dynamically adjusts the search strategy based on these predictions, significantly improving convergence speed and reducing the risk of getting trapped in local optima, without altering the basic framework of the local search. Experimental results show that this framework achieved a dual improvement in search efficiency and solution quality across multiple combinatorial optimization tasks, demonstrating the universal potential of deep models in search guidance.

In multi-objective combinatorial optimization, combining metaheuristic evolutionary frameworks with the feature learning capabilities of SL can significantly improve the accuracy and convergence stability of global search. [198] proposed a hybrid optimization framework based on CNN–LSTM–GA for the multi-objective FJSP. In this framework, the GA provides a dual-encoding global search mechanism, the CNN is used to extract local spatial features of the production process, and the LSTM captures the temporal dependencies and dynamic constraints of tasks. The combination of these methods allows the core patterns of the FJSP to be transformed into search directions that can be utilized by GA, thereby enhancing the globality and robustness of the search.

Overall, the integration of SL with metaheuristic algorithms has introduced data-driven learning capabilities into traditional search methods, enabling metaheuristics to gradually shift from experience-based parameter tuning toward adaptive optimization. These approaches have demonstrated strong scalability and computational efficiency in complex COPs such as path planning, scheduling, and multi-objective evolution.

4.2.4 Supervised Learning Combined with Hyper-Heuristic Algorithms

Hyper-heuristic algorithms balance optimization depth and efficiency through problem decomposition, but are limited in large-scale and dynamic scenarios due to rule dependence and poor adaptability. SL, by learning from historical data and operation requirements, reduces subproblem size and provides data-driven decision guidance, introducing constraint filtering to improve adaptability and solving efficiency.

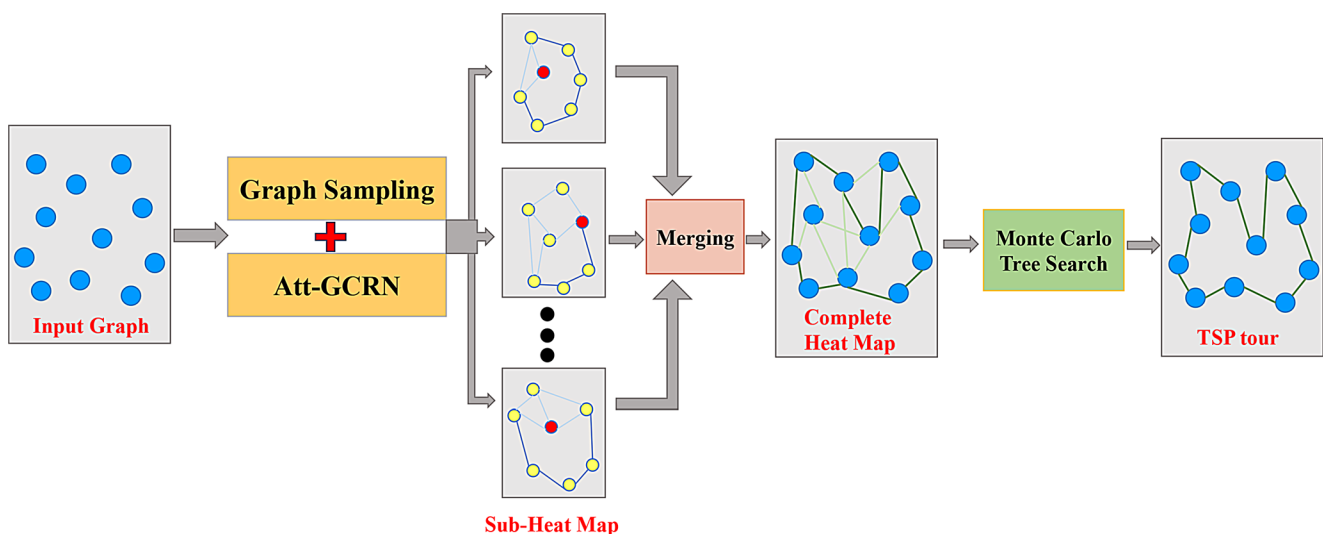


Fig. 12 Illustration of the lightweight neural combinatorial optimization framework

In the long-horizon FJSP scenario [199], proposed a learning-guided rolling horizon optimization framework (RHO), in which SL is used to reduce redundant computations, representing the core innovation. RHO first decomposes the long-horizon FJSP into overlapping short-horizon subproblems and solves them exactly using CP satisfiability models. The framework then introduces a NN model based on an MLP architecture to identify operations that do not require re-optimization, significantly reducing repeated solving overhead. Theoretical analysis further reveals the quantitative relationship between the prediction error of the NN model and optimization efficiency.

4.3 Comparative Analysis of Supervised Learning-Based Optimization Paradigms

Table 6 provides a comparative overview of representative supervised learning-based approaches for COPs. From the perspective of model functionality, these methods can be divided into two major paradigms: end-to-end learning-based

solvers and SL-OR collaborative solvers. End-to-end solvers directly learn the mapping from instances to decisions or solution structures, while SL-OR collaborative methods use learned models to assist key components of traditional optimization procedures. The comparison indicates that supervised learning contributes to COP solving mainly by improving representation learning, accelerating decision generation, and reducing the search burden of classical algorithms.

Solution Quality: End-to-end supervised learning solvers can achieve competitive solution quality when the problem structure is regular and the solution space can be represented as sequences, graphs, or priority decisions. For example, DSLO achieves near-optimal system utility exceeding 99% in MEC task offloading through CNN-based prediction [180], while PtrNet obtains high prediction accuracy on the maximum clique problem [165]. However, such methods heavily depend on high-quality labels and may suffer from unstable performance when constraints become complex or instance sizes increase. In contrast, SL-OR

Table 6 A comparative framework for supervised learning-based approaches in COPs

Approach	Forms	Problems	Qual.	Scal.	Gen.	Eff.	Data Inde.
Pointer Networks [53]	E2E / Pointer-net	TSP, Convex Hull, Delaunay	★★☆	★	★☆	★★★	☆
PtrNet-0-1 KP [164]	E2E / Pointer-net	0-1 Knapsack Problem	★★	★	★☆	★★★	☆
PtrNet-Maximum Clique [165]	E2E / Pointer-net	Maximum Clique Problem	★★☆	★	★☆	★★	☆
PtrNet-UBQP [166]	E2E / Pointer-net	UBQP	★★☆	★	★☆	★★★	★☆
Ptr-Net-PFSP [168]	E2E / Pointer-net	PFSP	★★	★	★☆	★★	☆
Hyper4Route [169]	E2E / Pointer-net	Last-mile route prediction	★★☆	★	★★	★★	★☆
MEC-PtrNet [170]	E2E / Pointer-net	MEC task scheduling	★★☆	★☆	★★	★★☆	★★
GCN [54]	E2E / GNN	TSP	★★☆	★★	★	★★☆	☆
GNN-SL [171]	E2E / GNN	D2D scheduling, RRA	★★☆	★★☆	★★☆	★★★	★★
Structure2vec Scheduler [173]	E2E / GNN	D2D link scheduling	★★☆	★★☆	★★	★★☆	★★
Heterogeneous GAT [174]	E2E / GNN	CMfg logistics scheduling	★★☆	★★	★☆	★★☆	★
AMDKD [176]	E2E / Transformer	TSP, CVRP	★★☆	★★	★★☆	★★☆	★★
Graph Transformer [177]	E2E / Transformer	TSP, JSSP	★★	★★	★★	★★	☆
SPP-Transformer [178]	E2E / Transformer	DJSSP	★★☆	★★	★★☆	★★☆	★
Permutation Invariant [179]	E2E / Other network	CVRP	★★☆	★★	★★	★★☆	☆
DSLO [180]	E2E / Other network	MEC Task Offloading	★★★	★★	★★★	★★★	☆
GNN-MIP [181]	SL-OR / Exact	MIP Branching	★★★	★★	★★	★★	★☆
GCNN [182]	SL-OR / Exact	MIP Branching	★★★	★★	★★	★★	★☆
SLR-EPN [184]	SL-OR / Exact	JSSP	★★☆	★★☆	★★☆	★★☆	★
BCxCP [185]	SL-OR / Exact	FJSP	★★☆	★★	★★	★★☆	★☆
EDNN-QP [186]	SL-OR / Exact	MEA power scheduling	★★★	★★☆	★★☆	★★★	★
GRU-DNN+MIP [187]	SL-OR / Exact	SCC scheduling	★★☆	★★	★★	★★	★☆
GCNN-SCIP [188]	SL-OR / Exact	JSSP, DSP	★★☆	★★	★☆	★★	★
NeuroLKH [189]	SL-OR / General Heuristic	TSP, CVRP, PDP, CVRPPTW	★★★	★★☆	★★☆	★★	★☆
SL for D-TSP [192]	SL-OR / General Heuristic	Dynamic TSP	★★☆	★★	★★	★★★	☆
GNN&GBDT [193]	SL-OR / General Heuristic	Integer Programming	★★★	★★★	★★☆	★★☆	★☆
GAT Partitioning [195]	SL-OR / General Heuristic	Real-time partitioned scheduling	★★	★★	★★	★★☆	★★
GNN-MCTS [196]	SL-OR / Metaheuristic	TSP	★★☆	★★★	★★☆	★★	★☆
GNN-GLS [197]	SL-OR / Metaheuristic	TSP	★★☆	★★☆	★★☆	★★☆	★☆
CNN-LSTM-GA [198]	SL-OR / Metaheuristic	MO-FJSP	★★	★☆	★☆	★★	★★
L-RHO [199]	SL-OR / Hyper-Heuristic	Long-horizon FJSP	★★★	★★★	★★★	★★★	★☆

collaborative methods usually provide more robust solution quality because learning models only assist key optimization decisions rather than replacing the entire solving process. NeuroLKH improves LKH by learning candidate edges and achieves a much smaller optimality gap on TSP-500 [189], while GNN&GBDT obtains better objective values than Gurobi and SCIP on large-scale integer programming instances [193]. Therefore, supervised learning is generally more reliable when used as an enhancement module within classical optimization frameworks.

Scalability: The scalability of end-to-end methods remains limited, especially for sequence-decoding architectures whose computational cost increases with solution length. For instance, PtrNet shows degraded accuracy on large-scale 0–1 knapsack instances [164]. GNN-based methods improve scalability by modeling relational structures, but they are still constrained by graph density and message-passing complexity; GCN-TSP can handle up to 1,000 nodes with beam search, yet its $O(n^2)$ complexity limits further scaling [54]. Transformer-based models enhance global dependency modeling, but the quadratic complexity of self-attention also restricts their application to large-scale COPs [177]. By comparison, SL-OR collaborative methods are more scalable because they reduce the effective search space through decomposition, candidate reduction, or rolling optimization. For example, GNN-MCTS improves large-scale TSP solving through graph decomposition and local search [196], while L-RHO reduces redundant computation in long-horizon FJSP through learning-guided rolling optimization [199].

Generalization: End-to-end supervised models are often sensitive to the training distribution. When test instances differ in scale, structure, or constraints, their performance may decline significantly. To improve generalization, some studies introduce structural priors or knowledge transfer mechanisms. AMDKD enhances cross-distribution generalization in routing problems through multi-teacher knowledge distillation [176], and SPP-Transformer improves adaptability in dynamic job shop scheduling by combining Transformer encoders with spatial pyramid pooling [178]. In real-world applications, Hyper4Route further incorporates courier profiles, urban contexts, and hypergraph structures to improve route prediction under complex delivery scenarios [169]. Compared with end-to-end methods, SL-OR collaborative methods tend to generalize more robustly because the underlying optimization model or search framework remains valid across different instances. For example, GCNN learns branching decisions while preserving the branch-and-bound framework [182], and GAT Partitioning imitates heuristic partitioning rules to improve transferability in real-time scheduling [195].

Computational Efficiency: End-to-end solvers have strong inference efficiency after training, making them suitable for real-time or near-real-time scenarios. For example, GNN-SL directly outputs resource allocation decisions in D2D communication [171], and MEC-PtrNet decodes task execution sequences with low inference latency in dynamic MEC scheduling [170]. Nevertheless, this efficiency advantage may be weakened when additional repair, validation, or local search is required. SL-OR collaborative methods improve efficiency by accelerating expensive components of traditional solvers. GNN-MIP approximates strong branching to reduce the computational burden of branch-and-bound [181], EDNN-QP simplifies mixed-integer quadratic programming by predicting integer variables [186], and BCxCP avoids unnecessary exact solving by predicting subproblem complexity [185]. Thus, computational efficiency is improved not only through fast neural inference, but also through the acceleration and simplification of classical optimization procedures.

Summary: Overall, Table 6 shows that supervised learning is most effective when it complements rather than replaces classical optimization. End-to-end methods offer fast inference and flexible modeling, but their performance is often constrained by data quality, instance scale, and constraint complexity. In contrast, SL-OR collaborative methods embed learning into branching, decomposition, candidate generation, variable prediction, and search guidance, achieving a more balanced performance in solution quality, scalability, generalization, and computational efficiency.

5 Reinforcement Learning for Combinatorial Optimization Problem

This section systematically reviews RL-based approaches to COPs. Since COPs often lack precise dynamics, related RL methods are generally model-free. To clarify the technical evolution, we divide methods by whether neural function approximation is used, such as traditional tabular methods without parameterized models, represented by classical Q-learning, and NN-based DRL methods the mainstream further organized into value-based, policy-based, and AC paradigms.

5.1 Q-Learning-OR Collaborative Solving

Having outlined these two categories, we first discuss traditional tabular methods. As a representative, Q-learning model-free and conceptually intuitive provided key ideas for early RL for COPs exploration. This section synthesizes typical paradigms where Q-learning is embedded and

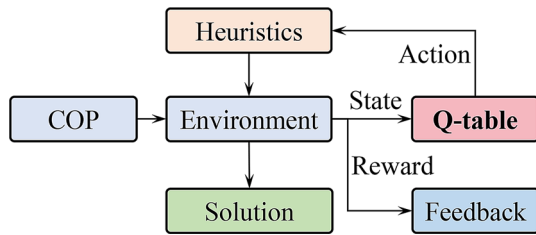


Fig. 13 Q-learning as decision evaluation

analyzes each; representative studies appear in Table. Figure 13 summarizes a general paradigm where Q-learning acts as a decision evaluator inside heuristic optimization for COPs.

5.1.1 Q-Learning Combination with Heuristics

To improve heuristic flexibility and performance [200], introduced RL into constructing heuristic evaluation functions via temporal-difference learning. Their non-network method does not directly control search behavior but uses learned evaluations to assist in selecting starting points, improving overall efficiency. This first explicitly positioned RL as an evaluator, laying a theoretical foundation for later Q-learning methods.

In hybrid frameworks, Q-learning typically plays the role of policy evaluator embedded at key decision points in the search, as in Fig. 14. At each iteration, when selecting among candidate operations such as neighborhoods, operators, dispatching rules, Q-learning queries the Q-table

based on the current state, scores actions, and decides via ϵ -greedy or greedy selection. The heuristic then executes the chosen action and updates Q via rewards from search feedback. This mechanism does not change the heuristic structure but replaces static rules with learned, feedback-driven decisions, improving adaptability and decision quality. Compared with traditional methods, this lightweight RL embedding preserves modeling and efficiency while adding dynamic, feedback-driven learning—now a common enhancement in CO.

Researchers [201] proposed a path-search method that uses RL for ranking candidate actions, integrating Q-learning into LKH for TSP and variants to learn scoring functions for candidate edges. In each k -opt operation, the current city is the state, target cities reached by candidate edges are actions, path-length reduction is the reward, and Q-learning or SARSA trains the edge value estimator. The learned Q-values replace LKH's α -values to sort candidate edges, guiding expansion order and improving performance and stability. This retains LKH's main search structure while embedding RL as a candidate scorer, achieving significant gains on standard TSP, TSPTW and CTSP datasets and delivering new optima on CTSP—showing practical potential for RL as an evaluation function.

Beyond ranking, RL also controls destroy-repair strategies in constructive heuristics. [202] proposed RL-based alpha-list iterated greedy for permutation flow shop problem, applying Q-learning within iterated greedy to control

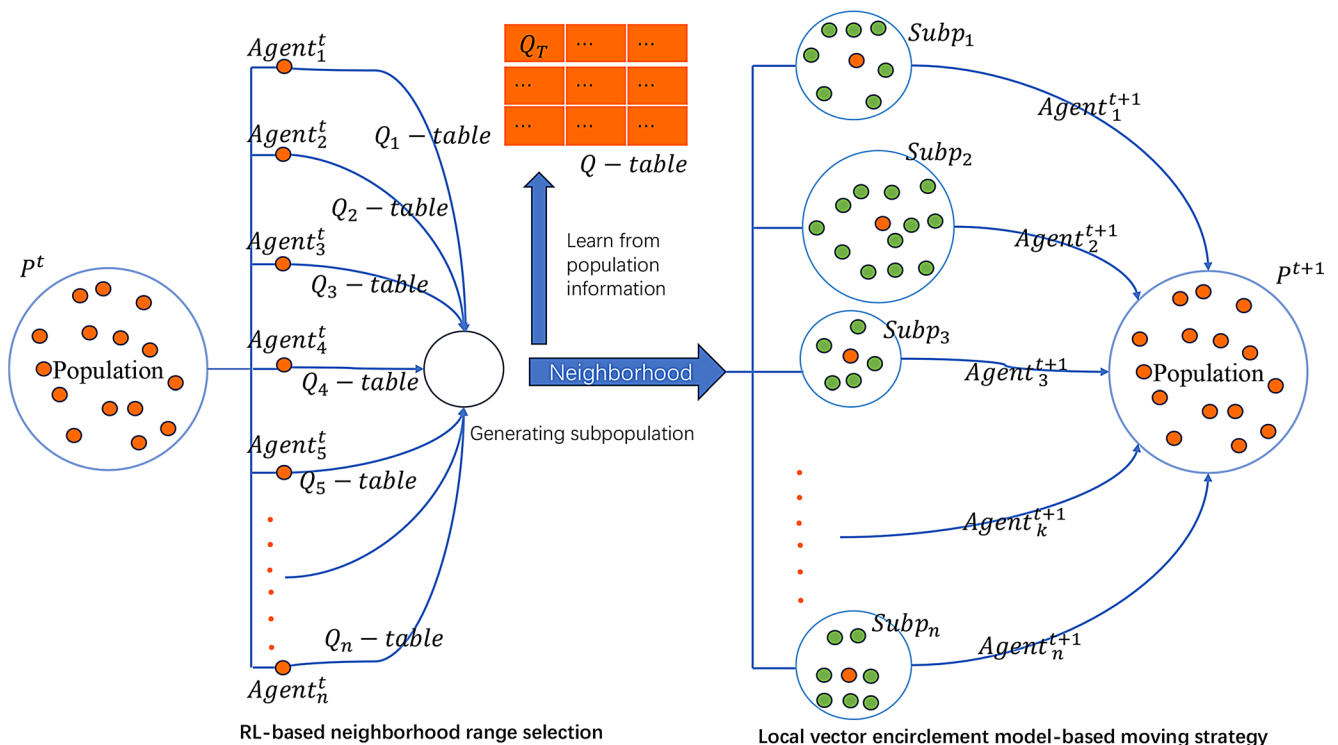


Fig. 14 Q-learning as a policy evaluator in neighborhood search

the α -list in the destroy step. By learning the effect of parameter configurations on solution quality, the model selects destruction intensity based on the current search state and history, improving stability and adaptability. [203] extended RL to biobjective energy-efficient FJSP via RL driven iterated greedy algorithm, with two Q-tables for route choice and machine speed control to balance makespan and energy. This enhances IG's control flexibility for energy-aware scheduling in industrial scenarios.

Beyond local construction, Q-learning is also used for general strategy scheduling such as neighborhood selection, perturbation control, or resource scheduling. [204] proposed RL-based multi-start neighborhood search for traveling thief problem, integrating RL into late-stage neighborhood selection. [205] proposed RL-based neighborhood search operator to control local perturbation operators and scopes in multimodal optimization. [206] built an RL-driven destroy-repair mechanism combined with automated guided vehicles (AGVs) co-scheduling for distributed shops with sequence dependent setup times. These methods introduce RL control into heuristic structures to achieve dynamic strategy adjustment-showing diverse integration and transferability.

5.1.2 Q-Learning Combination with Metaheuristics

Within metaheuristics, Q-learning similarly acts as a policy evaluator embedded at key decision points. At each iteration, when choosing among candidate operations such as neighborhoods or operators, perturbation strength, parameter settings and acceptance criteria, Q-learning maps the current search state, including recent best and mean improvement, stagnation steps, diversity, feasibility violations and multi-objective indicators to Q values and selects actions using ϵ -greedy, greedy or softmax strategies. The metaheuristic executes the selected action for one iteration or phase and uses objective improvement together with feasibility rewards or scalarized multiobjective gains as the immediate reward when needed. It then updates $Q(s, a)$ using temporal difference rules, with mechanisms such as sliding windows, forgetting factors and reward clipping to reduce non stationarity and noise. This approach does not alter the core metaheuristic but replaces static components with online learning, which supports adaptive control of exploration and exploitation, operator selection and tuning of intensities and thresholds. It is low cost, plug and play and transferable across instances. Compared with fixed rules, this lightweight RL loop continually corrects decisions based on observed performance and can therefore improve solution quality, stability and robustness in complex COPs.

Chen et al. [207] applied RL to operator and parameter selection in GA, proposing Self learning GA for FJSP. They designed a Q-learning controller in which crossover

and mutation probabilities and operator combinations are treated as actions, and the state aggregates information such as fitness improvement, diversity and the iteration phase, while the reward is based on solution quality improvement. In each generation, GA queries the Q-table and selects configuration via ϵ -greedy; after execution, Q-values are updated, gradually favoring better configurations. This lets GA switch exploration or exploitation across phases and avoids local traps. On Brandimarte benchmarks, SLGA shows faster convergence and better solutions than refined GAs, with statistical significance-confirming Q-learning as an effective GA operator and parameter evaluator.

[208] introduced Q-learning into GA for multi-agent task scheduling [209]; used Q-learning in a multiobjective immune algorithm for FJSP to adapt operators and neighborhood sequences under dynamics-optimizing makespan and energy. These works support Q-learning as a parameter-operator evaluator.

Beyond operators or parameters, RL targets broader learning objects. [210] proposed dynamic self-learning artificial bee colony (ABC) optimization algorithm for dynamic FJSP, adjusting exploration and exploitation in ABC when inserting jobs to maintain stability [211]; used RL-ABC to learn dispatching and subplot schemes, improving makespan [212]; leveraged Q-learning to adjust subpopulation sizes and operator allocation in multi-population EAs for joint scheduling and transport under finite resources [213]; treated operation-sequence moves as actions in cuckoo search with Q-learning based on feedback [214]; used Q-learning within RMOEA/D to learn neighborhood size and decomposition strategy, improving convergence and diversity under fuzzy environments [215]; embedded Q-learning into water wave optimization to learn acceptance rules, improving distributed assembly line scheduling. These show tabular RL's flexibility across local to system-level decisions and domain-specific rules.

5.1.3 Q-Learning Combination with Hyper-Heuristics

As research progressed, Q-learning was introduced into hyper-heuristic frameworks as a decision evaluator to control LLH selection and usage at a higher level, as shown in Fig. 16. Specifically, Q-learning abstracts the search as sequential decisions. The state describes the current solution or stage, actions correspond to candidate operators or acceptance strategies, and reward measures improvement. Over iterations, Q-learning updates Q-values to identify better-performing operators or combinations at different stages, balancing exploration and exploitation and enhancing robustness and cross-domain generalization. The iterative process is shown in Fig. 15.

In this direction [216], proposed QHH, a typical Q-learning driven automated hyper-heuristic design. The core is to treat pairs of LLH selection methods and move-acceptance criteria as high-level actions, with Q-learning evaluating and selecting them adaptively during search. Specifically, multiple LLH selection methods such as those based on improvement magnitude, per-time gains and call frequency and acceptance strategies such as accept-only-improving, probabilistic and SA-based are combined into an action set. The search state is represented by discretized indicators such as normalized solution quality to reduce dimensionality. Rewards are episodic, and after a complete ILS round including diversification via LLHs, intensification via neighborhoods or local search and acceptance, Q-values are updated. Thus, Q-learning acts as a high-level controller that dynamically adjusts operator-acceptance pairs across stages, avoiding handcrafted or static control and achieving adaptive hyper-heuristic design.

Q-learning is also used for constructive choices and trajectory learning. For set-union knapsack problem [217], modeled set selection as actions and the current solution as the state with objective improvement as reward-learning which selections yield better global optimization-improving quality and convergence. For route optimization with logistics constraints [218], used the current vehicle location as state and next-visit node as action with cost-reduction reward-effectively guiding multi-vehicle, heterogeneous-resource scenarios.

More research focuses on dynamic LLH selection. [219] for distributed assembly blocking flowshop SP reducing blocking or waiting via Q-learning-controlled operator selection (Fig. 16). [220] for corridor allocation problem and parallel row ordering problem using Q-learning to dynamically choose perturbation operators, improving diversity and convergence. [221] for dynamic task allocation of crowdsensing, where a Q-learning hyper-heuristic EA effectively replaces MILP on large instances. [222] for DFJSP introducing Q-learning to learn neighborhood calls, improving makespan and energy. [223] for multiobjective energy-efficient distributed blocking flow shop scheduling problem using Q-learning to balance different neighborhoods, achieving time-energy trade-offs. [224] for FJSP rescheduling under machine failures, showing greater stability. [225] for supply-chain multiobjective scheduling using self-adaptive hyper-heuristic with Q-learning for operator and parameter tuning [218]; for Stochastic mixed-model assembly line sequencing problem combining Q-learning and SA to reduce overload and idleness [226]; for semiconductor final testing, where Q-learning selects among eight LLHs, improving utilization. These embed Q-learning into hyper-heuristic frameworks to enable dynamic strategy control and robust performance with good transfer.

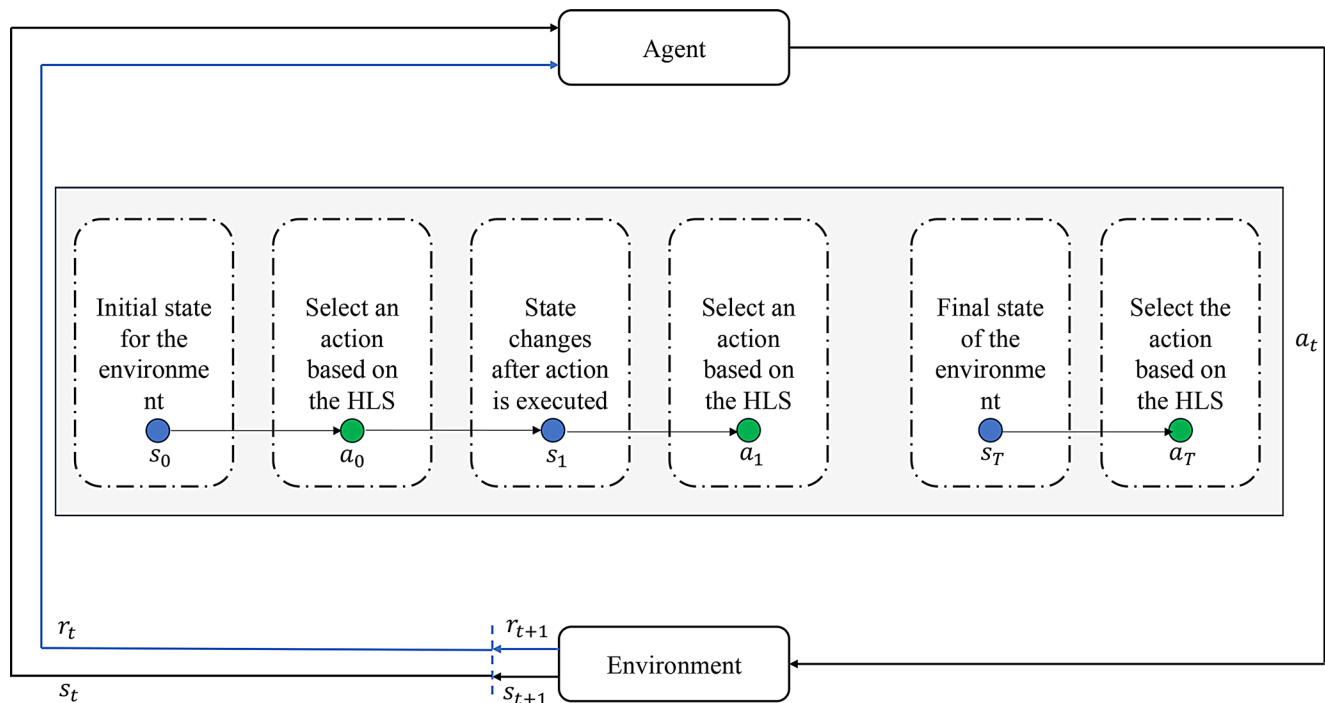


Fig. 15 Iterative process of Q-learning-driven HLS

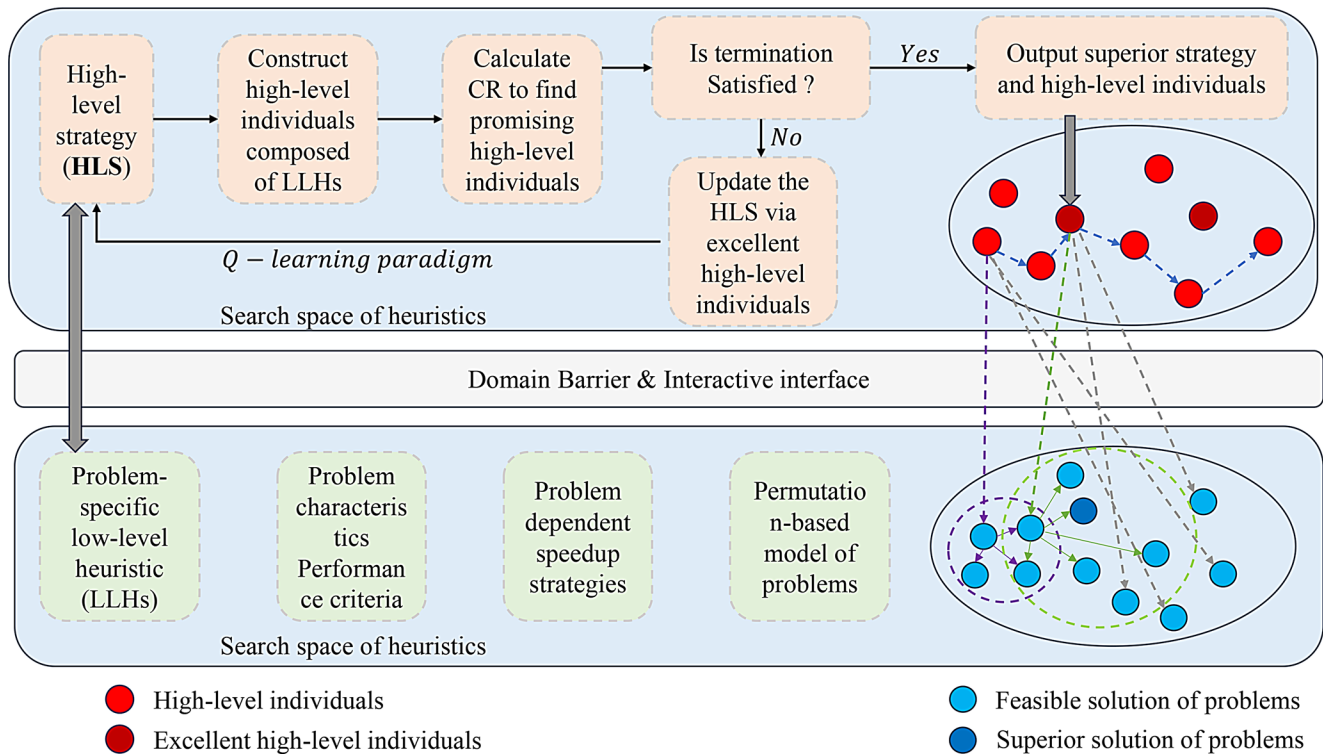


Fig. 16 Q-learning controlling selection of LLHs

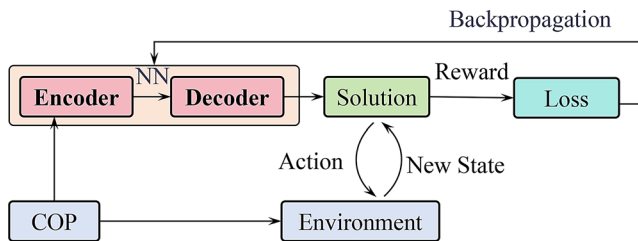


Fig. 17 DRL as End-to-End optimizer

5.2 Deep Reinforcement Learning for Combinatorial Optimization Problem

5.2.1 End-To-End Learning-Based Solvers

Compared to the previously discussed SL methods that rely on supervision signals or integrate with traditional optimizers, DRL provides an end-to-end learning paradigm for COPs. The core idea is to model the problem as a sequential decision-making process, where an agent continuously learns an optimization policy through interactions with the environment. These methods do not require explicit labels or external optimal solutions. Instead, they rely on a reward function to guide the search process, demonstrating strong adaptability in large-scale and complex constraint scenarios. The overall learning and interaction process of this paradigm is summarized in Fig. 17.

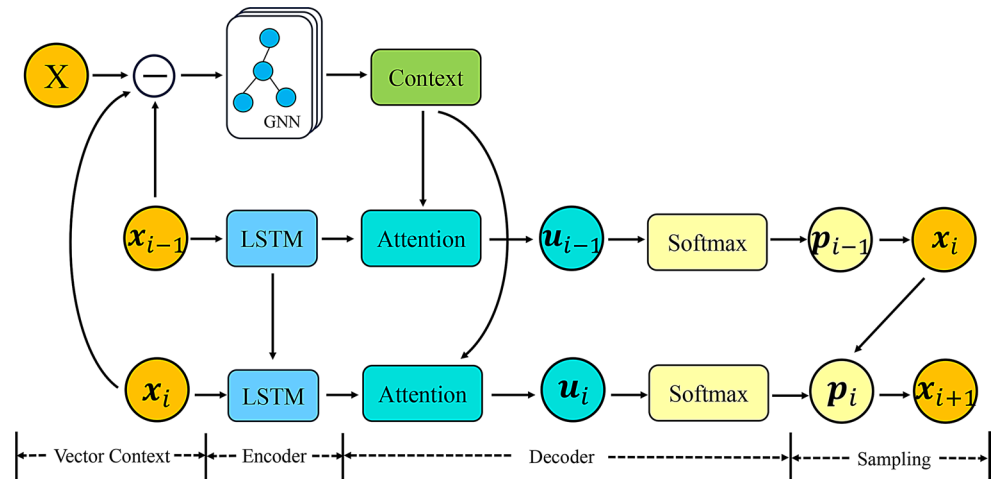
Pointer Networks

In DRL-based combinatorial optimization research, PtrNets leverage the attention mechanism to directly point to elements in the input sequence, making them suitable for sequence generation tasks such as path planning and scheduling. The core idea is to replace explicit supervision signals with RL strategies, optimizing the decision-making policy through interactions with the environment, providing strong adaptability and cross-distribution generalization ability.

In early research [227], proposed the GPN, which extends the traditional PtrNet by incorporating graph structural information and a hierarchical RL mechanism. GPN decomposes complex combinatorial optimization tasks into high-level task selection and low-level node assignment stages, achieving unified modeling for various graph optimization problems. The model's encoder consists of a point encoder and a graph encoder, the point encoder generates node embeddings for city nodes through linear mapping and LSTM, while the graph encoder uses GNNs to extract topological features between cities. Based on graph and point embeddings, the model calculates node selection probabilities using the attention mechanism and incorporates vector contexts to enhance decoding stability and generalization performance. This method laid the foundation for graph-structured PtrNets in DRL4COP research.

Building upon this [228], proposed the hybrid PtrNet (HPN), which has a similar overall structure to the GPN framework shown in Fig. 18 but further integrates multi-view

Fig. 18 GPN architecture diagram. The model consists of a point encoder, graph encoder, and attention decoder, realizing hierarchical optimization for high-level task selection and low-level node assignment



feature embeddings, including GCN, fully connected layer embeddings, and attention mechanisms, to capture both global and local graph features more comprehensively.

As research progresses, the application of PtrNets has gradually expanded to scheduling-related COPs. In the field of solving the JSSP [123], proposed an end-to-end scheduling method sequence-to-sequence advantage actor-critic (S2S-AC) based on DRL and PtrNets. This method models the DJSP as a markov decision process (MDP), comprehensively characterizing the scheduling state. In terms of algorithm design, S2S-AC combines an improved PtrNet with the advantage actor-critic (A2C) framework. The PtrNet is used to generate the probability of operation selection and employs a masking mechanism to shield completed operations and conflicting machines, ensuring solution feasibility. In A2C, the policy network samples actions based on the probability distribution from the PtrNet, while the value network estimates state values, reducing the variance of policy updates and addressing the lag and repetitive iterative computation issues in traditional static scheduling methods in scenarios with dynamic processing times.

For the coupled complexity of operation selection and machine allocation decisions in the FJSP [124], improved the traditional PtrNet structure. Their encoder replaces the recurrent structure with a 1D Convolution layer to enhance the ability to model unordered features. The decoder uses gated recurrent units (GRU) combined with attention mechanisms to simultaneously output operation sequencing and machine allocation probabilities, forming a compact architecture with dual outputs in a single network, laying the foundation for the application of PtrNets in the FJSP scenario. Following this [229], further proposed the GAT-Ptr strategy, which combines GAT with PtrNets to address the scheduling repair problem under machine failures. This method uses GAT to capture interactions between operations and machines, outputs the selection probabilities for operations to be repaired, and introduces a completion

decision module to determine the repair termination time, achieving a fast-response scheduling repair strategy with minimal interference.

In large-scale Flexible Flow Shop Scheduling [230], proposed an optimization framework based on Dual-layer Learning. The lower layer uses an improved GPN for local optimization, while the upper layer uses a DDQN to implement global decision control. The two layers work together to optimize and meet the high-dimensional and real-time demands of industrial scheduling.

Graph Neural Networks

DRL has shown broad potential in strategy learning and decision optimization for COPs, but traditional sequence modeling methods struggle to capture resource dependencies and constraint conflicts, limiting global perception and strategy generalization. GNNs address DRL's limitations in unstructured state modeling by structuring nodes and edges, becoming an important direction for end-to-end solutions.

[231] proposed the exploratory combinatorial optimization framework, which was the first to combine GNN with RL for typical tasks like the TSP and maximum cut. This method encodes graph instances with GNN and introduces exploratory rewards in the policy network to encourage the agent to step out of local optima during the search. Experimental results show that this framework not only improves solution quality but also demonstrates good scalability, generalizing to larger instances beyond the training scale. The introduction of the exploration mechanism combined with graph structure representation laid the methodological foundation for subsequent GNN-RL frameworks.

Building upon this [232], proposed a teacher-learning-based reinforcement framework called GREAT. This method constructs a hybrid training paradigm for graph optimization tasks. First, SL is used to pre-train the edge classification task, allowing the model to gain heuristic knowledge. then, in the unsupervised stage, RL fine-tunes the policy network to enhance generalization and convergence quality.

This two-stage training design balances the stability of SL with the exploratory nature of RL, and experimental results show that GREAT outperforms single-paradigm models on multiple TSP benchmark datasets, exhibiting advantages in cross-distribution generalization.

In the scheduling domain [233], proposed an end-to-end framework, learning to dispatch (L2D), which achieves the organic integration of GNNs and DRL. This method represents the JSSP as a disjunctive graph, using GIN to extract structural features between operations and machines, and automatically learns priority dispatching rules (PDR). Meanwhile, L2D employs an arc-adding strategy to retain only the directed arcs with established directions, maintaining graph sparsity and reducing computational costs, thus enabling cross-scale generalization. In terms of policy optimization, the PPO algorithm is used, combined with multi-trajectory sampling to improve learning stability.

Building on the L2D framework, subsequent research has mainly focused on feature representation and training efficiency optimization. [234] expanded the node feature dimensions and designed specialized feature update functions for predecessor, successor, disjunctive, and target nodes, making GNN more precise in capturing different types of constraint relationships. They also introduced a semi-MDP mechanism to skip machine idle states and improve training efficiency. [235] proposed bidirectional scheduling features, combining forward scheduling progress and backward scheduling estimated makespan to supplement state information, and introduced an invalid action mask to filter infeasible actions, significantly reducing the action space and improving decision accuracy.

As centralized methods mature, research has expanded to policy performance upgrades and distributed decision-making. [236] modeled production systems as machine-buffer bipartite graphs, combining custom message passing mechanisms with multi-agent PPO for distributed decision-making. [237] proposed a framework that integrates soft actor-critic (SAC), disjunctive graph embedding (DGE), and transformer encoding, using self-attention mechanisms to capture long-range dependencies and enhancing exploration capabilities through maximum entropy strategies.

In the FJSP, the coupled decisions of operation selection and machine allocation make modeling and solving more complex. [238] proposed the MPGN and multi-policy PPO, modeling the two decision dimensions through MDP. The operation encoder extracts constraint features based on GIN, and the machine encoder uses fully connected layers to obtain machine status, outputting action probability distributions. The DRL layer uses multi-PPO for independent optimization of sub-policies, achieving decision decoupling and collaboration.

As the complexity of FJSP increases, research has further focused on graph structure optimization and attention mechanism enhancement. [239] merged operation selection and machine allocation into composite actions, constructing operation-machine heterogeneous graphs and employing a two-phase embedding mechanism to reduce graph density and computational burden. [240] proposed relation-driven heterogeneous GNNs, decomposing and combining operation-operation dependencies and operation-machine constraints to improve feature representation accuracy. [241] constructed a dual-dynamic attention network, using GATv2 to parallel model operation and machine dependencies to enhance global perception. [242] designed a dual-attention network, using dynamic masking mechanisms to compress state space and optimize decision efficiency. [243] further proposed the GAT-DGL-CL framework, which integrates adaptive GAT, dynamic graph learning, and progressive curriculum learning, adaptively learning dynamic associations among scheduling entities to enhance feature representation, training stability, and generalization capability in FJSP solving.

In the multi-agent direction [244], decomposed FJSP into two independent MDPs, constructing a dual-agent collaborative framework with heterogeneous graphs and an improved GAT, and enhancing policy stability through multi-agent PPO. [245] designed a dual-agent framework for FJSP-AGV with AGV, using PPO and DQN to optimize operation sequencing and machine allocation. [246] further extended the decision dimensions to a triple of operation-machine-AGV to reduce AGV idle time and adapt to complex production processes. [247] proposed a lightweight end-to-end framework based on a multi-Layer Perceptron for efficient scheduling strategy learning, reducing computational overhead to adapt to resource-constrained scenarios.

Transformer

Due to its advantages in sequence modeling and capturing global dependencies, the transformer architecture has become a core framework in DRL for solving COPs. Compared to PtrNets, transformer flexibly models node relationships through a multi-head self-attention mechanism and improves solution quality and generalization through autoregressive or multi-branch decoders.

The earliest representative work of transformer in path and routing optimization problems came from Kool [55]. This study proposed an end-to-end solving framework based on the attention mechanism, discarding the structure of RNN and using a multi-head attention mechanism in the encoder to model interactions between all nodes, efficiently extracting global features from the graph structure. The decoder autoregressively selects the next node to visit based on the current locally constructed path, effectively simulating the sequence decision process in human path planning.

In terms of training, the model employs RL, combining the REINFORCE algorithm with a baseline rolling mechanism for end-to-end optimization, minimizing the path length as the objective function without any supervision signals.

Building on this, subsequent research has expanded in areas such as model structure, policy design, and generalization performance. [248] proposed the multi-decoder attention model (MDAM), which introduced a parallel decoding architecture and embedding-aware mechanism to enhance policy diversity and dynamic self-adaptation, significantly improving generalization performance in TSP and CVRP. [249] proposed the light encoding-heavy decoding (LEHD) architecture, which strengthens context reasoning and feature representation in the single-node autoregressive selection process, generalizing to large-scale tasks with thousands of nodes based on training with small-scale instances. [250] introduced the Pointerformer model, which integrates PtrNets with transformer, designing a multi-pointer decoding mechanism to strengthen local path modeling and improve convergence stability. [251] proposed a hierarchical optimization framework that combines coarse-grained partitioning with fine-grained construction to address the computational bottleneck of transformer in large-scale instances, achieving real-time solving. [252] proposed a multi-agent attention model (MAAM) that combines the Transformer decoder with multi-agent RL to tackle complex constraints in multi-vehicle and customer interactions, demonstrating strong adaptability in the soft time-window m-VRP. Overall, these studies significantly expanded the application boundaries of Transformer in path-based COPs through structural innovations and decoding strategy optimization.

In the JSSP domain [253], proposed the DGE with recurrent decoder transformer framework, which combines DGE with Attention Mechanism to form a universal global constraint modeling paradigm. The method first uses the improved node2vec algorithm to capture neighborhood constraint features of operations and generates low-dimensional operation embedding vectors. Then, a parallel encoder and recurrent decoder structure is constructed, where the encoder aggregates cross-machine operation dependencies through multi-head attention, and the decoder integrates real-time scheduling information and context vectors to output operation selection probabilities, ensuring solution feasibility through a masking mechanism. Finally, policy gradient methods are used for optimization, with a moving average baseline introduced to reduce reward bias.

Beyond the DGERD framework, other research has focused on dynamic scenarios and architecture simplification. [254] addressed the DJSP with job arrivals, using a three-layer transformer to capture long-range dependencies and combining double duel DQN (D3QN) with prioritized

experience replay mechanisms to construct a decision network, selecting optimal rules through data envelopment analysis. [255] proposed a graph RL and auxiliary strategy framework for machine failure and random processing time problems, using a hybrid graph transformer network (GTN) for multi-scale feature aggregation, and improving sample efficiency through a phasewise PPO algorithm with Rollback. [256] proposed a simplified transformer architecture without positional encoding. [257] proposed the gated attention mechanism with DRL model to address feature propagation noise, improving the model's stability in dynamic scenarios by jointly training with an improved gated attention module and PPO.

In the FJSP research, the multi-task modeling capability of transformer is further explored. [258] proposed the SGFormer-SAC (Simplified Graph Transformer with SAC) framework to optimize large-scale fuzzy FJSP. Its innovation lies in three aspects. First, structural complexity optimization by introducing a SGFormer, reducing complexity from $O(N^2)$ to $O(N)$ through graph sparsification and linear attention mechanisms. Second, adaptation to fuzzy scenarios by using fuzzy disjunctive graphs to describe system states and defining rewards as the fuzzy makespan difference between adjacent states to guide the policy to reduce delay uncertainty. Third, strategy optimization design, using SAC to balance exploration and exploitation, with a masking mechanism to limit the action space. Experimental results show that this method achieves significant performance improvement on fuzzy brandimarte and behnke datasets. [259] designed a single-agent MDP for deterministic large-scale FJSP, using a transformer encoder to capture global features, simplifying action dimensions with 16 composite scheduling rules, and optimizing the AC architecture using PPO. In addition [260], proposed the Mamba-CrossAttention framework, which formulates FJSP as a sequential decision-making process over operation-machine pairs. The method employs dual Mamba blocks to separately extract operation and machine sequence features, thereby capturing long-range dependencies with linear complexity. A lightweight cross-attention decoder is then used to fuse operation and machine representations, while permutation-invariant policy optimization is incorporated to enhance the model's robustness to input ordering and its generalization capability.

Other representative research includes [261] proposed the collaborative agent RL dual-agent collaborative framework. [262] proposed a dynamic rescheduling method combining diffusion transformer and twin delayed deep deterministic policy gradient, and [263] proposed the graph transformer-PPO framework, all of which demonstrate the scalability of transformer in multi-decision, dynamic, and fuzzy scheduling scenarios.

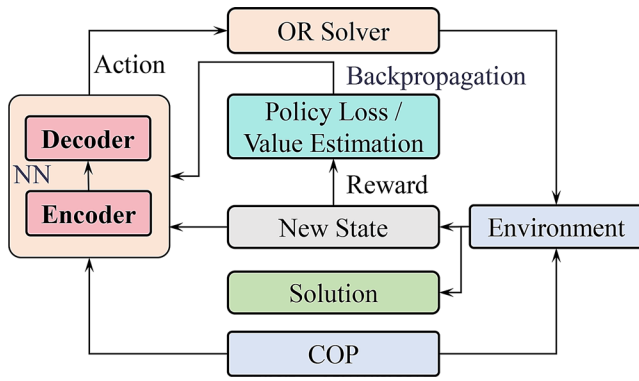


Fig. 19 DRL as decision evaluator

5.2.2 DRL-OR Collaborative Solving

End-to-end generative RL models often face the challenge of balancing feasibility and optimality when dealing with large-scale, highly constrained COPs. Embedding DRL as a decision evaluator within traditional exact optimization algorithms helps balance computational efficiency and solution quality by learning key decision processes and guiding branching choices, enhancing rule-based search logic. Figure 19 schematically illustrates how DRL is integrated as a decision evaluator within exact solvers.

Deep Reinforcement Learning Combined with Exact Algorithms

In this direction, DRL acts as an auxiliary evaluator embedded within the exact algorithm framework, accelerating the search process and ensuring optimality by predicting node values or guiding heuristic strategies. By learning from historical trajectories and problem structures, DRL replaces or enhances traditional heuristic scoring functions, and its integration with the interpretability and solution quality guarantees of exact algorithms has become a major research focus in recent years.

[264] proposed a backtracking trajectory-based training framework to address the sparse reward and credit assignment issues in RL during the branch-and-bound process. The method records the complete search trajectory during solving and uses a backtracking mechanism to reconstruct the long-term impact of each branching decision on subsequent searches, providing stable and efficient training signals for policy learning. As shown in Fig. 20, the model uses GNN to encode the constraint graph of MIP, and the policy network outputs branching actions. The training process is optimized using policy gradients. Experimental results show that this method significantly reduces the search tree size and the number of explored nodes across multiple MIP benchmarks.

Related research can be categorized into four main innovation directions, namely branch and variable selection, cutting-plane and pruning strategies, stopping-criterion learning, and presolving and original heuristic optimization.

- (1) In the area of branching and variable selection [265], pioneered the use of RL to directly learn branching variable selection strategies, replacing manual heuristic rules. [266] further modeled the branch-and-bound tree as a tree-MDP, capturing the long-term dependencies of the search process more precisely within the RL framework. Such research indicates that optimizing branching variable selection through policy learning can effectively reduce the size of the search tree and improve solving efficiency.
- (2) In the direction of cutting planes and pruning [267], first proposed using RL to automatically select cutting planes in branch-and-bound, improving convergence efficiency. [268] further proposed a Hierarchical Sequence Model, which predicts the proportion of cutting plane selections at the higher level and generates an ordered subset of cutting planes using a PtrNet at

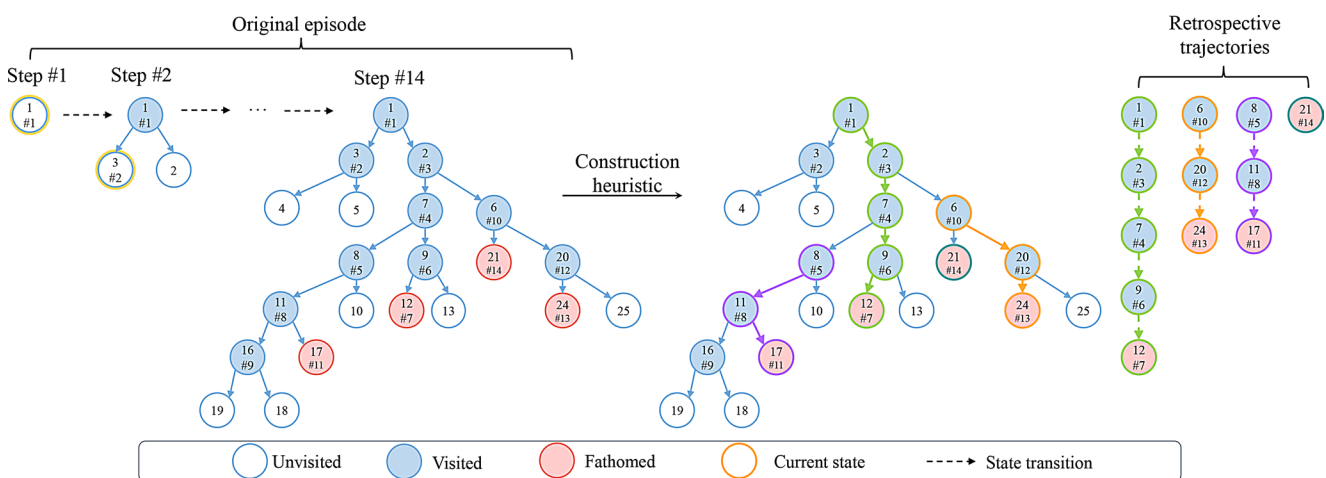


Fig. 20 Backtracking trajectory-based branching strategy training process

the lower level, thus addressing both the selection and the ordering issues. [269] further proposed a hierarchical sequence and set model, which introduces Set2Seq encoding and hierarchical PPO to improve sample efficiency and generalization performance.

- (3) For learning the stopping criteria in branch-and-cut algorithms [270], proposed a hybrid graph representation model to dynamically adjust the stopping threshold for different MIP instances. This method models the stopping problem as a MDP and uses dynamic bipartite graph encoding to capture constraint and variable features, trained using evolution strategies.
- (4) In the selection of presolvers and raw heuristics [271], proposed a hierarchical sequence model framework for presolver combination selection, learning the optimal presolver set by predicting ratios at the higher level and determining combinations at the lower level. [272] proposed a strategy learning model based on the hierarchical sequence model for the configuration optimization of raw heuristics.

Additionally [273], conducted a systematic review of the fusion path of ML and RL in solving MIP, outlining the application status of DRL in branching, pruning, cutting planes, and stopping criteria. The review also analyzes algorithm embedding patterns, dataset construction methods, and evaluation metrics, providing a methodological summary for empowering exact algorithms with DRL.

Deep Reinforcement Learning Combined with General Heuristic Algorithms

DRL can guide and evaluate the search process of classical heuristic algorithms, overcoming the high computational cost and limited guidance ability of traditional methods. By replacing the evaluation function with a value or policy network trained by DRL, which quickly predicts the long-term rewards of candidate search actions, it enables more intelligent search guidance and global optimization.

Early research mainly focused on guiding the 2-Opt heuristic with DRL. [57] pioneered the use of a self-attention mechanism-based policy network to guide the 2-Opt search. As shown in Fig. 21, the model uses self-attention to embed and encode the nodes, capturing the global structure

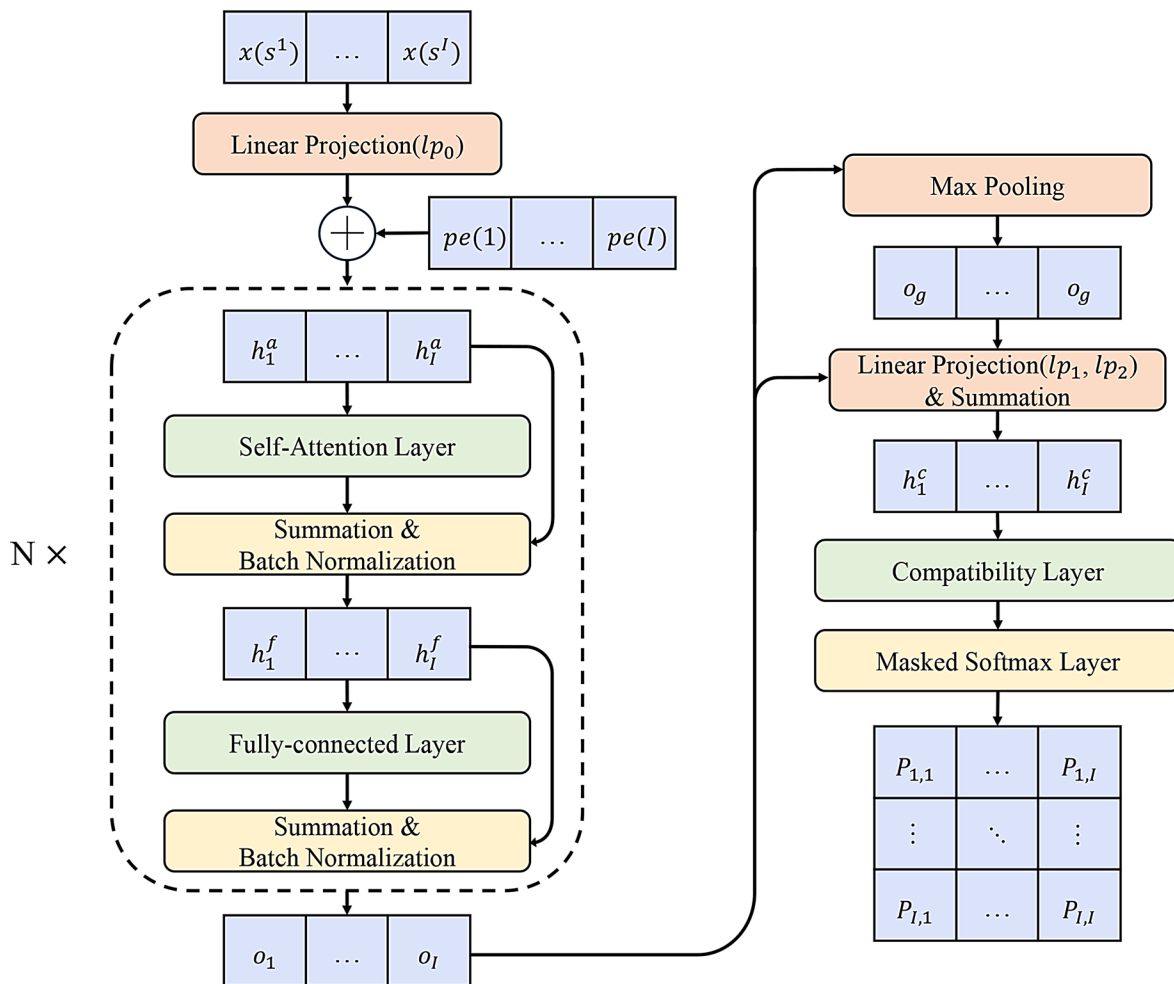


Fig. 21 Self-attention-based 2-opt policy network structure

between nodes and the dependencies between edges in the path. Then, the node selection module outputs a probability matrix of edge pairs to predict which edge pairs should be swapped, enabling the learning-driven 2-Opt operation. During training, a policy gradient method is used, with path length as the reward signal, guiding the network to identify high-value swap operations.

Building on Wu et al.'s work, subsequent research has made continuous improvements in encoder design, feature representation, and operation expansion. [274] combined GCN with LSTM networks, capturing both graph structure features and solution sequence context information, enhancing the encoder's ability to represent complex local patterns. [275] proposed the dual-aspect collaborative transformer (DACT) model, which uses a dual-stream transformer to separately model node and position features, avoiding feature mixing noise interference, and matrixes the output node pair distributions, significantly improving the guidance accuracy of 2-Opt operations. [276] extended this method to neural 3-Opt by introducing a sparse GCN and bidirectional LSTM encoder, and incorporating an edge selector and reconnection selector in the decoder to directly predict the deletion and reconnection of three edges, breaking the improvement limit of 2-Opt.

Further [277], proposed the flexible neural k-Opt framework, which guides the exchange process for variable k through neural networks and introduces a repair mechanism to transform high-quality infeasible solutions into feasible ones, significantly improving search flexibility and solution quality. On the other hand [201, 278], incorporated DRL into the LKH series of algorithms and proposed VSR-LKH, which dynamically guides edge selection and expansion through the policy network, replacing traditional fixed rules. Their subsequent work further extended to LKH-3 and various TSP variants, exhibiting stronger generalization ability in complex constraint scenarios.

In typical SPs such as the JSSP and FJSP, research on combining DRL with heuristic rules mainly focuses on two directions, parameter adaptation and dynamic rule selection. [279] addressed the total weighted tardiness problem in single-machine dynamic scheduling, using an apparent delay cost rule as the core heuristic and introducing deep deterministic policy gradient to dynamically adjust the parameter k . The model takes job processing time, weights, and relaxation time as inputs, outputs continuous adaptive parameter values, and stabilizes training through zero-bias optimization and experience replay mechanisms.

For complex dynamic scheduling scenarios such as semiconductor manufacturing [280], proposed a hybrid method based on edge computing and multiple DQN, dynamically selecting between seven heuristic rules to learn the matching relationship between system states and optimal rules.

In flexible scheduling [281], proposed a hierarchical DQN framework for FJSP with AGV transportation constraints. The high-level DQN is responsible for job selection, while the low-level uses the earliest finish time rule for machine and AGV allocation, thereby reducing the action space. [282] further investigated inference-time search enhancement and proposed a Hybrid-MCTS framework for FJSP, which combines pretrained GNN-DRL policies with Monte Carlo tree search. By enabling multi-step lookahead and refined candidate action selection, the framework improves solution quality and achieves a tunable quality–time trade-off without retraining the model.

In the area of rule optimization [283], proposed the RIDRL method, which uses PPO to dynamically select the optimal rule from 18 priority scheduling rules, and combines an Intelligent GA to generate semi-active scheduling solutions. [284] proposed seven composite rules for FJSP under machine failure, combining machine ranking with the D3QN framework for dynamic rule selection, exhibiting higher robustness. [285] proposed a DRL and heuristic hybrid optimization strategy for the delay minimization problem in Flow Shop Scheduling, using DRL to generate initial solutions and combining improved heuristic rules to optimize solution quality.

Deep Reinforcement Learning Combined with Metaheuristic Algorithms

Unlike the paradigm that integrates DRL with exact algorithms, the fusion of DRL and metaheuristics does not target global optimality. Instead, it improves the search and sampling abilities of metaheuristics through learning to seek higher-quality approximations in complex environments. The core idea is to insert a DRL-trained policy into the iteration or construction loop of a metaheuristic, adaptively tuning key control variables and guiding the search, reducing manual hyperparameter tuning and enabling data-driven exploration.

In routing [286], proposed *DeepACO*, a representative method. As shown in Fig. 22, the approach embeds a deep network into the classical ant colony optimization (ACO) framework to enhance pheromone guidance. The network learns structural features and route preferences between nodes, replacing the hand-crafted transition rule. The search then balances exploration and exploitation using both graph structure and trajectory history. Experiments show that DeepACO outperforms classical and improved ACO variants across multiple COPs. Gains are larger on large instances, with faster convergence and better solution quality.

Building on DeepACO [287], combined generative flow networks (GFlowNets) with ant-based construction. The sampler attains probabilistic consistency and high exploratory diversity. This reduces local-trap risks. [288] further

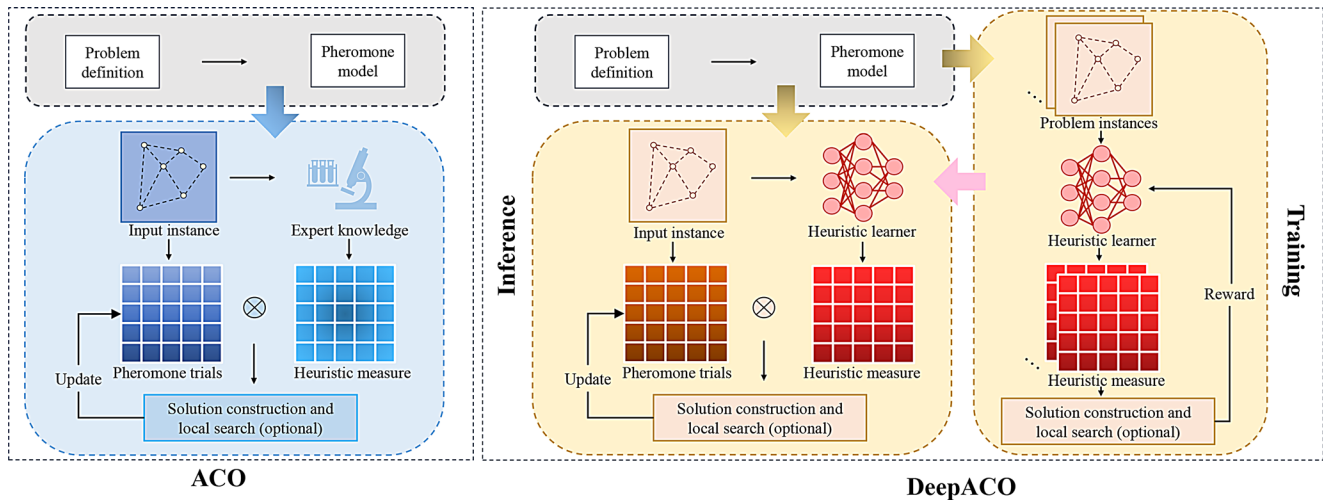


Fig. 22 Classical ACO vs. DeepACO

scheduled local-search operators via policy learning. A learned scheduler selects and composes operators conditioned on structure and search history. This improves efficiency and stabilizes convergence.

In complex scheduling scenarios, research has further investigated multi-layer fusion strategies combining DRL and metaheuristics. For instance [289], proposed a NichGP-assisted DRL framework for DFJSS, in which niching genetic programming first evolves a set of diverse and high-quality scheduling heuristics. These learned heuristics are subsequently employed as DRL actions, enabling the agent to select appropriate sequencing rules at different decision points. This approach reduces reliance on manually designed dispatching rules while enhancing both the diversity and effectiveness of the action space.

Similarly [290], introduced a dual-stage strategy integrating GTN and PPO for the integrated static-dynamic scheduling problem in MAJSP. In the static stage, GTN is utilized to optimize the differential ABC algorithm, capturing the relational features between operations and machines and minimizing ineffective searches. In the dynamic stage, PPO is employed for real-time policy adjustment, leveraging the learning acquired during the static stage to accelerate dynamic decision-making.

Furthermore [291], focused on machine utilization optimization, proposing a method that combines RL with D3QN and introduces a metaheuristic reward mechanism to mitigate the issue of sparse rewards.

Deep Reinforcement Learning Combined with Hyper-Heuristic Algorithms

Traditional hyper-heuristic algorithms achieve flexibility through high-level strategies and LLH, but their operator selection relies on manual experience and lacks adaptive learning capabilities. DRL, as the high-level control core, learns optimal selection strategies through state-action

mappings and combines the hierarchical advantages of hyper-heuristics with DRL's dynamic updates, balancing efficiency and solution quality.

To address the integrated optimization of processing and resource scheduling in multi-stage collaborative scheduling [292], proposed a DQN-based hyper-heuristic algorithm. This method first constructs a MIP model with the goal of minimizing the sum of makespans across all stages, defining coupling constraints between machines, transportation resources, and operations. Then a two-layer architecture is designed, with the lower layer including 12 local heuristics that cover both global exploration and local exploitation operators, and the high-level DQN module taking historical operator selection records, the number of solutions not recently updated, and global search states as inputs, using a reward function to evaluate both solution improvement and the ability to escape local optima and thereby dynamically select the optimal LLH.

5.3 Comparative Analysis of Reinforcement Learning Paradigms

Table 7 summarizes representative RL/DRL-based approaches for COPs, including classical Q-learning methods without neural networks and deep reinforcement learning methods with neural function approximation. Unlike supervised learning methods that rely on labeled solutions or expert decisions, RL/DRL methods learn optimization policies through reward feedback and interaction with the environment. Therefore, their main contribution is not to statically imitate optimal decisions, but to model COP solving as a sequential decision-making process and adapt search behavior according to optimization feedback. Existing methods can be broadly divided into two paradigms: end-to-end RL/DRL solvers, which directly generate routes,

Table 7 A comparative framework for deep reinforcement learning-based approaches in COP

Approach	Forms	Problems	Qual.	Scal.	Gen.	Eff.	Data Inde.
HHQL [223]	RL-OR/Hyperheuristic	EEDBFSP	★★☆	★★	★★	★★☆	★★☆
QHHEA [222]	RL-OR/Hyperheuristic	DFJSP	★★★	★★	★★	★★☆	★★☆
QHH-ABFSP [219]	RL-OR/Hyperheuristic	Distributed assembly blocking flowshop	★★☆	★★☆	★★☆	★★☆	★★★
HSA [218]	RL-OR/Hyperheuristic	Mixed-model sequencing	★★	★★	★★	★★	★★★
QHH [216]	RL-OR/Hyperheuristic	SAT, BP, FS, PS, TSP, VRP	★★	★★	★★★	★★☆	★★☆
RMOEA/D [214]	RL-OR/Metaheuristic	MOFFJSP	★★☆	★	★★	★★☆	★★☆
LCS [213]	RL-OR/Metaheuristic	FJSP with sequencing flexibility	★★	★★☆	★★	★★☆	★★★
LME0 [212]	RL-OR/Metaheuristic	FJSP-T	★★☆	★	★★	★★	★★☆
RL-GA [208]	RL-OR/Metaheuristic	Multiagent task scheduling	★	★★	★	★★☆	★★☆
SLGA [207]	RL-OR/Metaheuristic	FJSP	★★	★	★★	★★☆	★★☆
ReinforceNS [204]	RL-OR/Heuristic	TTP	★★☆	★★★	★★☆	★★	★★☆
RL-IG [203]	RL-OR/Heuristic	Energy-efficient FJSP	★★	★★	★★	★★☆	★★★
RAIG [202]	RL-OR/Heuristic	Production scheduling	★★	★★☆	★★	★★☆	★★☆
VSR-LKH [201]	RL-OR/Heuristic	TSP, TSPTW, CTSP	★★★	★★★	★★★	★★☆	★★☆
GPN [227]	E2E / Pointer-net	TSP, TSPTW	★★☆	★★☆	★★☆	★★★	★★☆
HPN [228]	E2E / Pointer-net	TSP	★★☆	★★☆	★★☆	★★★	★★☆
S2S-AC [123]	E2E / Pointer-net	DyJSP	★★☆	★★	★★	★★☆	★★★
DRL-FJSP [124]	E2E / Pointer-net	FJSP	★★	★★	★★	★★☆	★★★
ECO-DQN [231]	E2E / GNN	Max-Cut	★★★	★★★	★★☆	★★★	★★★
GREAT [232]	E2E / GNN	TSP, CVRP, OP	★★☆	★★	★★☆	★★	★★☆
L2D [233]	E2E / GNN	JSSP	★★☆	★★☆	★★☆	★★☆	★★★
GNN Scheduler [234]	E2E / GNN	JSSP	★★☆	★★☆	★★☆	★★	★★★
HGNN-DRL [239]	E2E / GNN	FJSP	★★☆	★★☆	★★☆	★★☆	★★★
MPGN [238]	E2E / GNN	FJSP	★★☆	★★☆	★★☆	★★☆	★★★
GraSP-RL [236]	E2E / GNN	JSSP, production planning	★★☆	★★	★★☆	★★☆	★★★
DGEAM-JSSP [237]	E2E / GNN	JSSP	★★★	★★☆	★★★	★★★	★★★
AM [55]	E2E / Transformer	TSP, CVRP, OP, PCTSP	★★☆	★★☆	★★	★★★	★★☆
MDAM [248]	E2E / Transformer	TSP, CVRP, OP, PCTSP	★★★	★★☆	★★	★★☆	★★☆
LEHD [249]	E2E / Transformer	TSP, CVRP	★★★	★★★	★★★	★★☆	★★
GLOP [251]	E2E / Transformer	TSP, ATSP, CVRP, PCTSP	★★★	★★★	★★☆	★★★	★★☆
MAAM [252]	E2E / Transformer	MVRPSTW	★★★	★★	★★☆	★★★	★★☆
DGERD Transformer [253]	E2E / Transformer	JSSP	★★☆	★★☆	★★☆	★★☆	★★★
Gated-Attention Model [257]	E2E / Transformer	DyJSP	★★☆	★★	★★☆	★★☆	★★★
SGFormer-SAC [258]	E2E / Transformer	Fuzzy FJSP	★★★	★★★	★★★	★★★	★★★
Retro Branching [264]	DRL-OR / Exact	MIP Branching	★★★	★★	★★	★★	★★
Branch Ranking [293]	DRL-OR / Exact	MILP branching	★★★	★★★	★★★	★★★	★★☆
HEM-Cut [294]	DRL-OR / Exact	MILP cut selection	★★★	★★☆	★★☆	★★★	★★☆
Presolver Selection [271]	DRL-OR / Exact	MILP presolver selection	★★☆	★★☆	★★☆	★★☆	★★☆
HYGRO / HEM++ [270]	DRL-OR / Exact	Cut stopping in MILP	★★★	★★★	★★★	★★★	★★☆
Learning Heuristics [272]	DRL-OR / Exact	Primal heuristics in MILP	★★☆	★★☆	★★☆	★★☆	★★☆
ML-Augmented B&B [273]	DRL-OR / Exact	B&B acceleration	★★★	★★★	★★★	★★★	★★☆
with 2-opt [57]	DRL-OR / General Heuristic	TSP, CVRP	★★★	★★	★★☆	★★	★★☆
DACT [275]	DRL-OR / General Heuristic	TSP, CVRP	★★★	★★	★★☆	★★	★★☆
Hybrid-MCTS [282]	DRL-OR / General Heuristic	FJSP	★★★	★★★	★★☆	★★☆	★★☆
MDQN [280]	DRL-OR / General Heuristic	JSSP	★★	★★	★★	★★☆	★★☆
HA-DQN [281]	DRL-OR / General Heuristic	FJSP-AGV	★★☆	★★☆	★★☆	★★★	★★★
RIDRL [283]	DRL-OR / General Heuristic	JSSP	★★	★★	★★	★★☆	★★☆
DeepACO [286]	DRL-OR / Metaheuristic	TSP, CVRP, OP, PCTSP, SMTWTP	★★☆	★★☆	★★☆	★★	★★☆
GFACS [287]	DRL-OR / Metaheuristic	TSP, CVRP, CVRPTW, PCTSP, OP	★★☆	★★☆	★★☆	★★	★★★
NeuroLS [288]	DRL-OR / Metaheuristic	JSSP, CVRP	★★☆	★★☆	★★☆	★★☆	★★★
NichGPDR [289]	DRL-OR / Metaheuristic	DFJSS	★★☆	★★☆	★★★	★★	★★★

Table 7 (continued)

Approach	Forms	Problems	Qual.	Scal.	Gen.	Eff.	Data Inde.
GTN-DABC [290]	DRL-OR / Metaheuristic	MAJSP with release dates	★★★	★★☆	★★☆	★★	★★★
HH-DQN [295]	DRL-OR / Hyper-Heuristic	FFSPDP	★★★	★★☆	★★☆	★★☆	★★★

scheduling actions, or operation sequences, and RL/DRL-OR collaborative methods, which embed Q-learning or DRL into exact algorithms, heuristics, metaheuristics, or hyper-heuristics to guide key optimization decisions.

Solution Quality: In terms of solution quality, end-to-end DRL methods can produce high-quality solutions when supported by effective structural representations. GNN- and Transformer-based models usually outperform early sequence-decoding methods because they better encode graph structures, precedence constraints, and global dependencies. For example, ECO-DQN improves graph optimization quality by combining GNN representations with exploratory rewards [231], while DGEAM-JSSP enhances scheduling decisions through graph embedding and attention-based global constraint modeling [237]. Transformer-based solvers further improve solution quality in complex scheduling scenarios; SGFormer-SAC, for instance, integrates a simplified graph Transformer with SAC for fuzzy FJSP [258]. In the collaborative paradigm, RL/DRL is mainly used to enhance critical search decisions rather than replace the whole optimization process. VSR-LKH improves LKH search by learning edge-selection strategies [201], and Branch Ranking improves MILP solving by learning branching priorities [293]. Overall, end-to-end methods are suitable for fast feasible-solution generation, whereas collaborative methods are more effective in improving solution quality while preserving the stability of classical optimization frameworks.

Scalability: The scalability of RL/DRL methods is mainly constrained by action-space growth, graph size, and attention computation. GNN-based methods improve cross-scale modeling by representing jobs, operations, machines, and constraints as graphs. For instance, L2D formulates JSSP as a disjunctive graph and learns dispatching policies across different instance sizes [233], while HGNN-DRL further improves scalability in FJSP by modeling operation-machine relations through heterogeneous graphs [239]. Transformer-based methods enhance global modeling ability, but often require structural simplification or hierarchical mechanisms to control computational cost. GLOP improves large-scale routing by combining coarse-grained partitioning with fine-grained construction [251], and SGFormer-SAC reduces the computational burden of attention-based modeling through graph sparsification and linear attention [258]. In collaborative methods, scalability is usually achieved through search-space reduction or inference-time

search enhancement. Hybrid-MCTS combines pretrained GNN-DRL policies with Monte Carlo tree search to provide a tunable quality–time trade-off for FJSP [282], while HYGRO/HEM++ improves MILP scalability by learning cut-stopping strategies [270].

Generalization: Although RL/DRL methods do not rely on fixed labels, their generalization is still affected by training distributions, state encodings, and action definitions. End-to-end models trained on specific instance scales or structures may still degrade when transferred to new settings. To improve generalization, some methods introduce structural representations, multi-stage training, or policy diversity. GREAT combines supervised pretraining with RL fine-tuning to enhance cross-distribution adaptability in graph optimization tasks [232], while LEHD strengthens transfer from small-scale training to large-scale routing through a light-encoding and heavy-decoding architecture [249]. In scheduling problems, DGEAM-JSSP and HGNN-DRL also improve cross-instance adaptability by encoding structural constraints rather than relying on fixed-size state features [237, 239]. In contrast, collaborative methods often generalize more robustly because the underlying optimization or search framework remains valid, while the learning component only controls local decisions or strategy selection. QHH uses Q-learning for low-level heuristic selection and shows promising cross-domain transferability [216], while NeuroLS learns local search control policies and improves adaptability across different COP settings [288].

Computational Efficiency: End-to-end DRL methods can make fast decisions after training, which makes them suitable for real-time or near-real-time optimization. The Attention Model directly generates routing solutions through attention-based policies, reducing repeated calls to traditional solvers [55], while MDAM improves inference effectiveness through multi-decoder policy diversification [248]. In scheduling scenarios, DGEAM-JSSP improves decision efficiency by combining graph embedding with attention-based state modeling [237]. However, this efficiency advantage may be weakened when graph construction, attention computation, or decoding becomes expensive. RL-OR collaborative methods improve efficiency mainly by reducing costly components in traditional solvers. Presolver Selection accelerates MILP solving by learning suitable presolver combinations [271], while Branch Ranking reduces search overhead by improving branching decisions [293]. For heuristic collaboration, HA-DQN reduces the action space in

FJSP-AGV through hierarchical decision-making [281], and lightweight Q-learning-based hyper-heuristics can improve operator selection and low-level heuristic scheduling with limited additional computational overhead.

Summary: Overall, Table 7 shows that RL/DRL provides an adaptive decision-making paradigm for COPs that differs from supervised learning. End-to-end RL/DRL solvers can rapidly generate routing or scheduling decisions and are suitable for structured problems with strong real-time requirements. However, their performance is still constrained by state representation, action-space design, and training distribution. In contrast, RL-OR collaborative methods embed learned policies into branching, pruning, local search, operator selection, and low-level heuristic control, achieving a more balanced performance across solution quality, scalability, generalization, and computational efficiency. Therefore, the main value of RL/DRL in COPs is not to fully replace classical optimization algorithms, but to serve as an adaptive controller that provides dynamic decision support within structured optimization workflows.

6 Unsupervised Learning for Combinatorial Optimization Problems

UL provides an alternative class of solutions for COPs by directly exploiting inherent structural constraints and objective functions, enabling models to learn latent solution distributions or optimization patterns without labeled data. As introduced in Sect. 3, generative models also fall under the UL umbrella and have been widely applied to COPs. Under this paradigm, methods can be broadly categorized into two routes: generative models and discriminative models.

Generative model approaches aim to explicitly learn solution distributions or latent variables through autoregressive models, VAEs, or diffusion models, sampling in the solution space followed by rejection sampling, re-scoring, or post-processing (see Fig. 7). Their strength lies in naturally producing diverse solutions compatible with uncertainty modeling. However, they require complex discrete or hybrid sampling mechanisms, incur relatively high training and inference costs, and most existing methods are not purely unsupervised, typically requiring labels from OR solvers.

Discriminative methods, by contrast, treat NN as approximate solvers or energy minimizers, using the original objective function as the training signal. Through continuous relaxation, policy gradients, or margin-based losses, they learn direct mappings from graphs to high-quality solutions. These approaches feature simpler training objectives, more direct optimization pathways, and good scalability, without

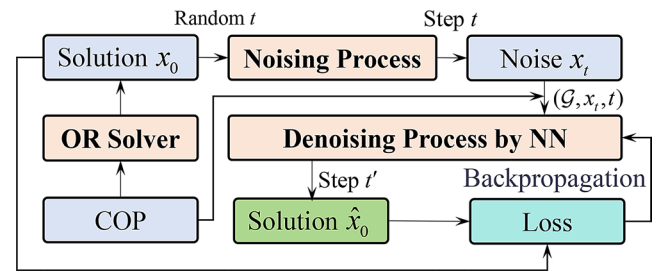


Fig. 23 Label-dependent diffusion model approach

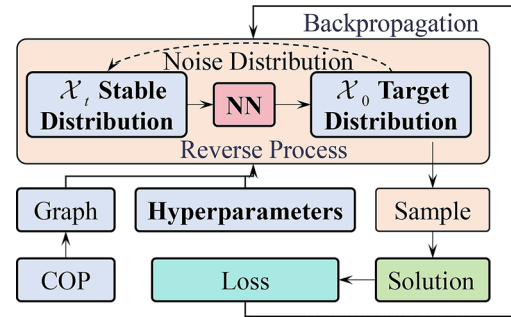


Fig. 24 Label-free diffusion model approach

requiring OR-generated labels. The following subsections elaborate on both perspectives.

6.1 Generative Model Methods

6.1.1 Diffusion Models

The core paradigm casts discrete optimization as probabilistic generative modeling: high-quality feasible solutions are gradually approached via iterative denoising. A forward noise-injection process and a corresponding reverse denoising process are defined to learn the target distribution over the solution space. Unlike image generation, where high-quality training data are abundant, obtaining high-quality COP solutions is considerably more difficult. Early studies reduced but did not eliminate the reliance on labeled data [58], as illustrated in Fig. 23.

With further advances, genuinely label-free methods have emerged: a fixed noise distribution is predefined, and the network is trained solely to denoise from a stationary distribution to the target, enabling near-optimal COP solutions via sampling. A typical pipeline is shown in Fig. 24.

[296] explores how pretrained diffusion models can serve as priors for combinatorial optimization by repeatedly evaluating data-point scores under different noise conditions. Building on this idea, DIFUSCO [58] is the first to systematically apply graph denoising diffusion models to NP-complete COPs. It formulates a discrete $\{0,1\}$ -vector optimization framework with a dual Gaussian/Bernoulli noise mechanism tailored to TSP and MIS. TSP solutions are

represented as adjacency matrices and iteratively denoised using a GCN. An innovative noise scheduling strategy dynamically adjusts noise levels to balance exploration and convergence, while a self-consistent multi-sample inference scheme further improves solution quality.

This pioneering work has inspired numerous follow-up studies. [297] addresses DIFUSCO's inference latency through progressive distillation, compressing multi-step denoising into few-step predictions that achieve equivalent refinement with significantly faster inference. [298] extends diffusion models to two-dimensional irregular bin packing via an attention-based gradient-field learning architecture with geometric encoding for polygon rotations and boundary-condition constraints throughout diffusion, surpassing RL baselines in material utilization while improving inference speed by an order of magnitude.

A further milestone is DiffUCO [299], the first to apply diffusion models to COPs in a fully label-free setting. By constructing a Joint Variational Upper Bound, it enables unsupervised training without tractable likelihoods, combining discrete diffusion with temperature annealing, GNN architectures, and improved conditional-expectation sampling. [300] further eliminates DIFUSCO's data dependence through the IC/DC framework, which directly optimizes the objective via self-supervised losses with embedded constraint-satisfaction mechanisms. For TSP-specific optimization [301], develops DEITSP, integrating a dual-modal graph Transformer for accelerated feature fusion with an iterative exploration strategy.

6.1.2 Generative Adversarial Networks

GANs are typically employed in combination with OR algorithms. The generator learns distributional characteristics of high-quality solutions, while the discriminator evaluates candidate quality; through adversarial training, generated solutions are injected into traditional optimizers (e.g., GA or local search) to enhance global search capability. This framework escapes local optima and alleviates sparse-reward issues common in conventional DRL [302], significantly improving efficiency on large-scale problems [303]. A typical framework is illustrated in Fig. 25.

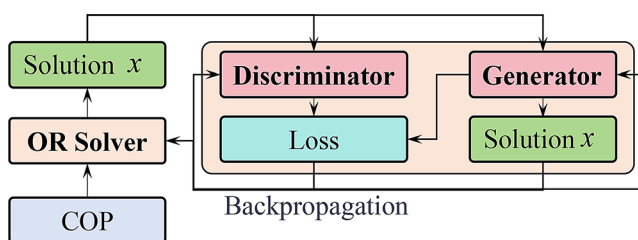


Fig. 25 GAN-based approach

[302] proposes the GIRL framework, which first integrates inverse RL with GANs. A self-attention-based policy network generates 2-opt heuristic rules, and a soft actor-critic algorithm jointly trains the discriminator, achieving end-to-end optimization without manually designed reward functions. [303] introduces the Adversarial Generative Flow Network, replacing the Transformer with a generative flow network to produce diverse routing solutions, where the discriminator improves quality by contrasting original solutions with locally optimized counterparts. For multi-objective optimization [304], uses non-dominated solution sets as real data for GAN training to learn the Pareto front distribution, achieving a 35% speedup over NSGA-II on the group-aware participant selection problem. The DCG-EA/I algorithm [305] addresses mode collapse through a negative-sample exploitation mechanism that reconstructs low-quality solutions into pseudo-positive samples, integrated with 2-opt local search for enhanced feasibility.

Further advances include [306]: combines policy gradients with Monte Carlo simulation in the PGAN-CEA framework, improving discrete solution generation quality [307]; applies a hybrid GAN-GA model to three-dimensional bin packing, improving packing utilization by 4.7%. Overall, GAN-augmented frameworks exhibit substantial potential in overcoming local convergence and insufficient solution diversity.

6.1.3 Variational Autoencoders

VAEs mitigate the curse of dimensionality inherent in high-dimensional discrete COPs by mapping the solution space to a continuous latent space, establishing a unified encode–optimize–decode framework. Discrete optimization problems are reformulated as probabilistic generative tasks: an encoder compresses solutions into latent representations, an embedded optimizer searches for high-quality feasible solutions, and a decoder reconstructs discrete solutions. Consequently, VAE-based methods are usually combined with other optimization algorithms. Figures 26 and 27 illustrate typical frameworks integrating VAEs with RL and meta-heuristics, respectively.

[308] proposes COIL, the first systematic integration of VAEs with optimizers for COPs. A VAE encoder compresses historical robotic scheduling plans into a low-dimensional latent distribution. Then, a GA searches in this latent space to maximize delivery throughput, with sampling ranges directly embedding robot parallelism constraints so that decoded schedules satisfy monitoring capacity limits. [309] extends COIL to dynamic network environments for service function chain deployment in 5G core networks, where a VAE compresses network states into latent representations upon which a DRL agent learns placement policies, with

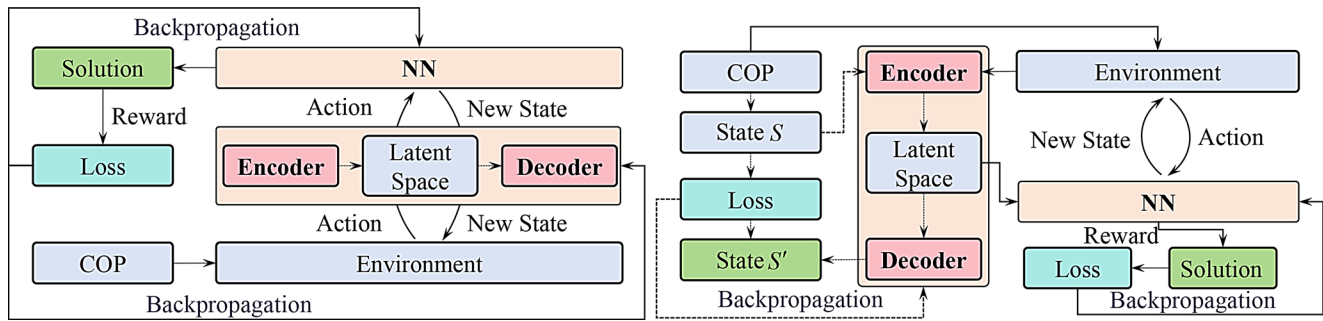


Fig. 26 A framework combining VAEs with DRL

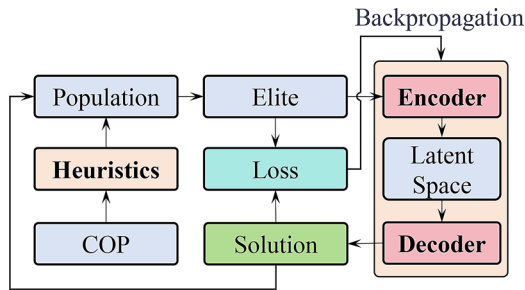


Fig. 27 A framework combining VAEs with heuristic algorithms

an end-to-end latency penalty satisfying sub-10 ms delay requirements.

[310] enhances COIL with the TD3 algorithm for continuous action spaces in active distribution network scheduling with distributed wind power. The VAE encodes multi-site energy-storage states into latent variables capturing spatio-temporal coupling, and the TD3 agent outputs charging/discharging strategies minimizing system operating costs. [311] embeds a VAE into an estimation-of-distribution algorithm, proposing VAE-EDA-Q for complex benchmarks such as Trap-k; adaptive variance scaling dynamically regulates sampling diversity, achieving a fivefold convergence speedup over BOA on Trap-13 with superior solution quality.

In communication resource allocation, several studies adopt conditional VAEs to integrate dynamic features. The CARMARL framework [312] jointly optimizes discrete-continuous hybrid decisions for task offloading and resource allocation with reward-constrained policy optimization, yielding a 25.2% increase in intelligent connected vehicle task completion rates. [313] uses a CVAE for user mobility prediction and small-cell content pre-caching to reduce handover latency. [314] combines VAEs with contrastive learning for semi-supervised O-RAN slicing resource allocation, improving PRB allocation accuracy by 14%.

For hybrid action-space modeling [315], proposes a digital-twin-based DT-VAE evaluation mechanism for closed-loop industrial sensor resource optimization. [316] introduces a self-attention-based CVAE-GAN routing

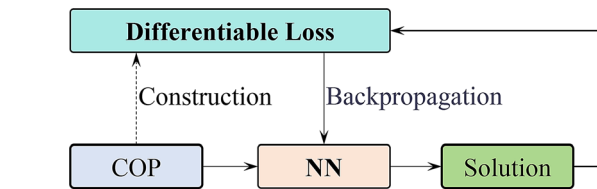


Fig. 28 A framework based on the discriminative model

paradigm, achieving a 22% energy consumption reduction for multipath routing in MANETs.

6.2 Discriminative Model Methods

The core idea is to embed the optimization objective directly into differentiable neural network training, learning near-optimal solutions without labeled data. Researchers typically adopt GNNs as the primary modeling tool, representing COPs as graph or hypergraph structures and extracting structural constraints via learnable message passing. Continuous relaxation or policy gradient methods then map COPs to differentiable objectives for end-to-end training. A common pipeline is illustrated in Fig. 28.

[317] first investigates unsupervised GNN performance on random MAX-CUT, demonstrating that GNNs can approximate the SDP benchmark without label supervision. [318] proposes a mathematically grounded framework that parameterizes distributions over sets using neural networks and optimizes an unsupervised loss function ensuring, with controllable probability, that the learned distribution contains low-cost feasible integer solutions. A derandomization strategy converts the existence guarantee into deterministic decoding, providing theoretical guarantees for both quality and feasibility across diverse constraint types (cliques, vertex covers, independent sets).

The physics-inspired GNN by [319] introduces Hamiltonian minimization from statistical physics into GNN architectures, reformulating COPs as QUBO tasks. A physics-inspired energy function translates the combinatorial objective into a network loss for fully unsupervised training, with continuous relaxation supporting gradient-based

optimization of binary variables. This work lays important theoretical and algorithmic foundations for direct COP solving with GNNs.

These methods have inspired substantial follow-up work on training stability, discrete-constraint expressiveness, and generality. [320] develops a general unsupervised recurrent GNN for binary maximum constraint satisfaction using a constraint graph with a lightweight LSTM unit to directly maximize satisfied constraints. [321] proposes an AST-based encoding framework that automatically maps combinatorial constraints to graph structures via a generic injective representation function, unifying diverse problems (SAT, graph coloring, TSP, knapsack) and supporting structural knowledge sharing across tasks.

To address convergence instability across task groups [322], introduces the Deep-Grouping framework, unifying graph and hypergraph partitioning with a Gini-index-based continuous relaxation annealing strategy that balances differentiable optimization and discrete convergence by dynamically adjusting output discretization levels. [323] proposes DyCO-GNN for dynamic combinatorial optimization, equipped with temporally continuous structural embeddings and GNN parameters shared across temporal snapshots for fast updates on continuous-time graphs without offline training data. [324] presents a permutation-invariant method for the quadratic assignment problem, introducing permutation-symmetric constraints to enhance generalization and scalability.

[325] proposes ULND for unsupervised cost-constrained network dismantling. Using a GAT with skip connections, ULND aggregates multi-hop neighborhood information to produce node dismantling scores. Its unsupervised objective minimizes removal cost while maximizing cascading effects via a neighbor-product term. Trained on small synthetic networks (30–50 nodes), it generalizes to large real-world networks (up to 20,000 nodes), with cost-aware reverse greedy fine-tuning further refining non-scale-free cases. Experiments on 15 real-world networks show ULND outperforms FINDER, GDM, GASC, and CoreHD, particularly on scale-free and transportation networks.

Overall, discriminative-model-based approaches have become a major trend in combinatorial optimization, exhibiting substantial potential in computational efficiency, scalability, and general applicability.

6.3 Comparative Analysis of Unsupervised Learning Paradigms

This subsection provides a systematic comparison of the four unsupervised learning paradigms—Diffusion models, VAEs, GANs, and Discriminative models—across five key dimensions: solution quality (Qual.), scalability (Scal.),

generalization (Gen.), computational efficiency (Eff.), and data independence (Data Inde.).

Solution Quality: Diffusion-based methods achieve the highest solution quality. DIFUSCO attains very low optimality gaps on TSP [58], while DEITSP maintains consistently low gaps [301]. DiffUCO, operating in a label-free setting, achieves low to moderate gaps on MIS and MaxCut problems [299]. Discriminative models (PI-GNN, RUN-CSP) achieve comparable quality, matching or exceeding SDP baselines on MaxCut benchmarks [319, 320]. GAN-based methods (GIRL) reach negligible gaps on TSP-100 and CVRP-100 benchmarks [302], while VAE-based methods typically serve as initialization modules with moderate standalone quality [308].

Scalability: Discriminative models excel in this dimension. PI-GNN scales to very large graphs with near-linear runtime [319]. Diffusion models handle large TSP instances but incur higher computational costs due to iterative denoising [58]. GAN-based and VAE-based methods typically operate at smaller scales [302, 324], with training stability and discrete output spaces as primary bottlenecks [305].

Generalization: Diffusion models demonstrate superior cross-scale generalization. DIFUSCO, trained on small TSP instances, generalizes effectively to much larger instances, maintaining solution quality near optimal [58]. Discriminative models generalize from small to large instances but degrade under distribution shift [318, 319]. GAN-based and VAE-based methods exhibit the weakest generalization, often requiring retraining for different scales [302, 311].

Computational Efficiency: Discriminative models are most efficient at inference, solving very large instances in short time via single forward passes [319, 323]. Diffusion models incur higher costs (moderate time for medium TSP instances, very fast for CVRP instances with optimized sampling) [58, 303]. GAN-based methods show intermediate efficiency [302], while VAE-based methods achieve fast inference once trained, as decoding requires only a single forward pass [308, 324].

Data Independence: Discriminative models achieve the highest data independence, operating fully unsupervised with only problem definitions [318, 319, 323]. Diffusion models exhibit mixed independence: some rely on solver-generated labels [58, 301], while others achieve true unsupervised training [299, 300]. GAN-based methods occupy an intermediate position, requiring expert demonstrations [302] or using evolution-generated samples [304]. VAE-based methods generally require moderate amounts of historical data but not optimal solutions [308, 311].

Summary: Table 8 summarizes the paradigm-level comparison. Discriminative models offer superior scalability, efficiency, and data independence, making them suitable for large-scale, label-scarce applications. Diffusion models

Table 8 A comparative framework for UL methods in COP

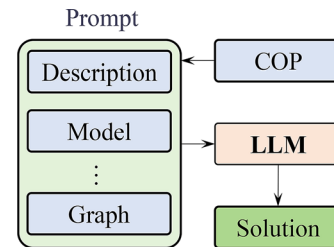
Approach	Forms	Problems	Qual.	Scal.	Gen.	Eff.	Data Inde.
DIFUSCO [58]	Diffusion	TSP	★★☆	★★☆	★★☆	★☆	★
DiffUCO [299]	Diffusion	MIS, MaxCut, MDS, MaxCI, MVC	★★☆	★★	★★☆	★★	★★★
DEITSP [301]	Diffusion	TSP	★★☆	★★☆	★★☆	★★★	★
IC/DC [300]	Diffusion	ATSP	★★☆	★★	★★☆	★★	★★★
COIL [308]	VAE	Robot Scheduling	★☆	★	★★	★☆	★☆
VAE-EDA-Q [311]	VAE	Trap-k, NK Landscapes	★☆	★	★★	★☆	★☆
CARMARL [312]	VAE (CVAE)	Task Offloading, Resource Allocation	★☆	★☆	★	★★	★☆
SS-VAE [314]	VAE (CVAE)	O-RAN Slicing	★☆	★☆	★	★★	★☆
GIRL [302]	GAN	TSP, CVRP	★★☆	★	★★	★☆	★☆
MOCGAO [304]	GAN	Multi-objective CO	★★	★☆	★★	★☆	★★★
DCG-EA/I [305]	GAN	TSP	★☆	★	★	★☆	★☆
PGAN-CEA [306]	GAN	TSP	★☆	★	★	★☆	★☆
GAN-GA [307]	GAN	3D Bin Packing	★☆	★	★	★☆	★☆
AGFN [303]	GFlowNet	CVRP, TSP	★★☆	★★☆	★★	★★★	★★★
PI-GNN [319]	Discriminative	MaxCut, MIS, Graph Coloring	★★☆	★★★	★★	★★★	★★★
RUN-CSP [320]	Discriminative	Max-2-SAT, MaxCut, 3-COL, Max-IS	★★☆	★★	★★☆	★★★	★★★
Erdős GNN [318]	Discriminative	Max Clique, Graph Partition	★★	★★	★☆	★★	★★★
Deep-Grouping [322]	Discriminative	Graph/Hypergraph Partitioning	★★☆	★★☆	★★	★★	★★★
DyCO-GNN [323]	Discriminative	Dynamic MaxCut, MIS, TSP	★★☆	★★☆	★★☆	★★★	★★★
PLUME search [324]	Discriminative	QAP	★☆	★	★☆	★★	★☆
Generic Graph Encoding [321]	Discriminative	SAT, TSP, GC, Knapsack	★★☆	★☆	★★☆	★★	★☆
ULND [325]	Discriminative	Network Dismantling	★★☆	★★☆	★★★	★★	★★★

deliver the highest solution quality and generalization capability at the cost of computational efficiency. GAN-based methods provide a flexible middle-ground solution with moderate data requirements. VAE-based methods are best suited for accelerating existing optimization pipelines or providing warm-start initializations. Unsupervised learning paradigms will receive increasing attention in the future due to their low data dependence and outstanding solution quality.

7 LLM for Combinatorial Optimization Problems

As introduced in Sect. 3, LLMs possess strong semantic understanding, reasoning capabilities, and flexible natural-language interaction, opening transformative paradigms for COPs. They can directly interpret optimization objectives and constraints from natural language, automatically perform modeling and feature extraction, and serve in multiple roles: as optimizers proposing candidate solutions, as programmers synthesizing heuristic algorithms, or as collaborators coupled with traditional solvers. Their natural-language generation ability further enables clear explanations and multi-perspective evaluations of optimization results, improving interpretability and practical usability.

Researchers are actively exploring intelligent expert systems that tightly integrate LLMs with domain knowledge

**Fig. 29** A framework based on LLM as Optimizer

and OR solvers, pushing combinatorial optimization toward an end-to-end paradigm: natural-language input, automatic modeling, automatic solving, and human-centric explanation.

7.1 Solving Combinatorial Optimization Problems

7.1.1 LLM as Optimizer

This paradigm treats the LLM itself as the iterative optimization agent. The OPRO framework [59], illustrated in Fig. 29, formulates optimization problems through natural-language meta-prompts, enabling the LLM to autonomously generate candidate solutions based on historical results and objective values. OPRO eliminates dependence on formal mathematical modeling and achieves performance comparable to specialized solvers under zero-shot conditions,

systematically verifying the feasibility of LLMs as general-purpose optimizers.

Subsequent research has explored multimodal and multi-agent strategies to enhance performance. [326] introduces a multimodal perception mechanism fusing XML-structured text with node-distribution images to strengthen the optimizer's understanding of spatial topology in high-dimensional problems. [327] proposes a multi-agent collaborative framework: Multi-agent 1 (initializer, evaluator, assessor) performs fine-grained optimization through role specialization, while Multi-agent 2 (initializer, evaluator) focuses on fast decision-making, achieving zero-shot m-TSP performance comparable to self-supervised methods. [328] constructs the first large-scale natural-language JSSP dataset (120k samples) and, combined with LoRA fine-tuning, achieves worktime optimization on par with GNNs. More recently [329], proposes an LLM-based parallel optimizer for joint port selection and beamforming in multiuser MISO-FAS, using GPT-2 with LoRA and Gumbel-Sinkhorn relaxation for end-to-end unsupervised training, outperforming CNN, Transformer, and sequential LLM baselines by up to 6.44 bps/Hz with millisecond-level latency.

7.1.2 LLM as Programmer

This paradigm leverages LLMs' code generation capability to translate natural-language descriptions and algorithmic templates into heuristic programs, which are iteratively refined through evolutionary operations with feedback-based selection. As shown in Fig. 30, the framework uses natural-language instructions to drive algorithm design while exploiting the LLM's generalization to explore diverse heuristic spaces, greatly reducing dependence on domain experts.

The FunSearch framework [60] marks a foundational milestone, introducing an evaluator-generator cooperative paradigm that operates directly in the function space rather than the solution space. The evaluator systematically filters code fragments, guiding the LLM-based generator to iteratively optimize programs, surpassing handcrafted algorithms on online bin packing and demonstrating that LLM-driven program evolution can outperform expert-designed heuristics.

Building upon FunSearch, several studies propose multi-dimensional improvements. [330] introduces EoH, employing dual-track evolution to simultaneously optimize natural-language descriptions and executable code, achieving superior quality and speed on bin packing. [331] presents a self-evolving meta-optimizer that autonomously constructs low-order mutation strategies, enhancing cross-scale generalization for TSP and bin packing. [332] proposes ReEvo, integrating evolutionary search with LLM

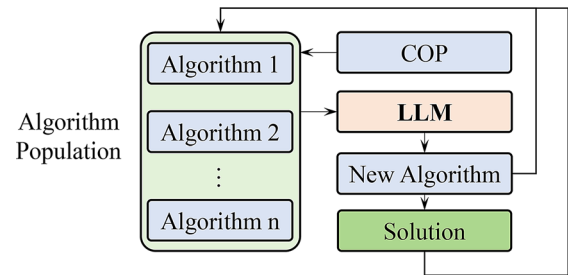


Fig. 30 A framework based on LLM as Programmer

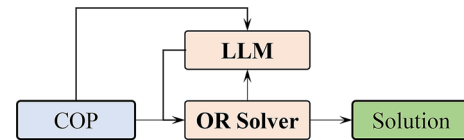


Fig. 31 A framework based on LLM as Collaborator

self-reflection to generate semantic gradients that surpass traditional hyper-heuristics. [333] introduces MCTS-AHD, using Monte Carlo Tree Search to manage heuristic evolution trees, retaining promising but underexplored candidates, with notable success on VRP.

Further advances span training strategies, problem adaptability, and framework expansion. [334] proposes HSEvo, integrating harmonic search to maintain population diversity. The SeEvo framework [335] surpasses genetic programming in dynamic task scheduling through self-evolutionary adaptation. For problem adaptability [336], introduces DRoC, a constraint-decomposition retrieval mechanism for complex VRP instances, while [337] enhances exploration efficiency through a quality-uncertainty balance criterion. Notably [338], analyzes code evolution trajectories, uncovering the trade-off between code complexity and performance, providing interpretability tools for LLM-based algorithm design.

7.1.3 LLM as Collaborator

Beyond direct optimization and algorithm generation, researchers have explored hybrid frameworks integrating LLMs with RL and neural solvers to enhance efficiency and generalization. Figure 31 illustrates a typical hybrid framework.

The EvoTune framework [339] integrates RL into an LLM-based evolutionary search loop. The system explores and evaluates LLM-generated algorithmic programs through evolutionary search, constructs preference data pairs based on performance, and fine-tunes the LLM via Direct Preference Optimization. This closed feedback mechanism enables the LLM to generate progressively higher-quality algorithms, outperforming FunSearch and significantly

narrowing the optimality gap under fixed computational budgets.

[340] incorporates LLMs into an evolutionary framework where the best generation strategies are transformed into attention bias signals injected into pretrained neural solvers. Fine-tuning with a small diverse set of large-scale instances effectively transfers LLM-produced strategic knowledge. On both synthetic and real-world TSP and CVRP datasets, this method achieves state-of-the-art performance, markedly improving generalization and solution quality for problems involving thousands of nodes.

7.2 Modeling and Explaining Combinatorial Optimization Problems

7.2.1 Modeling

A major emerging direction leverages LLMs to automatically translate natural-language descriptions into mathematical formulations of COPs, typically following a three-stage pipeline: problem understanding (extracting decision variables, constraints, and objective terms via named entity recognition), intermediate representation construction (dependency graphs and templated constraints), and mathematical model generation (LP or MILP form). This framework greatly reduces domain expertise requirements while improving modeling efficiency.

The foundational work by [341] established the NL4Opt competition dataset (713 LP problems across multiple domains), introduced constraint-decoding-based controllable generation for output format consistency, and developed a human-in-the-loop visual interface, setting benchmarks and highlighting key challenges in document-level semantic parsing.

Building on this [342], proposed a three-phase MILP synthesis framework with a constraint classification hierarchy and template library for handling logical constraints, significantly outperforming ChatGPT on the extended NL4Opt dataset. [343] developed LiP-LLM for multi-robot task planning, integrating LLM-based semantic parsing with LP solvers via skill-dependency graphs. [344] proposed LM4OPT progressive fine-tuning with noise-injected embeddings and domain-adaptive datasets, substantially narrowing the gap between 7B-parameter models and trillion-parameter models on NL4Opt.

Additional advances include [345]: proposed entity-type-based label embedding for enhanced intermediate representations [346]; combined conditional random fields with multi-prompt T5 generators and data augmentation [347]; introduced adversarial training and prompt engineering for cross-domain adaptability; and [348] extended the

paradigm by translating natural-language preferences into soft constraints for human–LLM collaborative optimization.

7.2.2 Explaining

LLMs show transformative potential for enhancing interpretability in combinatorial optimization by bridging mathematical abstraction and human cognition. In feasibility diagnosis, OptiChat [349] combines optimization solvers with chain-of-thought prompting to automatically generate natural-language explanations of constraint conflicts by identifying irreducible infeasible subsets, substantially reducing dependence on domain experts.

Multi-agent collaborative frameworks address challenges from implicit constraints and domain-specific terminology. The Chain-of-Experts (CoE) framework [350] deploys three specialized agents (terminology explanation, model construction, program implementation) under a coordinator performing forward reasoning and backward reflection, achieving significantly lower error rates than single-agent approaches. [351] proposes a multi-stage verification framework with a relation-identification agent for variable–constraint consistency checking, doubling verification efficiency without solver invocation.

Self-reflection mechanisms address instability in direct LLM outputs. For VRP [352], designs a self-debugging and self-verification framework that markedly improves GPT-4’s feasibility, optimality, and efficiency, while revealing capability discrepancies across models (Gemini 1.0 Pro fails to benefit similarly) and the critical role of linguistic precision, as omitting task details degrades GPT-4’s performance even when semantics remain unchanged.

The LLMFP framework [353] unifies planning problems by transforming them into constraint optimization models, enabling zero-shot generation of structured representations and executable solver code from natural-language descriptions. Future directions include lightweight fine-tuning to reduce reliance on proprietary models, hybrid LLM–solver architectures balancing flexibility and rigor, and interactive dialogue for iterative problem refinement [354].

7.3 LLM-Based Combinatorial Optimization Expert Systems

These systems leverage LLMs to transform natural-language problem descriptions into executable solutions end-to-end while providing human-understandable explanations, following a four-stage pipeline: natural-language parsing, mathematical modeling, solver code generation, and iterative verification. Their key advantages include automated closed-loop mechanisms, integrated self-debugging for

Table 9 A comparative framework for LLM-based approaches in COP

Approach	Forms	Problems	Qual.	Scal.	Gen.	Eff.	Data Inde.
OPRO [59]	LLM as Optimizer	TSP, LP, Prompt	★★	★	★☆	★☆	★
MLLM Optimization [326]	LLM as Optimizer	CVRP	★★	★☆	★★	★☆	★★★
Multi-Agent MLLM [327]	LLM as Optimizer	TSP, mTSP	★★	★☆	★★	★★☆	★★★
LLM for JSSP [328]	LLM as Optimizer	JSSP	★★	★☆	★★	★★	★
MISO-FAS [329]	LLM as Optimizer	MISO-Fluid Antenna Systems	★★	★★	★★	★★★	★★☆
FunSearch [60]	LLM as Programmer	Cap Set, Admissible, BPP	★★☆	★★☆	★★☆	★★	★
EoH [330]	LLM as Programmer	TSP, BPP, FSSP	★★☆	★★☆	★★☆	★★☆	★☆
MoH [331]	LLM as Programmer	TSP, BPP	★★☆	★★☆	★★★	★★	★
ReEvo [332]	LLM as Programmer	TSP, CVRP, OP, MKP, BPP, DPP	★★☆	★★☆	★★☆	★★☆	★☆
MCTS-AHD [333]	LLM as Programmer	TSP, KP, CVRP, MKP, BPP, ASP, BO	★★☆	★★	★★	★★	★★
HSEvo [334]	LLM as Programmer	BPO, TSP, OP	★★☆	★★	★★☆	★★☆	★☆
SeEvo [335]	LLM as Programmer	DJSSP	★★☆	★★	★★★	★★☆	★☆
DRoC [336]	LLM as Programmer	VRP (48 variants)	★★☆	★★	★★	★★★	★☆
QUBE [337]	LLM as Programmer	BPP, Cap Set, TSP	★★☆	★★	★★☆	★★	★★
Code Evolution Graphs [338]	LLM as Programmer	BBO, BPP, TSP	★★	★★	★★	★★	★★★
EvoTune [339]	LLM as Collaborator	BPP, TSP, Flatpack	★★	★★	★★	★★	★★
NCO+LLM Attention [340]	LLM as Collaborator	TSP, CVRP	★★★	★★☆	★★★	★★★	★☆

reliability, and broad problem coverage spanning LP, integer programming, and RPs [61, 355].

OptiMUS [61] pioneers a modular multi-agent architecture that parses natural language to extract variables and constraints, generates mathematical models, develops and debugs solver code, and iteratively refines models based on solution feedback. The framework adopts a divide-and-conquer strategy for long problem descriptions and integrates chain-of-thought prompting for logical coherence, demonstrating that LLMs can in principle replace human experts throughout the modeling–solving–explanation pipeline.

Building on this foundation, LLMOPT [356] introduces a five-element unified representation covering six problem classes with multi-instruction fine-tuning and self-correction mechanisms, achieving substantial accuracy gains across 20 domains. [357] designs a localized knowledge-enhanced framework for industrial privacy requirements, guiding local LLMs to generate mixed-integer programming code via domain knowledge bases without cloud data transfer. [358] proposes a symbolic translation–solver co-framework fine-tuning LLMs to translate natural-language requests into symbolic MILP constraints, reducing response latency.

Further contributions address data scarcity, task-specific evaluation, and benchmarking. [359] proposes ReSocratic, a reverse synthesis method generating 29,000 optimization instances, enabling open-source models (Llama-3-8B) to approach GPT-4–level performance. [360] systematically studies robotic routing models, finding that self-debugging increases TSP success rates by 18% while injecting additional mathematical background reduces success rates by 15%. [361] develops CO-Bench covering 36 real-world COPs, revealing that current LLM-based agents still lag behind traditional algorithms by approximately 23%.

Overall, LLM-based expert systems are reshaping the optimization paradigm through end-to-end automation and natural-language interaction, dramatically lowering usage thresholds. Closing the performance gap with classical methods and robustly handling complex constraints and cross-domain generalization remain crucial challenges for industrial-grade deployment.

7.4 Comparative Analysis of LLM-Based Optimization Paradigms

This subsection provides a systematic comparison of the three representative LLM-based paradigms for combinatorial optimization—LLM as Optimizer, LLM as Programmer, and LLM as Collaborator—across five key dimensions: solution quality (Qual.), scalability (Scal.), generalization (Gen.), computational efficiency (Eff.), and data independence (Data Inde.), as summarized in Table 9.

Solution Quality: Among all paradigms, LLM as Programmer methods achieve the highest and most stable solution quality. FunSearch discovers novel cap set constructions surpassing known results [60], while EoH and ReEvo generate heuristics that outperform human-designed baselines on TSP and BPP benchmarks [330, 332]. LLM as Optimizer methods, represented by OPRO, achieve moderate quality—GPT-4 attains negligible optimality gaps on very small TSP instances but degrades to moderate gaps as instance size increases [59]. LLM as Collaborator methods, such as NCO+LLM Attention, achieve SOTA performance by enhancing neural solvers, substantially reducing optimality gaps on large TSP instances [340].

Scalability: LLM as Programmer methods demonstrate strong scalability across problem sizes. FunSearch scales to

cap set dimensions up to $n=8$ (search space 3,1600) [60], while EoH and ReEvo handle TSP instances up to 1,000 nodes [330, 332]. LLM as Collaborator methods, particularly those integrated with neural solvers, scale to thousands of nodes, achieving efficient inference on TSP-1000 and CVRP-1000 [340]. LLM as Optimizer methods exhibit the weakest scalability, with performance degrading rapidly beyond 50-node TSP instances due to the increased complexity of solution representation [59].

Generalization: LLM as Programmer methods exhibit superior cross-scale and cross-problem generalization. EoH and ReEvo trained on TSP-50 generalize effectively to TSP-200 and TSP-500, maintaining competitive optimality gaps [330, 332]. MoH further enhances generalization through multi-task training, achieving robust performance across TSP-20 to TSP-1000 [331]. LLM as Collaborator methods demonstrate strong cross-distribution generalization, with NCO+LLM Attention transferring from synthetic to real-world TSPLIB and CVRPLIB instances [340]. LLM as Optimizer methods show limited generalization, with OPRO's performance varying significantly across different problem formulations and scales [59].

Computational Efficiency: LLM as Programmer methods show mixed efficiency. FunSearch requires approximately 1.5 million LLM queries to discover effective heuristics [60], while EoH and ReEvo achieve comparable results with only 1,000–2,000 evaluations through improved search strategies [330, 332]. LLM as Collaborator methods achieve the highest inference efficiency, with NCO+LLM Attention adding negligible overhead (0.2 seconds on TSP-1000) to base neural solvers [340]. LLM as Optimizer methods require multiple LLM calls per optimization step, with OPRO taking up to 195 steps for TSP-20 convergence, making them less suitable for real-time applications [59].

Data Independence: Across all three paradigms, a wide spectrum of data dependence is observed. LLM as Optimizer methods achieve the highest data independence, operating in zero-shot or few-shot settings without any training data or optimal solution labels [59, 326, 327]. LLM as Programmer methods exhibit moderate data dependence, requiring only problem instances and evaluation functions without optimal labels, though some approaches (e.g., FunSearch) require substantial computational budgets for evolutionary search [60, 330]. LLM as Collaborator methods show the highest data dependence, as they typically require pre-trained neural solvers and sometimes additional fine-tuning on diverse instance distributions [339, 340].

Summary: Table 9 consolidates the paradigm-level comparison. LLM as Programmer methods offer the best balance of solution quality, scalability, and generalization, making them suitable for discovering novel heuristics across diverse problem domains. LLM as Collaborator

methods achieve superior computational efficiency and strong generalization when combined with pre-trained neural solvers, making them ideal for real-time deployment scenarios. LLM as Optimizer methods provide the highest data independence and ease of use, serving as lightweight solutions for small-scale problems without training requirements. Methods that fuse LLMs with other techniques show great potential, but currently still lag behind other state-of-the-art methods across various dimensions, and also suffer from issues such as hallucination. Given the limitations of LLM-based approaches, current mainstream research often employs LLMs as components within a larger algorithmic framework to address specific problems.

8 Challenges and Future Directions

Although machine-learning-based methods for combinatorial optimization have achieved remarkable progress—from SL and RL to unsupervised generative models and LLMs—the field still faces core challenges stemming from inherent ML limitations and the stringent requirements of real-world COPs on reliability, efficiency, and adaptability. This chapter systematically examines bottlenecks along eight dimensions and outlines promising future research directions.

8.1 Generalization

Size generalization and distributional generalization remain two pressing core challenges. In terms of size generalization, existing models perform well at training scales but degrade significantly on larger instances. For the constructive algorithm proposed by Kool et al., when trained on TSP20 and evaluated on TSP200, the optimality gap increases sharply from 0.068% to 66.220%, because the solution space grows factorially with node count ($n!$), making cross-scale pattern capture inherently difficult [55]. In terms of distributional generalization, models trained on random datasets deteriorate on real-world distributions (e.g., TSPLIB), with an average optimality gap of 22.659% on real TSP datasets versus 2.359% on random TSP100 [55]. As confirmed in Table 5, end-to-end RL and SL methods generally receive moderate generalization ratings, whereas ML–OR collaborative methods partially mitigate this through structured solver frameworks.

Future research should explore several directions. First, hierarchical divide-and-conquer strategies can be pursued, where high-level policies select critical node subsets and lower-level processes construct local routes [196]. Second, re-decoder architectures [249] and diffusion-based generative models [58] should be investigated for cross-scale generalization. Third, graph sparsification and multi-sparsity

ensembles via attention masks or adjacency-matrix pruning [232] are promising. Fourth, coordinate augmentation (rotations, translations, scaling) with relative encodings can enhance cross-distribution robustness [362]. Additionally, transfer learning and knowledge distillation techniques should also be explored to rapidly adapt pretrained policies to new instances.

8.2 Balancing Computational Efficiency and Solution Quality

End-to-end models (PtrNets, Transformers) offer fast inference but struggle with global optimality under complex constraints [53, 55], while ML-enhanced exact algorithms (e.g., NeuroLKH) improve solution quality at substantial training and inference overhead [189]. Table 5 confirms this trade-off: end-to-end RL methods achieve high efficiency but moderate solution quality, whereas ML-OR exact methods deliver near-optimal quality with limited scalability.

Future research should develop dynamic solution strategies that automatically select paradigms (end-to-end, collaborative, or generative) based on instance characteristics and computational budgets, requiring mechanisms for rapid instance-difficulty assessment. Lightweight architectures—efficient GNN designs with dynamic sparsification [190], linear-attention Transformers, and COP-tailored pruning and quantization—are equally critical. Strong train-once, generalize-many-times models that eliminate repeated retraining across problem sizes constitute a fundamental avenue [249].

8.3 Theoretical Guarantees

Unlike classical algorithms with rigorous mathematical foundations (branch-and-bound for global optimality, approximation algorithms with ratio guarantees) [15, 16], deep-learning-based solvers generally lack interpretability and theoretical guarantees. Their black-box decision logic creates trust barriers for safety-critical applications. Most studies validate performance empirically without analyzing why models work, when they fail, or what theoretical upper bounds exist on optimality gaps.

Future research should formalize algorithmic priors within learning architectures—e.g., neural layers or loss functions respecting submodularity or matroids—to inherit structural approximation guarantees. Explainable AI techniques tailored to COPs (e.g., analyzing edge attention in TSP or job-ordering rationale in SPs) [57] and rigorous generalization-error frameworks for finite-data-to-large-instance transfer are urgently needed. Interactions with algorithmic game theory and online learning theory may

yield new tools for deriving regret bounds and robustness analyses.

8.4 Data Dependency and Labeling Cost

SL approaches and generative models relying on high-quality training signals are heavily constrained by data availability [65]. Optimal-solution labels for large-scale COPs depend on expensive exact solvers (Concorde, Gurobi), leading to high labeling costs and limited distributional coverage [54]. RL reduces label dependence but requires massive environment interactions with unstable training dynamics [56]. Table 5 shows that UL discriminative methods achieve the highest data independence, while SL methods score lowest.

Future research should strengthen truly label-free paradigms such as DiffUCO [299], which trains directly using objective functions. Effective data augmentation exploiting problem symmetries (permutations, rotations), generative synthesis of challenging instances resembling real distributions, few-shot and meta-learning for rapid new-domain adaptation, and transfer learning from related COPs (e.g., TSP to VRP) are all promising directions for reducing data requirements.

8.5 Dynamics and Constraint Scalability

Real-world COPs are often dynamic (real-time task arrivals, stochastic resource variations) and involve multiple interdependent constraints (time windows, capacities, priorities, heterogeneous resources) [118, 126]. Most end-to-end models are trained for static settings with fixed constraint sets and adapt poorly to online, time-varying scenarios. When constraints change, data collection and retraining are typically required [57].

Future research should develop online and continual learning frameworks for rapid policy adjustment from streaming feedback without catastrophic forgetting [126, 178]. Modeling problems as partially observable MDPs combined with recurrent or memory-augmented networks provides an effective approach. For constraint scalability, compositional neural architectures with modular constraint-encoding submodules, differentiable logic layers [177], and neuro-symbolic systems combining deep learning with symbolic constraint reasoning [61] deserve exploration.

8.6 Feasibility Assurance for Generated Solutions

A critical yet underexplored research gap is ensuring hard-constraint feasibility in solutions produced by neural solvers. As Table 5 reveals, end-to-end SL/RL constructive methods, UL generative models (diffusion, VAE, GAN),

and LLM-as-Optimizer approaches all receive low feasibility ratings, because their outputs are sampled from learned distributions without explicit constraint enforcement. Diffusion models such as DIFUSCO [58] and T2TCO [363] generate continuous relaxations that require post-hoc rounding or projection, often violating capacity, precedence, or coupling constraints. Similarly, LLM-as-Optimizer methods (e.g., OPRO [59]) directly generate candidate solutions from prompts with no formal mechanism to ensure constraint satisfaction, and Table 9 confirms their limited feasibility guarantees. In contrast, ML-OR collaborative methods partially inherit solver-side feasibility handling but cannot guarantee it end-to-end when ML modules override solver decisions.

Future research should develop differentiable constraint-satisfaction layers that can be embedded within neural architectures to enforce feasibility during forward passes, extending Lagrangian relaxation or penalty-based training to provide probabilistic feasibility certificates. Constrained decoding mechanisms for autoregressive and diffusion models—analogue to grammar-constrained generation in NLP—offer a promising direction for ensuring that each generation step respects combinatorial constraints. For LLM-based paradigms, integrating formal verification modules that check generated solutions against constraint specifications before output, combined with iterative repair loops, could substantially improve reliability. Research on theoretically grounded feasibility guarantees—such as proving that a learned policy always produces feasible solutions under specified conditions—remains an open and high-impact challenge.

8.7 Non-Stationary Environments and Risk-Aware Optimization

Most existing learning-based methods implicitly assume stationary problem distributions during both training and deployment. However, real-world COPs frequently operate in non-stationary environments where objective functions, constraint parameters, or instance distributions shift over time—for example, fluctuating demand patterns in logistics, evolving network topologies in telecommunications, or changing cost structures in supply chains. As our comparative analysis (Table 5) indicates, even methods with strong generalization on static benchmarks (e.g., RL-based end-to-end and ML-OR collaborative approaches) do not explicitly address distributional drift, and their performance under non-stationarity remains largely uncharacterized.

Furthermore, standard optimization formulations typically target expected-case performance (e.g., minimizing average tour length or makespan), neglecting risk-sensitive objectives that are essential in high-stakes applications. In

domains such as emergency logistics, healthcare scheduling, and financial portfolio routing, decision-makers require not only good average performance but also guarantees on worst-case outcomes, variance control, or conditional value-at-risk (CVaR) minimization [364]. Current ML-for-COP methods lack systematic integration of risk-aware objectives: RL-based approaches optimize expected cumulative reward without tail-risk control, generative models sample from learned distributions without risk shaping, and LLM-based paradigms have no mechanism for risk quantification.

Future research should develop distribution-shift-robust training paradigms, including distributionally robust optimization (DRO) formulations that optimize over worst-case distributions within ambiguity sets [365], and online adaptation mechanisms that detect and respond to distributional drift in real time. For risk-aware optimization, integrating CVaR or other coherent risk measures into RL reward functions and generative model objectives is a critical direction. Multi-objective frameworks that simultaneously optimize expected performance and risk metrics, combined with Pareto-front prediction networks, could enable flexible risk–return trade-offs [84]. Developing risk-aware LLM-based expert systems—where the LLM not only generates solutions but also estimates and communicates associated risks in natural language—represents a novel and impactful research frontier.

8.8 Toward Real-World Applications

Despite significant results on standard benchmarks (TSP, JSSP), deploying ML methods in complex industrial scenarios faces substantial obstacles [81, 82]. Real problems exhibit multiple objectives, large scale, hard constraints, uncertainty, and couplings among subproblems [84]. Additionally, system integration, legacy compatibility, incomplete or noisy data, and strict safety and interpretability requirements contribute to the gap between laboratory and industrial performance [87]. Table 9 further reveals that LLM-based expert systems, despite strong natural-language interaction, still lag behind traditional algorithms by approximately 23% on real-world COPs [361], underscoring the deployment challenge.

Future research must promote benchmark evolution by constructing datasets closer to real scenarios with multiple objectives, uncertainty, dynamics, and heterogeneous constraints [89]. Specialized architectures should be developed: decomposition-based or Pareto-front prediction networks for multi-objective optimization [84]; robust models jointly outputting decisions and risk estimates for stochastic programming; and hierarchical multi-agent cooperative frameworks for large-scale coupled problems [292]. Most critically, interdisciplinary collaboration must be

strengthened—including close work with domain experts for accurate business constraint modeling, development of user-friendly decision-support systems with natural-language interaction [59, 60], and design of solver interfaces seamlessly integrating with enterprise resource planning and manufacturing execution systems [366].

9 Conclusions

COPs, as an important branch of mathematical optimization, are ubiquitous in critical domains such as defense, manufacturing, logistics, communication networks, and decision management. While traditional exact and heuristic algorithms remain highly effective in many well-structured settings, they encounter bottlenecks in terms of computational complexity, parameter sensitivity, and limited generalization when faced with large-scale, real-time, and dynamically changing problems. This survey provides a systematic review of major advances in machine-learning-based methods for COPs during the period 2020–2026, with a unified view of four main learning paradigms (SL, RL, UL, and LLM-based approaches), their integration with classical optimization frameworks, and their instantiations on representative routing and scheduling problems as well as other canonical COPs. In addition, we collect and organize open-source implementations. A key contribution is the establishment of a unified comparative framework that evaluates representative methods across seven dimensions—solution quality, scalability, generalization, computational efficiency, data independence, feasibility assurance, and theoretical guarantees—enabling systematic cross-paradigm comparison. This analysis reveals that end-to-end RL methods achieve high efficiency but moderate solution quality and limited feasibility guarantees; ML–OR collaborative methods deliver the most balanced performance across all dimensions; unsupervised generative models attain the highest data independence but face challenges in constraint feasibility; and LLM-based methods show promising versatility but remain constrained in scalability and solution quality. Grounded in these findings, we synthesize the main challenges and research trends along eight dimensions: generalization, the trade-off between computational efficiency and solution quality, theoretical guarantees, data dependency and labeling cost, dynamics and constraint scalability, feasibility assurance for generated solutions, robustness under non-stationary environments with risk-aware optimization, and pathways toward complex real-world applications.

From the perspective of solution paradigms, the surveyed works can be broadly grouped into three classes. End-to-end learning solvers use deep architectures such as PtrNets,

GNNs, and Transformers to directly map problem instances to solutions, offering fast inference and a high degree of automation, but still struggling with optimality and feasibility on large-scale problems with complex constraints. Collaborative ML–optimization frameworks embed learned components into exact algorithms, heuristics, metaheuristics, or hyper-heuristics to guide branching, operator selection, or parameter tuning, thereby combining data-driven adaptivity with the structural strengths of traditional solvers. Generative solution-distribution models leverage architectures such as diffusion models, VAEs, and GANs to learn distributions over high-quality solutions and generate diverse candidates for subsequent refinement, showing potential for addressing uncertainty and multi-objective requirements. Across these paradigms, LLMs are emerging as a versatile tool: they can serve as optimizers that directly propose solutions, as programmers that automatically generate heuristic code, and as interactive collaborators that support problem modeling, explanation, and solver orchestration. Our comparative analysis further reveals cross-cutting patterns: across all paradigms, feasibility assurance remains a systematic weakness for methods that generate solutions from learned distributions rather than through constraint-enforcing procedures.

Looking ahead, ML-based approaches are expected to play an increasingly important role in combinatorial optimization, but fully realizing this potential requires addressing the above six dimensions in a coordinated way. For generalization, promising directions include hierarchical divide-and-conquer strategies, richer inductive biases and re-decoder architectures, diffusion-based generative models, graph sparsification and invariance-enhancing methods, as well as transfer learning and knowledge distillation. To better balance computational efficiency and solution quality, future work should design dynamic, instance-aware solution pipelines and lightweight GNN/Transformer architectures, together with train-once, generalize-many-times policies. Strengthening theoretical guarantees calls for embedding algorithmic priors and combinatorial structures into learning architectures, and developing explainable analysis tools and more rigorous frameworks for understanding generalization and failure modes. Reducing data dependency and labeling cost hinges on advancing self-supervised and unsupervised paradigms, symmetry-based data augmentation, realistic synthetic instances, and few-shot or meta-learning and transfer-learning frameworks. For dynamics and constraint scalability, online and continual learning, POMDP formulations combined with RNNs or memory-augmented networks, compositional neural architectures with constraint-satisfaction layers, and neuro-symbolic systems are all promising. Finally, moving toward real-world applications requires more realistic benchmarks that capture

multi-objective, uncertain, dynamic, and heterogeneous constraints, specialized architectures for complex industrial problems, and close interdisciplinary collaboration to build decision-support systems with natural-language interaction and explanation that can be seamlessly integrated into existing industrial infrastructures.

Ensuring feasibility of generated solutions is an increasingly urgent challenge as generative models and LLM-based methods gain traction. Future work should develop differentiable constraint-satisfaction layers embeddable within neural architectures, constrained decoding mechanisms for autoregressive and diffusion models that enforce combinatorial constraints at each generation step, and formal verification modules for LLM-generated solutions combined with iterative repair loops. Theoretical research on provable feasibility guarantees—establishing conditions under which a learned policy always produces feasible solutions—remains an open and high-impact problem.

Addressing non-stationary environments and risk-aware optimization is essential for deploying ML-based solvers in real-world settings where problem distributions, objectives, and constraints evolve over time. Distributionally robust optimization formulations, online drift detection and adaptation mechanisms, and the integration of coherent risk measures such as CVaR into RL reward functions and generative model objectives represent critical research frontiers. Developing multi-objective frameworks that simultaneously optimize expected performance and risk metrics, as well as risk-aware LLM-based systems capable of estimating and communicating solution risks, would significantly advance practical deployment.

Acknowledgements This research was supported by the Fundamental Research Funds for Central Universities (Grant No.: DUT25GF207), and partially funded by the China Postdoctoral Science Foundation funded project (Grant No.: 2025M773071).

Author Contributions Conceptualization: Xinwei Wang and Xinyong Yu; Methodology: Xinyong Yu, Zhaopan Wang, Guangyu Wu and Zeying Si; Investigation: Xinyong Yu, Zhaopan Wang, Xin Li, Guangyu Wu and Zeying Si; Project administration: Lei Wang and Xinwei Wang; Supervision: Xichao Su, Haijun Peng, Shaohui Qu and Biao Xiong; Visualization: Xinyong Yu, Zhaopan Wang, Guangyu Wu and Zeying Si; Writing – original draft: Xinwei Wang, Xinyong Yu, Zhaopan Wang, Guangyu Wu and Zeying Si; Writing – review & editing: Xinwei Wang, Xin Li, Haijun Peng, Shaohui Qu, Biao Xiong and Xichao Su; Funding acquisition: Lei Wang, Xinwei Wang and Xin Li.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Ethical Approval This article does not contain any studies with human participants or animals performed by any of the authors.

Competing Interests The authors declare no competing interests.

References

- Papadimitriou CH, Steiglitz K (1998) Combinatorial optimization: algorithms and complexity. Courier Corporation, Mineola, New York
- Festa P (2014) A brief introduction to exact, approximation, and heuristic algorithms for solving hard combinatorial optimization problems. In: International Conference on Transparent Optical Networks, IEEE, Graz, Austria, pp 1–20. <https://doi.org/10.1109/icton.2014.6876285>
- Yu X, Li X, Wang L, Jin J, Zhang G, Su X, Tao L, Lu C, Wang X (2025) Carrier platform-enhanced multiple-uav cooperative task assignment with dual heterogeneities. *Artif Intell Rev* 58(8):248. <https://doi.org/10.1007/s10462-025-11254-2>
- Halim AH, Ismail I (2019) Combinatorial optimization: comparison of heuristic algorithms in travelling salesman problem. *Arch Computat Methods Eng* 26(2):367–380. <https://doi.org/10.1007/s11831-017-9247-y>
- Salehi Sarbijan M, Behnamian J (2023) Emerging research fields in vehicle routing problem: a short review. *Arch Computat Methods Eng* 30(4):2473–2491. <https://doi.org/10.1007/s11831-022-09874-w>
- Ansari ZN, Daxini SD (2022) A state-of-the-art review on meta-heuristics application in remanufacturing. *Arch Computat Methods Eng* 29(1):427–470. <https://doi.org/10.1007/s11831-021-09580-z>
- Xia X (2025) Multi-stage decision-making models based on decision trees and 0–1 variable planning. *Proc Comp Sci* 262:685–694. <https://doi.org/10.1016/j.procs.2025.05.100>
- Wang Y, Zhang N, Zhuo Z, Kang C, Kirschen D (2018) Mixed-integer linear programming-based optimal configuration planning for energy hub: starting from scratch. *Appl Energy* 210:1141–1150. <https://doi.org/10.1016/j.apenergy.2017.08.114>
- Abanto-Leon LF, Maghsudi S (2025) Hierarchical functionality prioritization in multicast isac: optimal admission control and discrete-phase beamforming. *IEEE Commun Lett* 29(1):180–184. <https://doi.org/10.1109/LCOMM.2024.3506696>
- Antoniadis A, Eliáš M, Polak A, Venzin M (2024) Approximation algorithms for combinatorial optimization with predictions. <https://arxiv.org/abs/2411.16600>
- Shen Y, Sun Y, Li X, Eberhard A, Ernst A (2022) Enhancing column generation by a machine-learning-based pricing heuristic for graph coloring. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 36. AAAI, Virtual, pp. 9926–9934. <https://doi.org/10.1609/aaai.v36i9.21230>
- Tayebi D, Ray S, Ajwani D (2022) Learning to prune instances of k-median and related problems. In: Proceedings of the Symposium on Algorithm Engineering and Experiments, SIAM, Philadelphia, PA, pp 184–194. <https://doi.org/10.1137/1.9781611977042.15>
- Dong Z, Cao W, Zhang M, Tao D, Chen Y, Zhang X (2023) Ckt-GNN: circuit graph neural network for electronic design automation. <https://arxiv.org/abs/2308.16406>
- Zhou Y, Jia R, Lin H, Quan H, Zhao Y, Lyu X (2023) Improving graph matching with positional reconstruction encoder-decoder network. In: Advances in Neural Information Processing Systems, vol 36. Curran Associates, Inc, Virtual, pp 34557–34569. https://proceedings.neurips.cc/paper_files/paper/2023/file/6cd3ac24cd789beaa9f7145670fcac-Paper-Conference.pdf
- Lawler EL, Wood DE (1966) Branch-and-bound methods: a survey. *Oper Res* 14(4):699–719. <https://doi.org/10.1287/opre.14.4.699>

16. Wolsey LA, Nemhauser GL (1999) Integer and combinatorial optimization. John Wiley & Sons, New York
17. Bertsekas D (2012) Dynamic programming and optimal control: volume I. vol. 1. Athena scientific, Belmont, Massachusetts
18. Sniedovich M (2010) Dynamic programming: foundations and principles. CRC Press, Boca Raton, Florida
19. Loiola EM, de Abreu NMM, Boaventura-Netto PO, Hahn P, Querido T (2007) A survey for the quadratic assignment problem. *Eur J Oper Res* 176(2):657–690. <https://doi.org/10.1016/j.ejor.2005.09.032>
20. Ho W, Ho GTS, Ji P, Lau HCW (2008) A hybrid genetic algorithm for the multi-depot vehicle routing problem. *Eng Appl Artif Intel* 21(4):548–557. <https://doi.org/10.1016/j.engappai.2007.06.001>
21. Webster ST (1993) A priority rule for minimizing weighted flow time in a class of parallel machine scheduling problems. *Eur J Oper Res* 70(3):327–334. [https://doi.org/10.1016/0377-2217\(93\)90244-h](https://doi.org/10.1016/0377-2217(93)90244-h)
22. Irnich S, Funke B, Grünert T (2006) Sequential search and its application to vehicle-routing problems. *Comput Operations Res* 33(8):2405–2429. <https://doi.org/10.1016/j.cor.2005.02.020>
23. Selladurai V, Aravindan P, Ponnambalam SG, Gunasekaran A (1995) Dynamic simulation of job shop scheduling for optimal performance. *Int J Operations Production Manag* 15(7):106–120. <https://doi.org/10.1108/01443579510090444>
24. He J, Chen X, Chen X (2016) A filter-and-fan approach with adaptive neighborhood switching for resource-constrained project scheduling. *Comput Operations Res* 71:71–81. <https://doi.org/10.1016/j.cor.2016.01.012>
25. Rezoug A, Bader-El-Den M, Boughaci D (2018) Guided genetic algorithm for the multidimensional knapsack problem. *Memet Comp* 10(1):29–42. <https://doi.org/10.1007/s12293-017-0232-7>
26. Lin B, Sun X, Salous S (2016) Solving travelling salesman problem with an improved hybrid genetic algorithm. *JCC* 4(15):98–106. <https://doi.org/10.4236/jcc.2016.415009>
27. Teoh EJ, Tang H, Tan KC (2006) A columnar competitive model with simulated annealing for solving combinatorial optimization problems. In: International Joint Conference on Neural Network Proceedings, IEEE, BC, Canada, pp 3254–3259. <https://doi.org/10.1109/IJCNN.2006.247320>
28. Laarhoven PJM, Aarts EHL, Lenstra JK (1992) Job shop scheduling by simulated annealing. *Oper Res* 40(1):113–125. <https://doi.org/10.1287/opre.40.1.113>
29. Barnes JW, Laguna M (1993) Solving the multiple-machine weighted flow time problem using tabu search. *IIE Trans* 25(2):121–128. <https://doi.org/10.1080/07408179308964284>
30. Basu S (2012) Tabu search implementation on traveling salesman problem and its variations: a literature survey. *AJOR* 2(2):163–173. <https://doi.org/10.4236/ajor.2012.22019>
31. Deng W, Xu J, Zhao H (2019) An improved ant colony optimization algorithm based on hybrid strategies for scheduling problem. *IEEE Access* 7:20281–20292. <https://doi.org/10.1109/ACCESS.2019.2897580>
32. Zhong Y, Lin J, Wang L, Zhang H (2018) Discrete comprehensive learning particle swarm optimization algorithm with metropolis acceptance criterion for traveling salesman problem. *Swarm Evol Comput* 42:77–88. <https://doi.org/10.1016/j.swevo.2018.02.017>
33. Nouri M, Bekrar A, Jemai A, Niar S, Ammari AC (2018) An effective and distributed particle swarm optimization algorithm for flexible job-shop scheduling problem. *J Intell Manuf* 29(3):603–615. <https://doi.org/10.1007/s10845-015-1039-3>
34. Prado RS, Silva RC, Guimarães FG, Neto OM (2010) Using differential evolution for combinatorial optimization: a general approach. In: International Conference on Systems, Man and Cybernetics, IEEE, Istanbul, Turkey, pp. 11–18. <https://doi.org/10.1109/icsmc.2010.5642193>
35. Onwubolu GC, Davendra D (2008) Differential evolution: a Handbook for global permutation-based combinatorial optimization. In: Studies in computational intelligence, vol 175. Springer, Berlin, Heidelberg
36. Sánchez M, Cruz-Duarte JM, Ortiz-Bayliss JC, Ceballos H, Terashima-Marin H, Amaya I (2020) A systematic review of hyper-heuristics on combinatorial optimization problems. *IEEE Access* 8:128068–128095. <https://doi.org/10.1109/ACCESS.2020.3009318>
37. Branke J, Nguyen S, Pickardt CW, Zhang M (2016) Automated design of production scheduling heuristics: a review. *IEEE Trans Evol Comput* 20(1):110–124. <https://doi.org/10.1109/TEVC.2015.2429314>
38. Garcia-Villoria A, Salhi S, Corominas A, Pastor R (2011) Hyper-heuristic approaches for the response time variability problem. *Eur J Oper Res* 211(1):160–169. <https://doi.org/10.1016/j.ejor.2010.12.005>
39. Garrido P, Riff MC (2010) Dvrp: a hard dynamic combinatorial optimisation problem tackled by an evolutionary hyper-heuristic. *J Heuristics* 16(6):795–834. <https://doi.org/10.1007/s10732-010-9126-2>
40. Burke EK, Hyde MR, Kendall G, Woodward J (2012) Automating the packing heuristic design process with genetic programming. *Evol Comput* 20(1):63–89. https://doi.org/10.1162/EVCO_a_00044
41. Tay JC, Ho NB (2008) Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Comput Ind Eng* 54(3):453–473. <https://doi.org/10.1016/j.cie.2007.08.008>
42. Duan J, Liu F, Zhang Q, Qin J, Zhou Y (2024) Genetic programming hyper-heuristic-based solution for dynamic energy-efficient scheduling of hybrid flow shop scheduling with machine breakdowns and random job arrivals. *Expert Syst Appl* 254:124375. <https://doi.org/10.1016/j.eswa.2024.124375>
43. Hopfield JJ, Tank DW (1985) “Neural” computation of decisions in optimization problems. *Biol Cybern* 52(3):141–152. <https://doi.org/10.1007/bf00339943>
44. Kamgar-Parsi B (1990) On problem solving with hopfield neural networks. *Biol Cybern* 62(5):415–423. <https://doi.org/10.1007/bf00197648>
45. Philip Chen CL (1992) Design of a real-time and/or assembly scheduler on an optimization neural network. *J Intell Manuf* 3(4):251–261. <https://doi.org/10.1007/bf01473902>
46. Abe S, Gee AH (1995) Global convergence of the hopfield neural network with nonzero diagonal elements. *IEEE Trans Circuits Syst II* 42(1):39–45. <https://doi.org/10.1109/82.363543>
47. Chen L, Aihara K (1995) Chaotic simulated annealing by a neural network model with transient chaos. *Neural Networks* 8(6):915–930. [https://doi.org/10.1016/0893-6080\(95\)00033-v](https://doi.org/10.1016/0893-6080(95)00033-v)
48. Goodfellow I (2016) Deep learning. MIT press, Cambridge, Massachusetts
49. Krizhevsky A, Sutskever I, Hinton GE (2017) Imagenet classification with deep convolutional neural networks. *Commun ACM* 60(6):84–90. <https://doi.org/10.1145/3065386>
50. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. In: Advances in neural information processing systems, vol 30. Curran Associates, Inc, New Orleans, pp 5998–6008. <https://doi.org/10.1201/9781003561460-19>
51. Silver D, Schrittwieser J, Simonyan K, Antonoglou I, Huang A, Guez A, Hubert T, Baker L, Lai M, Bolton A, Chen Y, Lillicrap T, Hui F, Sifre L, van den Driessche G, Graepel T, Hassabis D (2017) Mastering the game of go without human knowledge. *Nature* 550(7676):354–359. <https://doi.org/10.1038/nature24270>
52. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S,

- Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler D, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D (2020) Language models are few-shot learners. In: *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc, Virtual, pp. 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
53. Vinyals O, Fortunato M, Jaitly N (2015) Pointer networks. In: *Proceedings of the 29th International Conference on Neural Information Processing Systems*, vol 2. MIT Press, Montreal, Canada, pp 2692–2700. <https://doi.org/10.5555/2969442.2969540>
 54. Joshi CK, Laurent T, Bresson X (2019) An efficient graph convolutional network technique for the travelling salesman problem. <https://arxiv.org/abs/1906.01227>
 55. Kool W, Hoof H, Welling M (2019) Attention, learn to solve routing problems! In: *International Conference on Learning Representations*, New Orleans. <https://openreview.net/forum?id=ByxBFsRqYm>
 56. Bello I, Pham H, Le QV, Norouzi M, Bengio S (2016) Neural combinatorial optimization with reinforcement learning. <https://arxiv.org/abs/1611.09940>
 57. Wu Y, Song W, Cao Z, Zhang J, Lim A (2022) Learning improvement heuristics for solving routing problems. *IEEE Trans Neural Netw Learn Syst* 33(9):5057–5069. <https://doi.org/10.1109/TNNLS.2021.3068828>
 58. Sun Z, Yang Y (2023) Difusco: graph-based diffusion solvers for combinatorial optimization. In: *Advances in Neural Information Processing Systems*, vol 36. Curran Associates, Inc, New Orleans, pp 3706–3731. https://proceedings.neurips.cc/paper_files/paper/2023/file/0ba520d93c3df592c83a611961314c98-Paper-Conference.pdf
 59. Yang C, Wang X, Lu Y, Liu H, Le QV, Zhou D, Chen X (2024) Large language models as optimizers. In: *International Conference on Learning Representations*. OpenReview, net, Vienna, Austria. <https://openreview.net/forum?id=Bb4VGOWELI>
 60. Romera-Paredes B, Barekatin M, Novikov A, Balog M, Kumar MP, Dupont E, Ruiz FJR, Ellenberg JS, Wang P, Fawzi O, Kohli P, Fawzi A (2024) Mathematical discoveries from program search with large language models. *Nature* 625(7995):468–475. <https://doi.org/10.1038/s41586-023-06924-6>
 61. AhmadiTeshnizi A, Gao W, Udell M (2024) Optimus: scalable optimization modeling with milp solvers and large language models. In: *International Conference on Machine Learning*. OpenReview.net, Vienna, Austria. <https://openreview.net/forum?id=YT1dtdLvSN>
 62. Bengio Y, Lodi A, Prouvost A (2021) Machine learning for combinatorial optimization: a methodological tour d’horizon. *Eur J Oper Res* 290(2):405–421. <https://doi.org/10.1016/j.ejor.2020.07.063>
 63. Talbi E-G (2022) Machine learning into metaheuristics: a survey and taxonomy. *ACM Comput Surv* 54(6):1–32. <https://doi.org/10.1145/3459664>
 64. Li K, Zhang T, Wang R, Qin W, He H, Huang H (2021) Research reviews of combinatorial optimization methods based on deep reinforcement learning. *Acta Autom Sin* 47(11):2521–2537. <https://doi.org/10.1145/3459664>
 65. Karimi-Mamaghan M, Mohammadi M, Meyer P, Karimi-Mamaghan AM, Talbi E-G (2022) Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: a state-of-the-art. *Eur J Oper Res* 296(2):393–422. <https://doi.org/10.1016/j.ejor.2021.04.032>
 66. Li B, Wu G, He Y, Fan M, Pedrycz W (2022) An overview and experimental study of learning-based optimization algorithms for the vehicle routing problem. *IEEE/CAA J Autom Sin* 9(7):1115–1138. <https://doi.org/10.1109/JAS.2022.105677>
 67. Mazyavkina N, Sviridov S, Ivanov S, Burnaev E (2021) Reinforcement learning for combinatorial optimization: a survey. *Comput Operations Res* 134:105400. <https://doi.org/10.1016/j.cor.2021.105400>
 68. Qin ZT, Zhu H, Ye J (2022) Reinforcement learning for ridesharing: an extended survey. *Transp Research Part C: Emerging Technol* 144:103852. <https://doi.org/10.1016/j.trc.2022.103852>
 69. Costa Oliveira AL, Britto A, Gusmão R (2023) Machine learning enhancing metaheuristics: a systematic review. *Soft Comput* 27(21):15971–15998. <https://doi.org/10.1007/s00500-023-08886-3>
 70. Zhang J, Liu C, Li X, Zhen H-L, Yuan M, Li Y, Yan J (2023) A survey for solving mixed integer programming via machine learning. *Neurocomputing* 519:205–217. <https://doi.org/10.1016/j.neucom.2022.11.024>
 71. Zhang C, Wu Y, Ma Y, Song W, Le Z, Cao Z, Zhang J (2023) A review on learning to solve combinatorial optimisation problems in manufacturing. *IET Collab Intel Manufact* 5(1):12072. <https://doi.org/10.1049/cim2.12072>
 72. Cappart Q, Chételat D, Khalil EB, Lodi A, Morris C, Veličković P (2023) Combinatorial optimization and reasoning with graph neural networks. *J Mach Learn Res* 24(130):1–61
 73. Szénási S, Légrádi G (2024) Machine learning aided metaheuristics: a comprehensive review of hybrid local search methods. *Expert Syst Appl* 258:125192. <https://doi.org/10.1016/j.eswa.2024.125192>
 74. Zhang C, Juraschek M, Herrmann C (2024) Deep reinforcement learning-based dynamic scheduling for resilient and sustainable manufacturing: a systematic review. *J Manuf Sys* 77:962–989. <https://doi.org/10.1016/j.jmsy.2024.10.026>
 75. Yang Y, Gao Y, Ding Z, Wu J, Zhang S, Han F, Qiu X, Gao S, Wang Y-G (2024) Advancements in q-learning meta-heuristic optimization algorithms: a survey. *WIREs Data Min Knowl* 14(6):1548. <https://doi.org/10.1002/widm.1548>
 76. Modrak V, Sudhakarapandian R, Balamurugan A, Soltysova Z (2024) A review on reinforcement learning in production scheduling: an inferential perspective. *Algorithms* 17(8):343. <https://doi.org/10.3390/a17080343>
 77. Jalali Khalil Abadi Z, Mansouri N, Javidi MM (2024) Deep reinforcement learning-based scheduling in distributed systems: a critical review. *Knowl Inf Syst* 66(10):5709–5782. <https://doi.org/10.1007/s10115-024-02167-7>
 78. Erdem E, ErKayman B, Aydin T (2026) Advances in ai-based solutions for the multi-depot vehicle routing problem: a review of recent trends and future directions. *Arch Computat Methods Eng* 33(2):1997–2035. <https://doi.org/10.1007/s11831-025-10356-y>
 79. Fu Y, Wang Y, Gao K, Huang M (2024) Review on ensemble meta-heuristics and reinforcement learning for manufacturing scheduling problems. *Comput Electr Eng* 120:109780. <https://doi.org/10.1016/j.compeleceng.2024.109780>
 80. Smit IG, Zhou J, Reijnen R, Wu Y, Chen J, Zhang C, Bukhsh Z, Zhang Y, Nuijten W (2025) Graph neural networks for job shop scheduling problems: a survey. *Comput Operations Res* 176:106914. <https://doi.org/10.1016/j.cor.2024.106914>
 81. Lv L, Zhang C, Fan J, Shen W (2025) Deep reinforcement learning for job shop scheduling problems: a comprehensive literature review. *Knowl-Based Syst* 321:113633. <https://doi.org/10.1016/j.knosys.2025.113633>
 82. Martins MSE, Sousa JMC, Vieira S (2025) A systematic review on reinforcement learning for industrial combinatorial optimization problems. *Appl Sci* 15(3):1211. <https://doi.org/10.3390/app15031211>
 83. Chung K, Lee C, Tsang Y (2025) Neural combinatorial optimization with reinforcement learning in industrial engineering: a survey. *Artif Intell Rev* 58(5):130. <https://doi.org/10.1007/s10462-024-11045-1>

84. Arishi A, Ahuja P (2025) Multi-agent reinforcement learning for truck–drone routing in smart logistics: a comprehensive review. *Comput Electr Eng* 127:110529. <https://doi.org/10.1016/j.compel.2025.110529>
85. Sui J, Ding S, Huang X, Yu Y, Liu R, Xia B, Ding Z, Xu L, Zhang H, Yu C, Bu D (2025) A survey on deep learning-based algorithms for the traveling salesman problem. *Front Comput Sci* 19(6):196322. <https://doi.org/10.1007/s11704-024-40490-y>
86. Wu X, Wu S-H, Wu J, Feng L, Tan KC (2025) Evolutionary computation in the era of large language model: survey and roadmap. *IEEE Trans Evol Comput* 29(2):534–554. <https://doi.org/10.1109/TEVC.2024.3506731>
87. Zhang R, Wang J, Liu C, Su K, Ishibuchi H, Jin Y (2025) Synergistic integration of metaheuristics and machine learning: latest advances and emerging trends. *Artif Intell Rev* 58(9):268. <https://doi.org/10.1007/s10462-025-11266-y>
88. Da Ros F, Soprano M, Di Gasparo L, Roitero K (2025) Large language models for combinatorial optimization: a systematic review. <https://arxiv.org/abs/2507.03637>
89. Tao P, Chen L (2025) Combinatorial optimization: from deep learning to large language models. *Sci China Math* 68(10):2519–2537. <https://doi.org/10.1007/s11425-023-2364-2>
90. Wang X, Yu X, Wang Z, Si Z, Wu G (2025) Machine learning for combinatorial optimization: survey resources. GitHub
91. Bellman R (1958) On a routing problem. *Quart Appl Math* 16(1):87–90. <https://doi.org/10.1090/qam/102435>
92. Dijkstra EW (1959) A note on two problems in connexion with graphs. *Numer Math* 1(1):269–271. <https://doi.org/10.1007/BF01386390>
93. Ford LR Jr., Fulkerson DR: (1956) Maximal flow through a network. *Can J Math* 8:399–404. <https://doi.org/10.4153/CJM-1956-045-5>
94. Ford LR Jr, Fulkerson DR (1962) *Flows in networks*. RAND Corporation Research Studies. Princeton University Press, Princeton, NJ
95. Sampson JR (1976) Adaptation in natural and artificial systems (John H. Holland). *SIAM Rev* 18(3):529–530. <https://doi.org/10.1137/1018105>
96. Storn R, Price K (1997) Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *J Global Optim* 11(4):341–359. <https://doi.org/10.1023/A:1008202821328>
97. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol 4. pp 1942–19484. <https://doi.org/10.1109/ICNN.1995.488968>
98. Mirjalili S, Lewis A (2016) The whale optimization algorithm. *Adv Eng Softw* 95:51–67. <https://doi.org/10.1016/j.advengsoft.2016.01.008>
99. Kirkpatrick S, Gelatt C, Vecchi M (1983) Optimization by simulated annealing. *Sci (New Y, N.Y.)* 220(4598):671–680. <https://doi.org/10.1126/science.220.4598.671>
100. Rao RV, Savsani VJ, Vakharia DP (2011) Teaching–learning-based optimization: a novel method for constrained Mechanical design optimization problems. *Comput-Aided Des* 43(3):303–315. <https://doi.org/10.1016/j.cad.2010.12.015>
101. Burke EK, Gendreau M, Hyde M, Kendall G, Ochoa G, Ozcan E, Qu R (2013) Hyper-heuristics: a survey of the state of the art. *J Oper Res Soc* 64(12):1695–1724. <https://doi.org/10.1057/jors.2013.71>
102. Burke EK, Kendall G, Newall J, Hart E, Ross P, Schulenburg S (2003) Hyper-heuristics: an emerging direction in modern search technology. In: Glover F, Kochenberger GA (eds) *Handbook of Metaheuristics*. International series in operations research and management science, vol 57. Springer, Boston, MA, pp 457–474. https://doi.org/10.1007/0-306-48056-5_16
103. Burke EK, Hyde M, Kendall G, Ochoa G, Zcan E, Woodward JR (2010) A classification of hyper-heuristic approaches. In: Gendreau M, Potvin J-Y (eds) *Handbook of Metaheuristics*. International series in operations research and management science, vol 146. Springer, New York, NY, pp 449–468. <https://doi.org/10.1007/978-1-4419-1665-515>
104. Ochoa G, Hyde M, Curtois T, Vázquez-Rodríguez JA, Walker J, Gendreau M, Kendall G, McCollum B, Parkes AJ, Petrovic S, Burke EK (2012) Hyflex: a benchmark framework for cross-domain heuristic search. In: Hao J-K, Middendorf M (eds) *Evolutionary computation in combinatorial optimization (EvoCOP 2012)*. Lecture notes in computer science, vol 7245. Springer, Berlin, Heidelberg, pp 136–147. <https://doi.org/10.1007/978-3-642-29124-112>
105. Gutekunst SC, Williamson DP (2023) The circlet inequalities: a New, circulant-based, facet-defining inequality for the TSP. *Math Or* 48(1):393–418. <https://doi.org/10.1287/moor.2022.1265>
106. He X, Pan Q, Gao L, Neufeld JS (2023) An asymmetric traveling salesman problem based matheuristic algorithm for flow-shop group scheduling problem. *Eur J Oper Res* 310(2):597–610. <https://doi.org/10.1016/j.ejor.2023.03.038>
107. Ye M, Bartolini E, Schneider M (2024) A general variable neighborhood search for the traveling salesman problem with time windows under various objectives. *Discrete Appl Math* 346:95–114. <https://doi.org/10.1016/j.dam.2023.12.006>
108. Lim A, Wang F, Xu Z (2005) The capacitated traveling salesman problem with pickups and deliveries on a tree. In: *Proceedings of the 16th International Conference on Algorithms and Computation*, Springer, Berlin, Heidelberg, pp 1061–1070. https://doi.org/10.1007/11602613_105
109. Dhanalakshmi R, Parthiban P, Anbuezhian N (2022) Optimisation of multiple travelling salesman problem using metaheuristic methods. *IJENM* 13(3):199–215. <https://doi.org/10.1504/IJENM.2022.125803>
110. Chen D, Imdahl C, Lai D, Woensel TV (2025) The dynamic traveling salesman problem with time-dependent and stochastic travel times: a deep reinforcement learning approach. *Transp Research Part C: Emerging Technol* 172:105022. <https://doi.org/10.1016/j.trc.2025.105022>
111. Zhang Z, Liu H, Zhou M, Wang J (2023) Solving dynamic traveling salesman problems with deep reinforcement learning. *IEEE Trans Neural Netw Learn Syst* 34(4):2119–2132. <https://doi.org/10.1109/TNNLS.2021.3105905>
112. Yeh W-C (2024) Time-reliability optimization for the stochastic traveling salesman problem. *Reliab Eng Syst Saf* 248:110179. <https://doi.org/10.1016/j.ress.2024.110179>
113. Climaco G, Simonetti L, Rosseti I (2021) A branch-and-cut and mip-based heuristics for the prize-collecting travelling salesman problem. *RAIRO-Oper Res* 55:S719–S726. <https://doi.org/10.1051/ro/2020002>
114. Santini A (2019) An adaptive large neighbourhood search algorithm for the orienteering problem. *Expert Syst Appl* 123:154–167. <https://doi.org/10.1016/j.eswa.2018.12.050>
115. Gendreau M, Laporte G, Vigo D (1999) Heuristics for the traveling salesman problem with pickup and delivery. *Comput Operations Res* 26(7):699–714. [https://doi.org/10.1016/S0305-0548\(98\)00085-9](https://doi.org/10.1016/S0305-0548(98)00085-9)
116. Pereira AH, Mateus GR, Urrutia SA (2022) Valid inequalities and branch-and-cut algorithm for the pickup and delivery traveling salesman problem with multiple stacks. *Eur J Oper Res* 300(1):207–220. <https://doi.org/10.1016/j.ejor.2021.07.051>
117. Bhat A, Gutow G, Vundurthy B, Ren Z, Rathinam S, Choset H (2024) A complete algorithm for a moving target traveling salesman problem with Obstacles. <https://arxiv.org/abs/2409.09852>
118. Zong Z, Tong X, Zheng M, Li Y (2024) Reinforcement learning for solving multiple vehicle routing problem with time window.

- ACM Trans Intell Syst Technol 15(2):1–19. <https://doi.org/10.1145/3625232>
119. Kool W, Hoof H, Gromicho J, Welling M (2022) Deep policy dynamic programming for vehicle routing problems. In: Integration of constraint programming, artificial intelligence, and operations research, vol 13292. Springer, Cham, pp 190–213. https://doi.org/10.1007/978-3-031-08011-1_14
 120. Habib Y, Filchenkov A (2022) Multi-agent reinforcement learning for multi vehicles one-commodity vehicle routing problem. Proc Comp Sci 212:418–428. <https://doi.org/10.1016/j.procs.2022.11.026>
 121. Bortfeldt A, Yi J (2020) The split delivery vehicle routing problem with three-dimensional loading constraints. Eur J Oper Res 282(2):545–558. <https://doi.org/10.1016/j.ejor.2019.09.024>
 122. Smit IG, Zhou J, Reijnen R, Wu Y, Chen J, Zhang C, Bukhsh Z, Zhang Y, Nuijten WPM (2025) Graph neural networks for job shop scheduling problems: a survey. Comput Operations Res 176:106914. <https://doi.org/10.1016/j.cor.2024.106914>
 123. Yang S, Guo H, Huang J, Han K (2024) A deep reinforcement learning based approach for dynamic job shop scheduling considering variable processing time. In: Proceedings of the 2024 4th International Conference on Artificial Intelligence, Automation and High Performance Computing, Association for Computing Machinery, Zhuhai, China, pp 368–374. <https://doi.org/10.1145/3690931.3690993>
 124. Han B, Yang JJ (2021) A deep reinforcement learning based solution for flexible job shop scheduling problem. Int J Simul Model 20(2):375–386. <https://doi.org/10.2507/IJSIMM20-2-CO7>
 125. Gu Y, Chen M, Wang L (2023) A self-learning discrete salp swarm algorithm based on deep reinforcement learning for dynamic job shop scheduling problem. Appl Intell 53(15):18925–18958. <https://doi.org/10.1007/s10489-023-04479-7>
 126. Zhang L, Feng Y, Xiao Q, Xu Y, Li D, Yang D, Yang Z (2023) Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times. J Manuf Sys 71:257–273. <https://doi.org/10.1016/j.jmsy.2023.09.009>
 127. Zhang Z, Fu Y, Gao K, Pan Q, Huang M (2024) A learning-driven multi-objective cooperative artificial bee colony algorithm for distributed flexible job shop scheduling problems with preventive maintenance and transportation operations. Comput Ind Eng 196:110484. <https://doi.org/10.1016/j.cie.2024.110484>
 128. Xu J, Yu Y, Song X (2025) An improved genetic algorithm based on q-learning for solving the hybrid flow shop scheduling problem. In: 2025 37th Chinese Control and Decision Conference (CCDC), IEEE, Xiamen, China, pp 2282–2287. <https://doi.org/10.1109/CCDC65474.2025.11090763>
 129. Deng Z, Liu X, Dou Y, Su X, Li H, Wang L, Wang X (2025) Autonomous sortie scheduling for carrier aircraft fleet under towing mode. Def Technol 43:1–12. <https://doi.org/10.1016/j.dt.2024.07.011>
 130. Ye H, Wang J, Cao Z, Liang H, Li Y (2023) Deepaco: neural-enhanced ant systems for combinatorial optimization. In: Advances in Neural Information Processing Systems, vol 36. Curran Associates, Inc, New Orleans, pp 43706–43728. https://proceedings.neurips.cc/paper_files/paper/2023/file/883105b282fe15275991b411e6b200c5-Paper-Conference.pdf
 131. Swezey R, Grover A, Charron B, Ermon S (2021) Pirank: scalable learning to rank via differentiable sorting. In: Advances in Neural Information Processing Systems, vol. 34. Curran Associates, Inc, Virtual, pp. 21644–21654. https://proceedings.neurips.cc/paper_files/paper/2021/file/b5200c6107fc3d41d19a2b66835c3974-Paper.pdf
 132. Liu C, Jiang Z, Wang R, Yan J, Huang L, Lu P (2020) Revocable deep reinforcement learning with affinity regularization for outlier-robust graph matching. <https://arxiv.org/abs/2012.08950>
 133. Sahoo SS, Paulus A, Vlastelica M, Musil V, Kuleshov V, Martius G (2022). Backpropagation through combinatorial algorithms: identity with projection works. arXiv preprint arXiv:2205.15213
 134. Nguyen DM, Nguyen H, Diep N, Pham TN, Cao T, Nguyen B, Swoboda P, Ho N, Albarqouni S, Xie P et al. (2023) Lvm-med: learning large-scale self-supervised vision models for medical imaging via second-order graph matching. In: Advances in Neural Information Processing Systems, vol 36. Curran Associates, Inc, New Orleans, pp 27922–27950. https://proceedings.neurips.cc/paper_files/paper/2023/file/58cc11cda2a2679e8af5c6317aed0af8-Paper-Conference.pdf
 135. Mandi J, Bucarey V, Tchomba MMK, Guns T (2022) Decision-focused learning: through the lens of learning to rank. In: International Conference on Machine Learning, PMLR, Baltimore, USA, pp 14935–14947. <https://proceedings.mlr.press/v162/mandi22a.html>
 136. Ma H, Lu W, Feng J (2024) Efficient combinatorial optimization via heat diffusion. In: Advances in Neural Information Processing Systems, vol 37. Curran Associates, Inc, Vancouver, Canada, pp 112104–112133. https://proceedings.neurips.cc/paper_files/paper/2024/file/cb2b9198daacf5f434372122671450ef-Paper-Conference.pdf
 137. Heydaribeni N, Zhan X, Zhang R, Eliassi-Rad T, Koushanfar F (2024) Distributed constrained combinatorial optimization leveraging hypergraph neural networks. Nat Mach Intell 6(6):664–672. <https://doi.org/10.1038/s42256-024-00833-7>
 138. Brusca L, Quaedvlieg LC, Skoulakis S, Chrysos G, Cevher V (2023) Maximum independent set: self-training through dynamic programming. In: Advances in Neural Information Processing Systems, vol 36. Curran Associates, Inc, New Orleans, pp 40637–40658. https://proceedings.neurips.cc/paper_files/paper/2023/file/7fe3170d88a8310ca86df2843f54236c-Paper-Conference.pdf
 139. Sanokowski S, Hochreiter S, Lehner S (2024) A diffusion model framework for unsupervised neural combinatorial optimization. <https://arxiv.org/abs/2406.01661>
 140. Wang R, Shen L, Chen Y, Yang X, Tao D, Yan J (2023) Towards one-shot neural combinatorial solvers: theoretical and empirical notes on the cardinality-constrained case. In: International Conference on Learning Representations. OpenReview, net, Kigali, Rwanda. <https://openreview.net/forum?id=h21yJhdzbwz>
 141. Zheng Z, Zhou C, Tong X, Yuan M, Wang Z (2024) Udc: a unified neural divide-and-conquer framework for large-scale combinatorial optimization problems. In: Advances in Neural Information Processing Systems, vol 37. Curran Associates, Inc, Vancouver, Canada, pp 6081–6125. https://proceedings.neurips.cc/paper_files/paper/2024/file/0b8e4c8468273ee3bafb288229c0acbc-Paper-Conference.pdf
 142. Xia Y, Zhang X (2024) A neural column generation approach to the vehicle routing problem with two-dimensional loading and last-in-first-out constraints. <https://arxiv.org/abs/2406.12454>
 143. Kim W, Iyengar G, Zeevi A (2023) Improved algorithms for multi-period multi-class packing problems with bandit feedback. In: International Conference on Artificial Intelligence and Statistics, PMLR, Valencia, Spain, pp 16458–16501. <https://proceedings.mlr.press/v202/kim23e.html>
 144. Sun Y, Ernst AT, Li X, Weiner J (2023) Learning to generate columns with application to vertex coloring. In: International Conference on Learning Representations. OpenReview, net, Virtual. <https://openreview.net/forum?id=JHW30A4DXtO>
 145. Ling C, Jiang J, Wang J, Thai MT, Xue R, Song J, Qiu M, Zhao L (2023) Deep graph representation learning and optimization for influence maximization. In: International Conference on Artificial Intelligence and Statistics, PMLR, Valencia, Spain, pp 21350–21361. <https://proceedings.mlr.press/v202/ling23b.html>

146. Cheng Y, Yang R, Xiao T, Li Z, Suo J, He K, Dai Q (2023) Cuts: neural causal discovery from irregular time-series data. <https://arxiv.org/abs/2302.07458>
147. Ferber AM, Huang T, Zha D, Schubert M, Steiner B, Dilkina B, Tian Y (2023) Surco: learning linear surrogates for combinatorial nonlinear optimization problems. In: International Conference on Artificial Intelligence and Statistics, PMLR, Valencia, Spain, pp 10034–10052. <https://proceedings.mlr.press/v202/ferber23a.html>
148. Geng H, Wang R, Wu F, Yan J (2023) Gal-vne: solving the vne problem with global reinforcement learning and local one-shot neural prediction. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, PMLR, NY, USA, pp 531–543. <https://doi.org/10.1145/3580305.3599358>
149. Guan J, Zhou X, Yang Y, Bao Y, Peng J, Ma J, Liu Q, Wang L, Gu Q (2024) DecompDiff: diffusion models with decomposed priors for structure-based drug design. <https://arxiv.org/abs/2403.07902>
150. Hang Z, Liu Q (2023) Learning subpocket prototypes for generalizable structure-based drug design. In: International Conference on Artificial Intelligence and Statistics, PMLR, Valencia, Spain, pp 41382–41398. <https://proceedings.mlr.press/v202/zhang23z.html>
151. Bengio Y, Courville A, Vincent P (2013) Representation learning: a review and new perspectives. *IEEE Trans Pattern Anal Mach Intell* 35(8):1798–1828. <https://doi.org/10.1109/TPAMI.2013.50>
152. Ho J, Jain A, Abbeel P (2020) Denoising diffusion probabilistic models. In: Advances in Neural Information Processing Systems, Curran Associates, Inc, Virtual-only. <https://proceedings.neurips.cc/paper/2020/hash/4c5bfcfec8584af0d967f1ab10179ca4b-Abstr.html>
153. Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial nets. In: Advances in Neural Information Processing Systems. Curran Associates, Inc, Montréal, Canada, pp 2672–2680. https://proceedings.neurips.cc/paper_files/paper/2014/file/f033ed80deb0234979a61f95710dbe25-Paper.pdf
154. Kingma DP, Welling M (2014) Auto-encoding variational bayes. In International Conference on Learning Representations, OpenReview.net, Banff, Canada. <https://arxiv.org/abs/1312.6114>
155. See A, Liu PJ, Manning CD (2017) Get to the point: summarization with pointer-generator networks. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, vol 1. Association for Computational Linguistics, Vancouver, Canada, pp 1073–1083. <https://doi.org/10.18653/v1/P17-1099>
156. Yang T, Wang Y, Yue Z, Yang Y, Tong Y, Bai J (2022) Graph pointer neural networks. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 36. AAAI, Virtual, pp. 8832–8839. <https://doi.org/10.1609/aaai.v36i8.20864>
157. Choromanski KM, Likhoshesterov V, Dohan D, Song X, Gane A, Sarlos T, Hawkins P, Davis JQ, Mohiuddin A, Kaiser L, Belanger DB, Colwell LJ, Weller A (2021) Rethinking attention with performers. In: International Conference on Learning Representations, OpenReview.net, Vienna, Austria. <https://openreview.net/forum?id=Ua6zuk0WRH>
158. Bahdanau D, Cho K, Bengio Y (2015) Neural machine translation by jointly learning to align and translate. In: International Conference on Learning Representations, San Diego, CA, USA. <https://doi.org/10.48550/arXiv.1409.0473>
159. Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations, OpenReview.net, Toulon, France. <https://openreview.net/forum?id=SJU4ayYgl>
160. Xu K, Hu W, Leskovec J, Jegelka S (2019) How powerful are graph neural networks? In: International Conference on Learning Representations, OpenReview.net, New Orleans, LA, USA. <https://openreview.net/forum?id=ryGs6iA5Km>
161. Velickovic P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y (2018) Graph attention networks. In: International Conference on Learning Representations, OpenReview.net, Vancouver, BC, Canada. <https://openreview.net/forum?id=rJXMpikCZ>
162. Kaplan J, McCandlish S, Henighan T, Brown TB, Chess B, Child R, Gray S, Radford A, Wu J, Amodei D (2020) Scaling laws for neural language models. <https://arxiv.org/abs/2001.08361>
163. Hoffmann J, Borgeaud S, Mensch A, Buchatskaya E, Cai T, Rutherford E, Las Casas D, Hendricks LA, Welbl J, Clark A, Hennigan T, Noland E, Millican K, Driessche G, Damoc B, Guy A, Osindero S, Simonyan K, Elsen E, Rae JW, Vinyals O, Sifre L (2022) Training compute-optimal large language models. In: Advances in Neural Information Processing Systems, Curran Associates, Inc, New Orleans, LA. https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f144be04a3e5-Paper-Conference.pdf
164. Gu S, Hao T (2018) A pointer network based deep learning algorithm for 0–1 knapsack problem. In: 2018 Tenth International Conference on Advanced Computational Intelligence, IEEE, Xiamen, China, pp 473–477. <https://doi.org/10.1109/ICACI.2018.8377505>
165. Gu S, Yao H (2020) A pointer network based deep learning algorithm for maximum clique problem. In: 2020 10th International Conference on Information Science and Technology, IEEE, Bath, London, and Plymouth, UK, pp 229–233. <https://doi.org/10.1109/ICIST49303.2020.9202001>
166. Gu S, Hao T, Yao H (2020) A pointer network based deep learning algorithm for unconstrained binary quadratic programming problem. *Neurocomputing* 390:1–11. <https://doi.org/10.1016/j.neucom.2019.06.111>
167. Chen S, Wang X (2024) Ptrtasking: pointer network based task scheduling for multi-connectivity enabled mec services. *IEEE Trans Mob Comput* 23(10):9398–9409. <https://doi.org/10.1109/TMC.2024.3363662>
168. Zheng P, Zuo L, Wang J, Zhang J (2018) Pointer networks for solving the permutation flow shop scheduling problem. In: CIE48 Proceedings, The University of Auckland, Auckland, New Zealand, pp 1–8
169. Wei W, Zhang X, Yang J, Zheng Y (2025) A dual-layer hypergraph model integrating urban context and courier profiles for route prediction. *Transportmetrica A: Transp Sci* 2494111. <https://doi.org/10.1080/23249935.2025.2494111>
170. Yonkeu A, Amini M, Kantarci B (2025) Fast and adaptive task management in mec: a deep learning approach using pointer networks. In: International Conference on Network of the Future, IEEE, Montreal, Canada, pp 126–133. <https://doi.org/10.1109/NoF66640.2025.11223320>
171. Chen T, Zhang X, You M, Zheng G, Lambotaran S (2022) A gnn-based supervised learning framework for resource allocation in wireless iot networks. *IEEE Internet Things J* 9(3):1712–1724. <https://doi.org/10.1109/JIOT.2021.3091551>
172. Liu S, Cao Z, Yin N, Ong Y-S (2026) Scale-net: a hierarchical u-net framework for cross-scale generalization in multi-task vehicle routing. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 40. pp 14295–14303. <https://doi.org/10.1609/aaai.v40i17.38444>
173. Lee M, Yu G, Li GY (2021) Graph embedding-based wireless link scheduling with few training samples. *IEEE Trans Wireless Commun* 20(4):2282–2294. <https://doi.org/10.1109/TWC.2020.3040983>
174. Fomin D, Makarov I, Voronina M, Strimovskaya A, Pozdnyakov V (2024) Heterogeneous graph attention networks for scheduling in cloud manufacturing and logistics. *IEEE Access* 12:196195–196206. <https://doi.org/10.1109/ACCESS.2024.3522020>
175. Olmos L, Kaplan A, Jalalitar M (2025) Enhancing course planning with bert-infused gat embeddings and integer linear

- programming. In: 2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC), IEEE, Las Vegas, Nevada, USA, pp 769–775. <https://doi.org/10.1109/CCWC62904.2025.10903901>
176. Bi J, Ma Y, Wang J, Cao Z, Chen J, Sun Y, Chee YM (2022) Learning generalizable models for vehicle routing problems via knowledge distillation. In: Advances in neural information processing systems. Curran associates. Inc, New Orleans. <https://openreview.net/forum?id=sOVNpUEgKMp>
177. Wang T, Payberah AH, Vlassov V (2023) Graph representation learning with graph transformers in neural combinatorial optimization. In: 2023 International Conference on Machine Learning and Applications (ICMLA), IEEE, Jacksonville, FL, USA, pp 488–495. <https://doi.org/10.1109/ICMLA58977.2023.00074>
178. Chen S, Huang Z, Guo H (2022) An end-to-end deep learning method for dynamic job shop scheduling problem. *Machines* 10(7):573. <https://doi.org/10.3390/machines10070573>
179. Thyssens D, Falkner J, Schmidt-Thieme L (2022) Supervised permutation Invariant networks for solving the CVRP with bounded fleet size. <https://openreview.net/forum?id=415iO9eoh3f>
180. Yang S, Lee G, Huang L (2022) Deep learning-based dynamic computation task offloading for mobile edge computing networks. *Sensors* 22(11):4088. <https://doi.org/10.3390/s22114088>
181. Khalil E, Bodic PL, Song L, Nemhauser G, Dilkina B (2016) Learning to branch in mixed integer programming. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 30. AAAI, Phoenix, Arizona USA. <https://doi.org/10.1609/aaai.v30i1.10080>
182. Gasse M, Chételat D, Ferroni N, Charlin L, Lodi A (2019) Exact Comb Optim Graph Convolutional Neural Networks. <https://arxiv.org/abs/1906.01629>
183. Khalil EB, Morris C, Lodi A (2022) Mip-gnn: a data-driven framework for guiding combinatorial solvers. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 36. AAAI, Vancouver, Canada, pp 10219–10227. <https://doi.org/10.1609/aaai.v36i9.21230>
184. Liu A, Luh PB, Sun K, Bragin MA, Yan B (2024) Integrating machine learning and mathematical optimization for job shop scheduling. *IEEE Trans Automat Sci Eng* 21(3):4829–4850. <https://doi.org/10.1109/TASE.2023.3303175>
185. Echeverria I, Murua M, Santana R (2025) Leveraging constraint programming in a deep learning approach for dynamically solving the flexible job-shop scheduling problem. *Expert Syst Appl* 265:125895. <https://doi.org/10.1016/j.eswa.2024.125895>
186. Xu B, Guo F, Xing L, Wang Y, Zhang W-A (2023) Accelerated and adaptive power scheduling for more electric aircraft via hybrid learning. *IEEE Trans Ind Electron* 70(1):833–845. <https://doi.org/10.1109/TIE.2022.3150107>
187. Shin W-J, Lee S-W, Lee J-H, Song M-H, Kim H-J (2025) Scheduling of steelmaking-continuous casting process by integrating deep neural networks with mixed integer programming. *Int J Prod Res* 63(11):4180–4201. <https://doi.org/10.1080/00207543.2024.2439369>
188. Juros J, Brcic M, Koncic M, Kovac M (2022) Exact solving scheduling problems accelerated by graph neural networks. In: 2022 45th Jubilee International Convention on Information, Communication and Electronic Technology (MIPRO), IEEE, Opatija, Croatia, pp 865–870. <https://doi.org/10.23919/MIPRO5190.2022.9803345>
189. Xin L, Song W, Cao Z, Zhang J (2021) Neurokh: combining deep learning model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. In: Advances in Neural Information Processing Systems, vol. 34. Curran Associates, Inc., Virtual, pp. 7472–7483. https://proceedings.neurips.cc/paper_files/paper/2021/file/3d863b367aa379f71c7afc0c9cdca41d-Paper.pdf
190. Li M, Tu S, Xu L (2023) Generalizing graph network models for the traveling salesman problem with lin-kernighan-helsgaun heuristics. In: Neural Information Processing - 30th International Conference, vol 14447. Springer, Changsha, China, pp 528–539. https://doi.org/10.1007/978-981-99-8079-6_41
191. Lyu Z, Islam MZ, Yu AJ (2024) A scalable and adaptable supervised learning approach for solving the traveling salesman problems. *IEEE Trans Intell Transp Syst* 25(11):17092–17104. <https://doi.org/10.1109/TITS.2024.3410691>
192. Jiang J, Gao J (2022) A sampling algorithm based on supervised learning for a dynamic traveling salesman problem. In: Proceedings of the Romanian Academy Series A - Mathematics Physics Technical Sciences Information Science. vol. 23, pp 79–88
193. Ye H, Xu H, Wang H, Wang C, Jiang Y (2023) Gnn and gbdtt guided fast optimizing framework for large-scale integer programming. In: International Conference on Machine Learning, vol 202. PMLR, Honolulu, Hawaii, USA, pp 39864–39878. <https://proceedings.mlr.press/v202/ye23e.html>
194. Khriiss A, Elmiad AK, Badaoui M, Mzili T, Alkhazaleh HA (2026) Garnet: integrating random walk encoding and graph rewiring for routing optimization. *Artif Intell Rev* 59(7). <https://doi.org/10.1007/s10462-026-11544-3>
195. Lee S, Lee J (2024) A graph attention network approach to partitioned scheduling in real-time systems. In: IEEE Embedded Systems Letters, vol 16. pp 457–460. <https://doi.org/10.1109/LES.2024.3376801>
196. Fu Z-H, Qiu K-B, Zha H (2021) Generalize a small pre-trained model to arbitrarily large tsp instances. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 35. AAAI, Vancouver, Canada, pp 7474–7482. <https://doi.org/10.1609/aaai.v35i8.16916>
197. Hudson B, Li Q, Malencia M, Prorok A (2022) Graph neural network guided local search for the traveling salesperson problem. In: International Conference on Learning Representations, Virtual. <https://openreview.net/forum?id=ar92oEosBlg>
198. Shang X (2023) A study of deep learning neural network algorithms and genetic algorithms for fjsp. *J Appl Math* 2023:1–13. <https://doi.org/10.1155/2023/4573352>
199. Li S, Ouyang W, Ma Y, Wu C (2025) Learning-guided rolling horizon optimization for long-horizon flexible job-shop scheduling. In: International Conference on Learning Representations, OpenReview.net, Singapore. <https://doi.org/10.48550/arXiv.2502.15791>
200. Boyan J, Moore AW (2001) Learning evaluation functions to improve optimization by local search. *J Mach Learn Res* 1:77–112. <https://doi.org/10.1162/15324430152733124>
201. Zheng J, He K, Zhou J, Jin Y, Li C-M (2023) Reinforced Lin-Kernighan-helsgaun algorithms for the traveling salesman problems. *Knowl-Based Syst* 260:110144. <https://doi.org/10.1016/j.knsys.2022.110144>
202. Ying K-C, Pourhejazy P, Cheng S-H (2024) Reinforcement learning-based alpha-list iterated greedy for production scheduling. *Intell Syst Appl* 24:200451. <https://doi.org/10.1016/j.iswa.2024.200451>
203. Bao H, Wang C, Pan Q, Wang B, Rong M, Yang A, Wang X (2024) A reinforcement learning driven iterated greedy algorithm for energy-efficiency flexible job-shop scheduling problem. In: 2024 4th International Conference on Computer, Control and Robotics (ICCCR), pp 374–379. <https://doi.org/10.1109/ICCCR61138.2024.10585347>; <https://ieeexplore.ieee.org/document/10585347>
204. Wu T, Cui H, Guan T, Wang Y, Jin Y (2024) Reinforcens: reinforcement learning-based multi-start neighborhood search for solving the traveling thief problem. In: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence

- publisher, Jeju, Korea, pp 7038–7046. <https://doi.org/10.24963/ijcai.2024/778>
205. Hong J, Shen B, Pan A (2024) A reinforcement learning-based neighborhood search operator for multi-modal optimization and its applications. *Expert Syst Appl* 246:123150. <https://doi.org/10.1016/j.eswa.2024.123150>
 206. Liu Z, Sang H, Zhang B, Meng L, Meng T (2025) A learning-based two-stage multi-thread iterated greedy algorithm for co-scheduling of distributed factories and automated guided vehicles with sequence-dependent setup times. *IEEE Trans Emerg Top Comput Intell* 9(3):2208–2218. <https://doi.org/10.1109/TETCI.2025.3540405>
 207. Chen R, Yang B, Li S, Wang S (2020) A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Comput Ind Eng* 149:106778. <https://doi.org/10.1016/j.cie.2020.106778>
 208. Li Z, Wei X, Jiang X, Pang Y (2021) A kind of reinforcement learning to improve genetic algorithm for multiagent task scheduling. *Math Probl Eng* 2021(1):1–12. <https://doi.org/10.1155/2021/1796296>
 209. Chen X, Li J, Xu Y (2023) Q-learning based multi-objective immune algorithm for fuzzy flexible job shop scheduling problem considering dynamic disruptions. *Swarm Evol Comput* 83:101414. <https://doi.org/10.1016/j.swevo.2023.101414>
 210. Long X, Zhang J, Zhou K, Jin T (2022) Dynamic self-learning artificial bee colony optimization algorithm for flexible job-shop scheduling problem with job insertion. *Processes* 10(3):3571. <https://doi.org/10.3390/pr10030571>
 211. Li Y, Liao C, Wang L, Xiao Y, Cao Y, Guo S (2023) A reinforcement learning-artificial bee colony algorithm for flexible job-shop scheduling problem with lot streaming. *Appl Soft Comput* 146:110658. <https://doi.org/10.1016/j.asoc.2023.110658>
 212. Pan Z, Wang L, Zheng J, Chen J, Wang X (2023) A learning-based multipopulation evolutionary optimization for flexible job shop scheduling problem with finite transportation resources. *IEEE Trans Evol Comput* 27(6):1590–1603. <https://doi.org/10.1109/TEVC.2022.3219238>
 213. Lin C, Cao Z, Zhou M (2023) Learning-based cuckoo search algorithm to schedule a flexible job shop with sequencing flexibility. *IEEE Trans Cybern* 53(10):6663–6675. <https://doi.org/10.1109/TCYB.2022.3210228>
 214. Li R, Gong W, Lu C (2022) A reinforcement learning based rmoea/d for bi-objective fuzzy flexible job shop scheduling. *Expert Syst Appl* 203:117380. <https://doi.org/10.1016/j.eswa.2022.117380>
 215. Zhao F, Zhang L, Cao J, Tang J (2021) A cooperative water wave optimization algorithm with reinforcement learning for the distributed assembly no-idle flowshop scheduling problem. *Comput Ind Eng* 153:107082. <https://doi.org/10.1016/j.cie.2020.107082>
 216. Choong SS, Wong L-P, Lim CP (2018) Automatic design of hyper-heuristic based on reinforcement learning. *Inf Sci* 436:437–89–107. <https://doi.org/10.1016/j.ins.2018.01.005>
 217. Ozsoydan FB, Gölcük İ (2023) A reinforcement learning based computational intelligence approach for binary optimization problems: the case of the set-union knapsack problem. *Eng Appl Artif Intel* 118:105688. <https://doi.org/10.1016/j.engappai.2022.105688>
 218. Mosadegh H, Fatemi Ghomi SMT, Süer GA (2020) Stochastic mixed-model assembly line sequencing problem: mathematical modeling and q-learning based simulated annealing hyper-heuristics. *Eur J Oper Res* 282(2):530–544. <https://doi.org/10.1016/j.ejor.2019.09.021>
 219. Zhang Z-Q, Qian B, Hu R, Yang J-B (2023) Q-learning-based hyper-heuristic evolutionary algorithm for the distributed assembly blocking flowshop scheduling problem. *Appl Soft Comput* 146:110695. <https://doi.org/10.1016/j.asoc.2023.110695>
 220. Liu J, Zhang Z, Liu S, Zhang Y, Wu T (2023) Parallel hyper heuristic algorithm based on reinforcement learning for the corridor allocation problem and parallel row ordering problem. *Adv Eng Inf* 56:101977. <https://doi.org/10.1016/j.aei.2023.101977>
 221. Ji J-J, Guo Y-N, Gao X-Z, Gong D-W, Wang Y-P (2023) Q-learning-based hyperheuristic evolutionary algorithm for dynamic task allocation of crowdsensing. *IEEE Trans Cybern* 53(4):2211–2224. <https://doi.org/10.1109/TCYB.2021.3112675>
 222. Zhang Z-Q, Wu F-C, Qian B, Hu R, Wang L, Jin H-P (2023) A q-learning-based hyper-heuristic evolutionary algorithm for the distributed flexible job-shop scheduling problem with crane transportation. *Expert Syst Appl* 234:121050. <https://doi.org/10.1016/j.eswa.2023.121050>
 223. Zhao F, Di S, Wang L (2023) A hyperheuristic with q-learning for the multiobjective energy-efficient distributed blocking flow shop scheduling problem. *IEEE Trans Cybern* 53(5):3337–3350. <https://doi.org/10.1109/TCYB.2022.3192112>
 224. Naimi R, Nouiri M, Cardin O (2021) A q-learning rescheduling approach to the flexible job shop problem combining energy and productivity objectives. *Sustainability* 13(23):13016. <https://doi.org/10.3390/su132313016>
 225. Mahmud S, A A, Chakraborty RK, Ryan MJ (2022) A self-adaptive hyper-heuristic based multi-objective optimisation approach for integrated supply chain scheduling problems. *Knowl-Based Syst* 251:109190. <https://doi.org/10.1016/j.knosys.2022.109190>
 226. Lin J, Li Y, Song H (2022) Semiconductor final testing scheduling using q-learning based hyper-heuristic. *Expert Syst Appl* 187:115978. <https://doi.org/10.1016/j.eswa.2021.115978>
 227. Ma Q, Ge S, He D, Thaker D, Drori I (2019) Combinatorial optimization by graph Pointer networks and hierarchical reinforcement learning. <https://arxiv.org/abs/1911.04936>
 228. Stohy A, Abdelhakam H-T, Ali S, Elhenawy M, Hassan AA, Masoud M, Glaser S, Rakotonirainy A (2021) Hybrid pointer networks for traveling salesman problems optimization. *PLoS One* 16(12):0260995. <https://doi.org/10.1371/journal.pone.0260995>
 229. Lv L, Fan J, Zhang C, Shen W (2025) Schedule repair for flexible job shops under machine breakdowns by deep reinforcement learning. *Comput Ind Eng* 207:111256. <https://doi.org/10.1016/j.cie.2025.111256>
 230. Li L, Fu X, Zhen H-L, Yuan M, Wang J, Lu J, Tong X, Zeng J, Schnieders D (2022) Bilevel learning for large-scale flexible flow shop scheduling. *Comput Ind Eng* 168:108140. <https://doi.org/10.1016/j.cie.2022.108140>
 231. Barrett TD, Clements WR, Foerster JN, Lvovsky AI (2020) Exploratory combinatorial optimization with reinforcement learning. In: *Proceedings of Thirty-fourth AAAI Conference on Artificial Intelligence, AAAI, New York, USA*, pp 3243–3250. <https://cdn.aaai.org/ojs/5723/5723-13-8948-1-10-20200513.pdf>
 232. Lischka A, Rydin F, Wu J, Chehreghani MH, Kulcsár B (2025) A GREAT architecture for edge-based graph problems like TSP. <https://arxiv.org/abs/2408.16717>
 233. Zhang C, Song W, Cao Z, Zhang J, Tan PS, Xu C (2020) Learning to dispatch for job shop scheduling via deep reinforcement learning. In: *Advances in Neural Information Processing Systems*, vol 33, Curran Associates, Inc., Virtual, pp 1621–1632. https://proceedings.neurips.cc/paper_files/paper/2020/file/11958dfec29b6709f48a9ba0387a2431-Paper.pdf
 234. Park J, Chun J, Kim SH, Kim Y, Park J (2021) Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *Int J Prod Res* 59(11):3360–3377. <https://doi.org/10.1080/00207543.2020.1870013>
 235. Yuan E, Cheng S, Wang L, Song S, Wu F (2023) Solving job shop scheduling problems via deep reinforcement learning. *Appl Soft Comput* 143:110436. <https://doi.org/10.1016/j.asoc.2023.110436>

236. Hameed MSA, Schwung A (2023) Graph neural networks-based scheduler for production planning problems using reinforcement learning. *J Manuf Sys* 69:91–102. <https://doi.org/10.1016/j.jmsy.2023.06.005>
237. Zhang W, Zhao F, Yang C, Du C, Feng X, Zhang Y, Peng Z, Mei X (2024) A novel soft actor–critic framework with disjunctive graph embedding and autoencoder mechanism for job shop scheduling problems. *J Manuf Sys* 76:614–626. <https://doi.org/10.1016/j.jmsy.2024.08.015>
238. Lei K, Guo P, Zhao W, Wang Y, Qian L, Meng X, Tang L (2022) A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem. *Expert Syst Appl* 205:117796. <https://doi.org/10.1016/j.eswa.2022.117796>
239. Song W, Chen X, Li Q, Cao Z (2023) Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Trans Ind Inf* 19(2):1600–1610. <https://doi.org/10.1109/TII.2022.3189725>
240. Tang H, Dong J (2024) Solving flexible job-shop scheduling problem with heterogeneous graph neural network based on relation and deep reinforcement learning. *Machines* 12(8):584. <https://doi.org/10.3390/machines12080584>
241. Zhao Y, Chen Y, Wu J, Guo C (2024) Dual dynamic attention network for flexible job scheduling with reinforcement learning. In: 2024 International Joint Conference on Neural Networks (IJCNN), IEEE, Yokohama, Japan, pp 1–8. <https://doi.org/10.1109/IJCNN60899.2024.10649988>
242. Wang R, Wang G, Sun J, Deng F, Chen J (2024) Flexible job shop scheduling via dual attention network-based reinforcement learning. *IEEE Trans Neural Netw Learn Syst* 35(3):3091–3102. <https://doi.org/10.1109/TNNLS.2023.3306421>
243. Ma N, Yao X, Hu K (2026) Progressive curriculum learning with adaptive graph attention networks for flexible job shop scheduling problem. *Appl Soft Comput* 194:115016. <https://doi.org/10.1016/j.asoc.2026.115016>
244. Wan L, Fu L, Li C, Li K (2025) An effective multi-agent-based graph reinforcement learning method for solving flexible job shop scheduling problem. *Eng Appl Artif Intel* 139:109557. <https://doi.org/10.1016/j.engappai.2024.109557>
245. Cheng W, Zhang C, Meng L, Gao K, Zhang B, Sang H (2025) A cooperative agent deep reinforcement learning framework for solving flexible job shop scheduling problem with automated guided vehicles. *Expert Syst Appl* 287:128142. <https://doi.org/10.1016/j.eswa.2025.128142>
246. Wang Y, Wang R, Sun J, Deng F, Wang G, Chen J (2025) Attention enhanced reinforcement learning for flexible job shop scheduling with transportation constraints. *Expert Syst Appl* 282:127671. <https://doi.org/10.1016/j.eswa.2025.127671>
247. Yuan E, Wang L, Cheng S, Song S, Fan W, Li Y (2024) Solving flexible job shop scheduling problems via deep reinforcement learning. *Expert Syst Appl* 245:123019. <https://doi.org/10.1016/j.eswa.2023.123019>
248. Xin L, Song W, Cao Z, Zhang J (2021) Multi-decoder attention model with embedding glimpse for solving vehicle routing problems. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 35. AAAI, Vancouver, Canada. <https://doi.org/10.1609/aaai.v35i13.17430>
249. Luo F, Lin X, Liu F, Zhang Q, Wang Z (2023) Neural combinatorial optimization with heavy decoder: toward large scale generalization. In: Advances in Neural Information Processing Systems, vol 36. Curran Associates, Inc, New Orleans, pp 8845–8864. http://proceedings.neurips.cc/paper_files/paper/2023/file/1c10d0c087c14689628124bbe8fa69f6-Paper-Conference.pdf
250. Jin Y, Ding Y, Pan X, He K, Zhao L, Qin T, Song L, Bian J (2023) Pointerformer: deep reinforced multi-pointer transformer for the traveling salesman problem. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 37. AAAI, Washington DC, USA, pp 8132–8140. <https://doi.org/10.1609/aaai.v37i7.25982>
251. Ye H, Wang J, Liang H, Cao Z, Li Y, Li F (2024) Glop: learning global partition and local construction for solving large-scale routing problems in real-time. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 38. AAAI, Vancouver, Canada. <https://doi.org/10.1609/aaai.v38i18.30009>
252. Zhang K, Li M, Zhang Z, Lin X, He F (2020) Multi-vehicle routing problems with soft time windows: a multi-agent reinforcement learning approach. *Transp Research Part C: Emerging Technol* 121:102861. <https://doi.org/10.1016/j.trc.2020.102861>
253. Chen R, Li W, Yang H (2023) A deep reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for the job-shop scheduling problem. *IEEE Trans Ind Inf* 19(2):1322–1331. <https://doi.org/10.1109/TII.2022.3167380>
254. Song L, Li Y, Xu J (2023) Dynamic job-shop scheduling based on transformer and deep reinforcement learning. *Processes* 11(12):3434. <https://doi.org/10.3390/pr11123434>
255. Liu Z, Mao H, Sa G, Liu H, Tan J (2024) Dynamic job-shop scheduling using graph reinforcement learning with auxiliary strategy. *J Manuf Sys* 73:1–18. <https://doi.org/10.1016/j.jmsy.2024.01.002>
256. Zhao L, Shen W, Zhang C, Peng K (2022) An end-to-end deep reinforcement learning approach for job shop scheduling. In: 2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD), IEEE, Hangzhou, China, pp 841–846. <https://doi.org/10.1109/CSCWD54268.2022.9776116>
257. Gebreyesus G, Fellek G, Farid A, Fujimura S, Yoshie O (2023) Gated-attention model with reinforcement learning for solving dynamic job shop scheduling problem. *IEEE Trans Elec Engng* 18(6):932–944. <https://doi.org/10.1002/tee.23788>
258. Zhang W, Zhao F, Feng B, Mei X (2025) A novel reinforcement learning framework based on simplified graph transformer for large-scale fuzzy flexible job shop scheduling problem. *Eng Appl Artif Intel* 158:111295. <https://doi.org/10.1016/j.engappai.2025.111295>
259. Xu S, Li Y, Li Q (2024) A deep reinforcement learning method based on a transformer model for the flexible job shop scheduling problem. *Electronics* 13(18):3696. <https://doi.org/10.3390/electronics13183696>
260. Cao Z, Zhang C, Wu Y, Hou Y, Ge H (2026) Mamba-CrossAttention: solving flexible job shop scheduling via sequence modeling and reinforcement learning. *Swarm Evol Comput* 105:102390. <https://doi.org/10.1016/j.swevo.2026.102390>
261. Zhang W, Zhao F, Li Y, Du C, Feng X, Mei X (2024) A novel collaborative agent reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for flexible job shop scheduling problem. *J Manuf Sys* 74:329–345. <https://doi.org/10.1016/j.jmsy.2024.03.012>
262. Wan L, Cui X, Zhao H, Fu L, Li C (2024) A novel method for solving dynamic flexible job-shop scheduling problem via difformer and deep reinforcement learning. *Comput Ind Eng* 198:110688. <https://doi.org/10.1016/j.cie.2024.110688>
263. Zhang W, Zhao F, Feng B, Mei X (2025) A reinforcement learning control framework based on scalable graph transformer for large-scale fuzzy job shop scheduling problems. *IEEE Trans Neural Netw Learn Syst* 36(9):16521–16533. <https://doi.org/10.1109/TNNLS.2025.3569868>
264. Parsonson CWF, Laterre A, Barrett TD (2023) Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 37. AAAI, Washington DC, USA, pp 4061–4069. <https://doi.org/10.1609/aaai.v37i4.25521>

265. Etheve M, Ales Z, Bissuel C, Juan O, Kedad-Sidhoum S (2020) Reinforcement learning for variable selection in a branch and bound algorithm. In: *Integration of constraint programming, artificial intelligence, and operations research*. Springer, Vienna, Austria, pp 176–185. https://doi.org/10.1007/978-3-030-58942-4_12
266. Scavuzzo L, Chen FY, Chételat D, Gasse M, Lodi A, Yorke-Smith N, Aardal K (2022) Learning to branch with tree mdps. In: *Advances in Neural Information Processing Systems*, vol 35. Curran Associates, Inc, New Orleans, pp 18514–18526. https://proceedings.neurips.cc/paper_files/paper/2022/file/756d74cd58592849c904421e3b2ec7a4-Paper-Conference.pdf
267. Tang Y, Agrawal S, Faenza Y (2020) Reinforcement learning for integer programming: learning to cut. In: *Proceedings of the 37th International Conference on Machine Learning*. JMLR.org, Virtual. <https://arxiv.org/abs/1906.04859>
268. Wang Z, Li X, Wang J, Kuang Y, Yuan M, Zeng J, Zhang Y, Wu F (2023) Learning cut selection for mixed-integer linear programming via hierarchical sequence model. In: *The Eleventh International Conference on Learning Representations*, OpenReview.net, Kigali, Rwanda. <https://openreview.net/forum?id=Zob4P9bRNcK>
269. Wang J, Wang Z, Li X, Kuang Y, Shi Z, Zhu F, Yuan M, Zeng J, Zhang Y, Wu F (2024) Learning to cut via hierarchical sequence/set model for efficient mixed-integer programming. *IEEE Trans Pattern Anal Mach Intell* 46(12):9697–9713. <https://doi.org/10.1109/TPAMI.2024.3432716>
270. Ling H, Wang Z, Wang J (2024) Learning to stop cut generation for efficient mixed-integer linear programming. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, AAAI, Vancouver, Canada, pp 20759–20767. <https://doi.org/10.1609/aaai.v38i18.30064>
271. Song W, Gu N (2024) Learning presolver selection for mixed-integer linear programming. In: *Proceedings of the 2024 16th International Conference on Machine Learning and Computing (ICMLC)*, Association for Computing Machinery, Shenzhen, China, pp 635–641. <https://doi.org/10.1145/3651671.3651739>
272. Wang Z, Song W, Lei Y, Wang Y, Gu N (2024) Learning heuristics approaches in mixed integer linear programming. In: *International Conference on Computer and Communication Systems*, IEEE, Xi'an, China, pp 1319–1324. <https://doi.org/10.1109/ICC CS61882.2024.10603216>
273. Scavuzzo L, Aardal K, Lodi A, Yorke-Smith N (2024) Machine learning augmented branch and bound for mixed integer linear programming. *Math Program*. <https://doi.org/10.1007/s10107-024-02130-y>
274. Costa PR, Rhuggenaath J, Zhang Y, Akcay A, Kaymak U (2021) Learning 2-opt heuristics for routing problems via deep reinforcement learning. *SN Comput Sci* 2(5). <https://doi.org/10.1007/s42979-021-00779-2>
275. Ma Y, Li J, Cao Z, Song W, Zhang L, Chen Z, Tang J (2021) Learning to iteratively solve routing problems with dual-aspect collaborative transformer. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., Virtual. <https://openreview.net/forum?id=63pC59XOZLZ>
276. Sui J, Ding S, Xia B, Liu R, Bu D (2024) Neuralgls: learning to guide local search with graph convolutional network for the traveling salesman problem. *Neural Comput Appl* 36(17):9687–9706. <https://doi.org/10.1007/S00521-023-09042-6>
277. Ma Y, Cao Z, Chee YM (2023) Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. In: *Advances in Neural Information Processing Systems*, vol 36. Curran Associates, Inc, New Orleans, pp 49555–49578. https://proceedings.neurips.cc/paper_files/paper/2023/file/9bae70d354793a95fa18751888cea07d-Paper-Conference.pdf
278. Zheng J, He K, Zhou J, Jin Y, Li C-M (2021) Combining reinforcement learning with lin-kernighan-helsgaun algorithm for the traveling salesman problem. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol 35. AAAI, Virtual, pp 12445–12452. <https://doi.org/10.1609/aaai.v35i14.17476>
279. Min B, Kim CO (2022) State-dependent parameter tuning of the apparent tardiness cost dispatching rule using deep reinforcement learning. *IEEE Access* 10:20187–20198. <https://doi.org/10.1109/ACCESS.2022.3152192>
280. Lin C-C, Deng D-J, Chih Y-L, Chiu H-T (2019) Smart manufacturing scheduling with edge computing using multiclass deep q network. *IEEE Trans Ind Inf* 15(7):4276–4284. <https://doi.org/10.1109/TII.2019.2908210>
281. Dong X, Wan G, Zeng P (2025) A heuristic-assisted deep reinforcement learning algorithm for flexible job shop scheduling with transport constraints. *Complex Intell Syst* 11(5):210. <https://doi.org/10.1007/s40747-025-01828-6>
282. Zhang Y, Dong S, Wen T, Yuan Z, Xiao J, Diao Z (2026) Budget-scalable inference-time hybrid mcts for enhancing drl-based flexible job shop scheduling. *Comput Ind Eng* 214:111897. <https://doi.org/10.1016/j.cie.2026.111897>
283. Su J, Lyu S, Lu X, Wan Y, Li F (2022) Ridrl: a deep reinforcement learning based on multiple dispatching rules and iga algorithm for jsp. In: *International Computer Conference on Wavelet Active Media Technology and Information Processing*, IEEE, Chengdu, China, pp 1–4. <https://doi.org/10.1109/ICCWAMTIP56608.2022.10016480>
284. Ren F, Liu H (2024) Dynamic scheduling for flexible job shop based on machinerank algorithm and reinforcement learning. *Sci Rep* 14(1):29741. <https://doi.org/10.1038/s41598-024-79593-8>
285. Dong Z, Ren T, Weng J, Qi F, Wang X (2022) Minimizing the late work of the flow shop scheduling problem with a deep reinforcement learning based approach. *Appl Sci* 12(5):2366. <https://doi.org/10.3390/app12052366>
286. Ye H, Wang J, Cao Z, Liang H, Li Y (2023) Deepaco: neural-enhanced ant systems for combinatorial optimization. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc, New Orleans. <https://doi.org/10.52202/075280-1895>
287. Kim M, Choi S, Kim H, Son J, Park J, Bengio Y (2025) Ant colony sampling with gflownets for combinatorial optimization. In: *The 28th International Conference on Artificial Intelligence and Statistics*, PMLR, Mai Khao, Thailand. <https://openreview.net/forum?id=kYJSaG7Try>
288. Falkner JK, Thyssens D, Bdeir A, Schmidt-Thieme L (2022) Learning to control local search for combinatorial optimization. In: *Machine learning and knowledge discovery in databases*. Springer, Grenoble, France, pp 361–376. https://doi.org/10.1007/978-3-031-26419-1_22
289. Xu M, Mei Y, Zhang F, Zhang M (2026) Niching genetic programming to learn actions for deep reinforcement learning in dynamic flexible scheduling. *IEEE Trans Evol Comput* 30(1):61–75. <https://doi.org/10.1109/TEVC.2024.3395699>
290. Dong Z, Ren T, Qi F, Weng J, Bai D, Yang J, Wu C-C (2025) A reinforcement learning-based approach for solving multi-agent job shop scheduling problem. *Int J Prod Res* 63(10):3512–3537. <https://doi.org/10.1080/00207543.2024.2423807>
291. Chen C, Yin C, Wang R (2024) Dynamic job shop scheduling method based on ensemble reinforcement learning. In: *2024 6th International Conference on Internet of Things, Automation and Artificial Intelligence (IoTAAI)*, IEEE, Guangzhou, China, pp 431–435. <https://doi.org/10.1109/IoTAAI62601.2024.10692885>
292. Zheng J, Zhao Y, Li J, She W, Li Y (2025) Integrated scheduling of material delivery and processing. *Comput Ind Eng* 201:110863. <https://doi.org/10.1016/j.cie.2025.110863>
293. Huang Z, Chen W, Zhang W, Shi C, Liu F, Zhen H-L, Yuan M, Hao J, Yu Y, Wang J (2023) Branch ranking for efficient mixed-integer programming via offline ranking-based policy learning.

- In: Amini M-R, Canu S, Fischer A, Guns T, Novak PK, Tsoumakas G (eds) Machine learning and knowledge discovery in databases. Springer, Cham, pp 377–392.
294. Wang Z, Li X, Wang J, Kuang Y, Yuan M, Zeng J, Zhang Y, Wu F (2023) Learning cut selection for mixed-integer linear programming via hierarchical sequence model. <https://arxiv.org/abs/2302.00244>
 295. Zeng Y-J, Liao Z, Dai Y, Wang R, Li X, Yuan B (2022) Hybrid intelligence for dynamic job-shop scheduling with deep reinforcement learning and attention mechanism. <https://arxiv.org/abs/2201.00548>
 296. Graikos A, Malkin N, Jojic N, Samaras D (2022) Diffusion models as plug-and-play priors. *Adv Neural Inf Process Syst* 35:14715–14728
 297. Huang J, Sun Z, Yang Y (2023) Accelerating diffusion-based combinatorial optimization solvers by progressive distillation. <https://arxiv.org/abs/2308.06644>
 298. Xue T, Lu L, Liu Y, Wu M, Dong H, Zhang Y, Han R, Chen B (2024) Gfpack++: improving 2D irregular packing by learning gradient field with attention. <https://arxiv.org/abs/2406.07579>
 299. Sanokowski S, Hochreiter S, Lehner S (2024) A diffusion model framework for unsupervised neural combinatorial optimization. <https://arxiv.org/abs/2406.01661>
 300. Hong S-H, Kim H-S, Jang Z, Yoon D, Song H, Lee B-J (2024) Unsupervised training of diffusion models for feasible solution generation in neural combinatorial optimization. <https://arxiv.org/abs/2411.00003>
 301. Wang M, Zhou Y, Cao Z, Xiao Y, Wu X, Pang W, Jiang Y, Yang H, Zhao P, Li Y (2025) An efficient diffusion-based non-autoregressive solver for traveling salesman problem. <https://arxiv.org/abs/2501.13767>
 302. Wang Q, Hao Y, Zhang J (2023) Generative inverse reinforcement learning for learning 2-opt heuristics without extrinsic rewards in routing problems. *J Educ Chang King Saud Univ - Comput Inf Sci* 35(9):101787. <https://doi.org/10.1016/j.jksuci.2023.101787>
 303. Zhang N, Yang J, Cao Z, Chi X (2025) Adversarial generative flow network for solving vehicle routing problems. <https://arxiv.org/abs/2503.01931>
 304. Guo Y-N, Ji J, Tan Y, Cheng S (2020) Multi-objective combinatorial generative adversarial optimization and its application in crowdsensing. In: *International Conference on Swarm Intelligence*, Springer, Belgrade, Serbia, pp 423–434. https://doi.org/10.1007/978-3-030-53956-6_38
 305. Chen M, Yu R, Xu S, Luo Y, Yu Z (2019) An improved algorithm for solving scheduling problems by combining generative adversarial network with evolutionary algorithms. In: *Proceedings of the 3rd International Conference on Computer Science and Application Engineering*, Sanya, China, pp 1–7. <https://doi.org/10.1145/3331453.3361639>
 306. Yu R, Wen H, Xu Y (2022) Evolutionary discrete optimization inspired by zero-sum game theory. In: *International Conference on Mobility, Sensing and Networking*, IEEE, Guangzhou, China, pp 984–989. <https://doi.org/10.1109/MSN57253.2022.00159>
 307. Zhang B, Yao Y, Kan H, Luo W (2024) A gan-based genetic algorithm for solving the 3d bin packing problem. *Sci Rep* 14(1):7775. <https://doi.org/10.1038/s41598-024-56699-7>
 308. Bentley P, Lim SL, Arcaini P, Ishikawa F (2023) Using a variational autoencoder to learn valid search spaces of safely monitored autonomous robots for last-mile delivery. In: *Proceedings of the Genetic and Evolutionary Computation Conference*, Lisbon, Portugal, pp 1303–1311. <https://doi.org/10.1145/3583131.359045>
 309. Onsu MA, Lohan P, Kantarci B, Janulewicz E (2025) Genai assistance for deep reinforcement learning-based vnf placement and sfc provisioning in 5g cores. In: *IEEE International Conference on Communications*, IEEE, Montreal, Canada, pp 482–487. <https://doi.org/10.1109/ICC52391.2025.11161524>
 310. Chen S, Guo Z, Zhou B, Liu Y, Zang T, Luo H (2023) Coordinated optimization scheduling for active distribution networks considering schedulable capacity of energy storage for 5g base stations. *Power System Technol* 47(12):5225–5241
 311. Bhattacharjee S, Gras R (2019) Estimation of distribution using population queue based variational autoencoders. In: *IEEE Congress on Evolutionary Computation*, IEEE, Wellington, New Zealand, pp 1406–1414. <https://doi.org/10.1109/CEC.2019.8790077>
 312. Wang E, He J, Liu B, Shi H, Han W, Wu L (2024) Task offloading and resource allocation for icvs in cfns via constrained marl. In: *IEEE International Symposium on Product Compliance Engineering-Asia*, IEEE, Wuhan, China, pp 1–6. <https://doi.org/10.1109/ISPCE-ASIA64773.2024.10756255>
 313. Yao Z, Chen S, Yang J, Liu Y, Xie H (2019) Mobility-aware pre-caching based on unsupervised deep generative model for small cell networks. In: *2019 13th International Conference on Signal Processing and Communication Systems (ICSPCS)*, IEEE, Gold Coast, Australia, pp 1–7. <https://doi.org/10.1109/ICSPCS47537.2019.9008496>
 314. Nouri S, Motalleb MK, Shah-Mansouri V, Shariatpanahi SP (2024) Semi-supervised learning approach for efficient resource allocation with network slicing in O-RAN. <https://arxiv.org/abs/2401.08861>
 315. Li M, Gao J, Zhou C, Zhao L, Shen X (2024) Digital-twin-empowered resource allocation for on-demand collaborative sensing. *IEEE Internet Things J* 11(23):37942–37958. <https://doi.org/10.1109/IJOT.2024.3446801>
 316. Shanmugham K, Rangan R, Dhatchnamurthy S, Pundir S (2024) An efficient self-attention-based conditional variational auto-encoder generative adversarial networks based multipath cross-layer design routing paradigm for manet. *Expert Syst Appl* 238:122097. <https://doi.org/10.1016/j.eswa.2023.122097>
 317. Yao W, Bandeira AS, Villar S (2019) Experimental performance of graph neural networks on random instances of max-cut. In: *Wavelets and Sparsity XVIII*, vol 11138. SPIE, San Diego, California, United States, pp 242–251. <https://doi.org/10.1117/12.2529608>
 318. Karalias N, Loukas A (2020) Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In: *Advances in Neural Information Processing Systems*, vol 33. Curran Associates, Inc., Virtual, pp 6659–6672. https://proceedings.neurips.cc/paper_files/paper/2020/file/49f85a9ed090b20c8bed85a5923c669f-Paper.pdf
 319. Schuetz MJ, Brubaker JK, Katzgraber HG (2022) Combinatorial optimization with physics-inspired graph neural networks. *Nat Mach Intell* 4(4):367–377. <https://doi.org/10.1038/s42256-022-00468-6>
 320. Tönshoff J, Ritzert M, Wolf H, Grohe M (2021) Graph neural networks for maximum constraint satisfaction. *Front Artif Intell* 3:580607. <https://doi.org/10.3389/frai.2020.580607>
 321. Boisvert L, Verhaeghe H, Cappart Q (2024) Towards a generic representation of combinatorial problems for learning-based approaches. In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, Springer, Uppsala, Sweden, pp 99–108. https://doi.org/10.1007/978-3-031-60597-0_7
 322. Bai S, Yang C, Bai X, Zhang X, Jiang Z (2025) Deep k-grouping: an unsupervised learning framework for combinatorial optimization on graphs and Hypergraphs. <https://arxiv.org/abs/2505.20972>
 323. Liao Y, Koushanfar F, Naghizadeh P (2025) Learning for dynamic combinatorial optimization without training data. <https://arxiv.org/abs/2505.19497>
 324. Min Y, Gomes CP (2025) Unsupervised learning for quadratic assignment. <https://arxiv.org/abs/2503.20001>

325. Hong L, Qing F, Liu Y, Liu K (2026) An unsupervised-learning-based method for cost-constrained complex network dismantling. *Physica A* 693:131564. <https://doi.org/10.1016/j.physa.2026.131564>
326. Huang Y, Zhang W, Feng L, Wu X, Tan KC (2025) How multi-modal integration boost the performance of llm for optimization: case study on capacitated vehicle routing problems. In: Symposium for Multidisciplinary Computational Intelligence Incubators, IEEE, Trondheim, Norway, pp 1–7. <https://doi.org/10.1109/MCII64973.2025.11032980>
327. Elhenawy M, Abdelhay A, I Ashqar TIA, Aradat J, Jaber A, Glaser S, Rakotonirainy A (2024) Eyeballing combinatorial problems: a case study of using multimodal large language models to solve traveling salesman problems. In: International Conference on Intelligent Systems, Blockchain, and Communication Technologies, Springer, Egypt, pp 341–355. https://doi.org/10.1007/978-3-031-82377-0_29
328. Abgaryan H, Harutyunyan A, Cazenave T (2024) Llms can schedule. <https://arxiv.org/abs/2408.06993>
329. Guo F, Liang K, Zheng G, Chu X, Zhou G, Wong K-K, Chae C-B (2026) Llm-based port selection and beamforming for multi-user mimo with fluid antenna systems. *IEEE Trans Netw Sci Eng* 13:4026–4042. <https://doi.org/10.1109/TNSE.2025.3626002>
330. Liu F, Xialiang T, Yuan M, Lin X, Luo F, Wang Z, Lu Z, Zhang Q (2024) Evolution of heuristics: towards efficient automatic algorithm design using large language model. In: International Conference on Machine Learning, Vienna, Austria. <https://openreview.net/forum?id=BwAkaxqiLB>
331. Shi Y, Zhou J, Song W, Bi J, Wu Y, Zhang J (2025) Generalizable heuristic generation through large language models with meta-optimization. In: Workshop on Differentiable Learning of Combinatorial Algorithms, San Diego, United States. <https://openreview.net/forum?id=1yrYnaQSzi>
332. Ye H, Wang J, Cao Z, Berto F, Hua C, Kim H, Park J, Song G (2024) Reevo: large language models as hyper-heuristics with reflective evolution. In: Advances in Neural Information Processing Systems, vol 37. Curran Associates, Inc, Vancouver, Canada, pp 43571–43608. <https://openreview.net/forum?id=483IPG0HWL>
333. Zheng Z, Xie Z, Wang Z, Hooi B (2025) Monte carlo tree search for comprehensive exploration in llm-based automatic heuristic design. In: International Conference on Machine Learning, vol 267. PMLR, Vancouver, Canada, pp 78338–78373. <https://openreview.net/forum?id=Do1OdZzYHr>
334. Dat PVT, Doan L, Binh HTT (2025) Hsevo: elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using llms. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 39. AAAI, Pennsylvania, USA, pp 26931–26938. <https://doi.org/10.1609/aaai.v39i25.34898>
335. Huang J, Li X, Gao L, Liu Q, Teng Y (2024) Automatic programming via large language models with population self-evolution for dynamic job shop scheduling problem. <https://arxiv.org/abs/2410.22657>
336. Jiang X, Wu Y, Zhang C, Zhang Y (2025) Droc: elevating large language models for complex vehicle routing via decomposed retrieval of constraints. In: International Conference on Learning Representations. OpenReview, net, Singapore. <https://openreview.net/forum?id=s9zoyICZ4k>
337. Chen Z, Zhou Z, Lu Y, Xu R, Pan L, Lan Z (2024) Qube: enhancing automatic heuristic design via quality-uncertainty balanced evolution. <https://arxiv.org/abs/2412.20694>
338. Stein N, Kononova AV, Kotthoff L, Bäck T (2025) Code evolution graphs: understanding large language model driven design of algorithms. In: Proceedings of the Genetic and Evolutionary Computation Conference, ACM, Malaga, Spain, pp 943–951. <https://doi.org/10.1145/3712256.3726328>
339. Surina A, Mansouri A, Quaedvlieg L, Seddas A, Viazovska M, Abbe E, Gulcehre C (2025) Algorithm discovery with llms: evolutionary search meets reinforcement learning. <https://arxiv.org/abs/2504.05108>
340. Tran CD, Nguyen-Tri Q, Binh HTT, Thanh-Tung H (2025) Large language models powered neural solvers for generalized vehicle routing problems. In: International Conference on Learning Representations. OpenReview, net, Singapore. <https://openreview.net/forum?id=EVqlVjvlt8>
341. Ramamonjison R, Li H, Yu T, He S, Rengan V, Banitalebi-Dehkordi A, Zhou Z, Zhang Y (2022) Augmenting operations research with auto-formulation of optimization models from problem descriptions. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp 29–62. <https://doi.org/10.18653/v1/2022.emnlp-industry.4>
342. Li Q, Zhang L, Mak-Hau V (2023) Synthesizing mixed-integer linear programming models from natural language descriptions. <https://arxiv.org/abs/2311.15271>
343. Obata K, Aoki T, Horii T, Taniguchi T, Nagai T (2025) Lip-llm: integrating linear programming and dependency graph with large language models for multi-robot task planning. *IEEE Robot Autom Lett* 10(2):1122–1129. <https://doi.org/10.1109/lra.2024.3518105>
344. Ahmed T, Choudhury S (2024) Lm4opt: unveiling the potential of large language models in formulating mathematical optimization problems. *INFOR Inf Syst Oper Res* 62(4):559–572. <https://doi.org/10.1080/03155986.2024.2388452>
345. Jang S (2022) Tag embedding and well-defined intermediate representation improve auto-formulation of problem description. <https://arxiv.org/abs/2212.03575>
346. He J, Vignesh S, Kumar D, Uppal A et al. (2022) Linear programming word problems formulation using ensemblecrf ner labeler and t5 text generator with data augmentations. <https://arxiv.org/abs/2212.14657>
347. Ning Y, Liu J, Qin L, Xiao T, Xue S, Huang Z, Liu Q, Chen E, Wu J (2023) A novel approach for auto-formulation of optimization problems. <https://arxiv.org/abs/2302.04643>
348. Dhanaraj N, Jeon M, Kang JH, Nikolaidis S, Gupta SK (2024) Preference elicitation and incorporation for human-robot task scheduling. In: International Conference on Automation Science and Engineering, IEEE, Bari, Italy, pp 3103–3110. <https://doi.org/10.1109/CASE59546.2024.10711695>
349. Chen H, Constante-Flores GE, Li C (2024) Diagnosing infeasible optimization problems using large language models. *INFOR Inf Syst Oper Res* 62(4):573–587. <https://doi.org/10.1080/03155986.2024.2385189>
350. Xiao Z, Zhang D, Wu Y, Xu L, Wang YJ, Han X, Fu X, Zhong T, Zeng J, Song M, Chen G (2024) Chain-of-experts: when LLMs meet complex operations research problems. In: International Conference on Learning Representations, OpenReview.net, Vienna, Austria. <https://openreview.net/forum?id=HobyL1B9CZ>
351. Mostajabdaveh M, Yu TT, Ramamonjison R, Carenini G, Zhou Z, Zhang Y (2024) Optimization modeling and verification from problem specifications using a multi-agent multi-stage llm framework. *INFOR Inf Syst Oper Res* 62(4):599–617. <https://doi.org/10.1080/03155986.2024.2381306>
352. Huang Z, Shi G, Sukhatme GS (2024) From words to routes: applying large language models to vehicle routing. <https://arxiv.org/abs/2403.10795>
353. Hao Y, Zhang Y, Fan C (2024) Planning anything with rigor: general-purpose zero-shot planning with llm-based formalized programming. <https://arxiv.org/abs/2403.10795>
354. Li B, Zhang K, Sun Y, Zou J (2024) Research on travel route planning optimization based on large language model. In: International Conference on Data-driven Optimization of Complex

- Systems, IEEE, Hangzhou, China, pp 352–357. <https://doi.org/10.1109/DOCS63458.2024.10704489>
355. Zhang J, Wang W, Guo S, Wang L, Lin F, Yang C, Yin W (2024) Solving general natural-language-description optimization problems with large language models. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol 6. Association for Computational Linguistics, Mexico City, Mexico, pp 483–490. <https://doi.org/10.18653/v1/2024.naacl-industry.42>
 356. Jiang C, Shu X, Qian H, Lu X, Zhou J, Zhou A, Yu Y (2025) LLMOPT: learning to define and solve general optimization problems from scratch. In: International Conference on Learning Representations, OpenReview.net, Singapore. <https://openreview.net/forum?id=9OMvtboTJg>
 357. Peng M, Chen Z, Yang J, Huang J, Shi Z, Liu Q, Li X, Gao L (2025) Automatic milp model construction for multi-robot task allocation and scheduling based on large language models. In: International Conference on Intelligent Robots and Systems, Hangzhou, China, pp 20291–20296. <https://doi.org/10.1109/IROS60139.2025.11246517>
 358. Ju D, Jiang S, Cohen A, Foss A, Mitts S, Zharmagambetov A, Amos B, Li X, Kao JT, Fazel-Zarandi M, Tian Y (2024) To the globe (TTG): towards language-driven guaranteed travel planning. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Miami, USA, pp 240–249. <https://doi.org/10.18653/v1/2024.emnlp-demo.25>
 359. Yang Z, Wang Y, Huang Y, Guo Z, Shi W, Han X, Feng L, Song L, Liang X, Tang J (2025) Optibench meets resocratic: measure and improve LLMs for optimization modeling. In: International Conference on Learning Representations, OpenReview.net, Singapore. <https://openreview.net/forum?id=fsDZwS49uY>
 360. Huang Z, Shi G, Sukhatme GS (2024) Can large language models solve robot routing? <https://arxiv.org/abs/2403.10795>
 361. Sun W, Feng S, Li S, Yang Y (2025) Co-bench: benchmarking language model agents in algorithm search for combinatorial optimization. <https://arxiv.org/abs/2504.04310>
 362. Ouyang W, Wang Y, Weng P, Han S (2024) Generalization in deep rl for tsp problems via equivariance and local search. *SN Comput Sci* 5(4):369. <https://doi.org/10.1007/s42979-024-02689-5>
 363. Li Y, Guo J, Wang R, Yan J (2023) T2t: from distribution learning in training to gradient search in testing for combinatorial optimization 36:50020–50040
 364. Rockafellar RT, Uryasev S et al. (2000) Optimization of conditional value-at-risk. *JOR* 2(3):21–41. <https://doi.org/10.21314/JOR.2000.038>
 365. Rahimian H, Mehrotra S (2019) Co-bench: benchmarking language model agents in algorithm search for combinatorial optimization. <https://arxiv.org/abs/1908.05659>
 366. Lin B, Ghaddar B, Nathwani J (2022) Deep reinforcement learning for the electric vehicle routing problem with time windows. *IEEE Trans Intell Transp Syst* 23(8):11528–11538. <https://doi.org/10.1109/TITS.2021.3105232>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.